

Python Foundations

PARTICIPANT GUIDE



The material contained in this document, including all attachments, is copyright © 2025 DDLs Australia Pty Ltd, trading as Lumify Group. No part may be reproduced, modified, transmitted, used, distributed, or stored in a retrieval system, in any form or by any means (electronic, mechanical, photocopy, recording or otherwise) for any purpose other than the purpose for which it is provided, without the prior written consent of Lumify Group.

Table of Contents

1. Introduction to Python & Development Environment.....	6
What is Python and where it is used?.....	6
Installing Python and development tools	11
Running Python scripts and interactive mode	17
Understanding Python syntax and structure	22
Writing your first Python program	28
2. Variables, Data Types, and Operators	33
Variables and naming conventions	33
Built-in data types (int, float, string, bool)	39
Type conversion	45
Operators (arithmetic, comparison, logical)	50
Working with strings	56
3. Control Flow – Conditionals and Loops.....	62
Conditional statements (if, elif, else).....	62
Nested conditions	67
Looping constructs (for, while).....	73
Break and continue statements	78
Practical decision-making logic	83
4. Program Design with Pseudocode and Flowcharts.....	90
Problem-solving strategies.....	90
Breaking problems into steps.....	95
Writing pseudocode	100
Designing flowcharts.....	106
Translating design into Python code	111
5. Functions and Modular Programming	117
Defining and calling functions.....	117
Parameters and return values.....	121
Scope of variables	126
Reusability and modular design.....	131
6. Data Structures	138

Lists (indexing, slicing, methods)	138
Tuples and when to use them	143
Sets and uniqueness	148
Dictionaries (key-value pairs)	153
Iterating through collections	160
7. File Handling and Regular Expressions	166
Reading and writing text files	166
Working with CSV-style data	173
File paths and error handling	179
Introduction to regular expressions	185
Pattern matching and data filtering	191
8. User Input and Error Handling	197
Accepting user input	197
Input validation	202
Exception handling (try/except)	209
Writing robust programs	214
9. Using Python Libraries	220
Importing modules and packages	220
Standard library overview	225
Working with requests (web calls)	231
Working with datetime	237
Working with json	243
10. Debugging and Testing	250
Common errors (syntax, runtime, logical)	250
Debugging techniques	257
Using print vs debugger	263
Writing simple test cases	268
11. Introduction to Object-Oriented Programming	274
Classes and objects	274
Attributes and methods	279
Constructors (__init__)	284
Basic inheritance	289



When to use OOP.....	294
----------------------	-----

Course Overview

This three-day, instructor-led course provides a practical introduction to Python programming.

- Participants will learn how to design, write, and debug Python programs
- Covers variables, control flow, functions, and data structures
- Emphasises hands-on labs and real-world examples
- Focuses on problem-solving techniques and building applications

Target Audience

- Beginners with no prior programming experience
- IT professionals transitioning into development or automation
- Data analysts and engineers needing foundational Python skills
- Students preparing for further study in software development or AI

Prerequisites

- Basic computer literacy
- Familiarity with file systems and simple command-line usage
- No prior programming experience required

Learning Outcomes

- Understand Python syntax, data types, and control flow
- Design solutions using pseudocode and flowcharts
- Write programs using variables, operators, and expressions
- Implement conditional logic and loops
- Create reusable code using functions
- Work with lists, tuples, sets, and dictionaries
- Read and write files, and filter data using regular expressions
- Handle user input and perform basic data processing
- Use Python libraries for tasks such as web requests and data handling
- Apply debugging techniques to identify and fix errors
- Understand core object-oriented programming concepts

1. Introduction to Python & Development Environment

WHAT IS PYTHON AND WHERE IT IS USED?

Python is a versatile, high-level programming language used for web development, data analysis, automation, artificial intelligence, and more.

Python is a popular programming language known for its simplicity and readability. Many people choose Python because it is easy to learn for beginners, but powerful enough for experienced developers. Python is used in a wide range of fields, making it one of the most versatile languages in use today.

When you hear that Python is **versatile**, it means the language can be used for many types of projects. Python is not limited to just one area of computing. Its clear structure and large standard library help you quickly build programs for different purposes.

Python is a high-level language. This means you write instructions that are closer to human language, rather than machine code. As a result, Python hides many of the complex details, which helps you focus on solving problems rather than worrying about how the computer operates under the hood.

- **Web development:** Building websites and web applications with frameworks like Django and Flask.
- **Data analysis and visualisation:** Processing data, creating graphs, and running statistics with libraries like pandas and matplotlib.
- **Automation and scripting:** Writing scripts to automate repetitive tasks across your computer or the web.
- **Artificial intelligence (AI) and machine learning:** Developing smart algorithms with tools like TensorFlow and scikit-learn.
- **Software development:** Creating stand-alone desktop applications or supporting code for other software.
- **Internet of Things (IoT):** Controlling devices and sensors.
- **Education:** Used as a first programming language in schools and universities.

```
# A simple Python script to print a greeting
def greet(name):
    print(f"Hello, {name}!")
greet("Rachel")
```

Area	Example Use
Web Development	Building websites with Django
Data Analysis	Processing survey results with pandas
Automation	Renaming files in a folder automatically
AI & Machine Learning	Training a model to recognise images
Education	Teaching programming basics to students

Note: Important tip: Python's large online community means you will find plenty of resources, guides, and free tools to support your learning.

It is known for its readability, simplicity, and wide community support.

Python is highly regarded for being easy to read and understand, making it a popular choice for new and experienced programmers alike. Its straightforward syntax closely matches everyday English, which helps reduce the learning curve and allows you to focus on problem-solving rather than battling complex code rules.

Python's **simplicity** lies in its ability to let you express concepts in fewer lines of code than many other programming languages. This means you can create powerful programs without needing to memorise a lot of complex rules. Simpler code is also less likely to contain errors, easier to maintain, and faster to update when requirements change.

Readability is a key design goal of Python. The language uses indentation (spacing at the start of lines) to structure code, rather than curly braces or symbols used in other languages. This forces you, and everyone else, to format code in a consistent way, making it easier to follow and understand.

- **Plain and clear syntax** that reads almost like English
- **Uses indentation** for code structure, not confusing symbols
- **Code examples** are often short and easy to follow
- **Encourages good programming** habits from the very beginning

```
print("Hello, world!")

for i in range(3):
    print(f"Counting: {i}")
```


Feature	Benefit
Readability	Makes code easy for anyone to understand and review
Simplicity	Lets you do more with fewer lines of code
Community support	Access to help, libraries, and guides from users all over the world

Another reason for Python's popularity is its vibrant and active community. No matter your problem or question, there's a good chance someone has already encountered it and shared a solution online. This means you will find many tutorials, forums, and helpful guides. The large community also creates and maintains thousands of reusable libraries for almost any task, from data analysis to web development and automation.

Note: Important tip: If you ever get stuck while learning Python, try searching online or visiting Python community forums. Chances are, you'll find clear answers because so many people use the language and are happy to help.

Python can be used by beginners and professionals across various industries.

Python is a flexible, high-level programming language. It is well suited for both beginners and experienced professionals. Its clear syntax, powerful libraries, and supportive community make it a popular choice around the world.

You do not need previous coding experience to start learning Python. Its design emphasises readability and simplicity. **Professionals use Python too**, thanks to its ability to handle complex tasks in data science, automation, web development, and more.

Let us look at why Python is good for both new learners and skilled developers. Its enjoyable learning curve allows beginners to focus on solving real problems instead of memorising strict grammar. At the same time, advanced features like object-oriented programming, large standard libraries, and integration with other technologies offer professionals the power they need.

- **Wide range of resources:** There are many tutorials, guides, and discussion forums for all skill levels.
- **Large, active community:** Someone else has probably faced the same problem as you.
- **Lots of external libraries:** Professionals can use ready-made tools for tasks like scientific
- **Easy to read and write:** Python code looks like plain English.
- **Computing, analysis, automation,** and networking.
- **Cross-industry use:** Python is used in science, finance, healthcare, web development, education, and more.

```
# Example: A simple Python script for everyone
print('Hello, world!')

# Example: Reading a CSV file (for professionals)
import csv
with open('data.csv') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Industry	Python Usage
Education	Teaching programming, learning automation basics
Data Science	Analysing data, statistical modelling, visualisation
Web Development	Building websites and APIs (e.g., using Django, Flask)
Finance	Risk calculation, financial analysis, automated trading
Healthcare	Medical research, bioinformatics, managing patient data
Robotics	Automating tasks, controlling physical devices

Note: Important tip: Python is an excellent language to start your programming journey, but it will also keep up with your needs as you tackle more advanced projects in your career.

Popular applications of Python include building websites, automating repetitive tasks, analysing large datasets, and creating machine learning models.

Python is a widely used programming language known for its flexibility and readability. People from many industries use Python for a range of tasks. Understanding its most popular applications will help you decide how to use Python in your own projects.

Python can be used almost anywhere software runs. It is **commonly used to build websites, automate repetitive tasks, analyse large datasets, and create machine learning models**. Each of these areas plays an important role in modern technology.

Let us break down these applications to see how Python is used in each.

- **Building websites:** Python can be used to create web applications, from simple blogs to large e-commerce sites.
- **Automating repetitive tasks:** Python scripts can handle tasks like renaming files, scraping websites, or processing emails automatically.

- **Analysing large datasets:** Scientists and businesses use Python to make sense of thousands or millions of pieces of data.
- **Creating machine learning models:** Python provides tools for teaching computers to recognise patterns and make decisions.

```
# Example: Automation with Python
import os

for filename in os.listdir('.'):
    if filename.endswith('.txt'):
        print('Found text file:', filename)
```

Application Area	Popular Python Tools/Frameworks
Web Development	Django, Flask
Task Automation	os, subprocess, Selenium
Data Analysis	Pandas, NumPy
Machine Learning	scikit-learn, TensorFlow

Note: Python's popularity in these applications is due to its easy-to-read syntax and large community support. If you are thinking of a task you want to automate or analyse, chances are someone has already used Python for something similar.

Python's flexibility makes it an ideal choice for rapid prototyping, educational purposes, and expanding into new fields like robotics or IoT.

Python is well-known for its flexibility. This flexibility is a major reason for its popularity in various emerging areas and for people who are starting out with programming.

Python's design allows you to **quickly test ideas** and build working solutions. This rapid cycle makes it excellent for prototyping and exploring new concepts.

In education, Python serves as a first programming language due to its easy-to-read syntax and the large amount of learning resources available. It helps learners focus on problem-solving rather than worrying about complex language rules.

- **Simple, readable syntax** means fewer barriers for beginners.
- **Large library support** for almost any application area.
- **Active community** that contributes tools and resources.

When developing new hardware or exploring new science and engineering fields, technologies can shift rapidly. Python keeps pace by being easy to adapt and expand, helping professionals and hobbyists alike get up and running without wasted effort.

If you want to **explore robotics or the Internet of Things (IoT)**, Python provides tools for connecting to sensors, sending commands to devices, and processing data quickly.

- **Rapid prototyping:** Make and test new programs in less time.
- **Education:** Focus on understanding, not syntax errors.
- **Robotics:** Use Python libraries like 'pySerial' or 'RPi.GPIO' to control hardware.
- **IoT:** Send and receive data with packages like 'requests' or 'paho-mqtt'.

```
import time
print('Hello, prototype!')
for i in range(3):
    print(f'Step {i+1}')
    time.sleep(1)
print('Done!')
```

Use Case	Python Benefit
Rapid Prototyping	Start projects quickly and refine as you go
Education	Clear syntax and error messages aid learning
Robotics/IoT	Interface easily with hardware and process data

Note: Important tip: Python's versatility means you can switch from a small project to a larger, real-world application without needing to learn a new language. This helps you gain experience and adapt as your interests or career grow.

INSTALLING PYTHON AND DEVELOPMENT TOOLS

[Learn what Python is and discover its common use cases in industry and academia.](#)

Python is a popular programming language known for its simplicity and readability. It was created in the late 1980s by Guido van Rossum and has become one of the most widely used languages in the world. Python allows you to write programs quickly and with fewer lines of code compared to other languages. This makes it a great choice for beginners learning programming concepts.

Python is an **interpreted** language. This means your Python code is read and executed line by line by the Python interpreter, rather than being compiled into machine code beforehand. As a result, Python is flexible, easy to test, and allows rapid prototyping of ideas.

Python is also cross-platform. This means programs written in Python can run on different operating systems like Windows, macOS, and Linux, usually without any modifications. Python is free to use, and its code is open source, so many people contribute to improving it.

- Simple and clear syntax, making it easy to read and write code.
- Large community and extensive support libraries.
- Suitable for both small scripts and large applications.

Python's design makes it useful for a wide range of tasks. It is used both in industry and academia across many fields. In business, Python helps automate tasks, process data, and create software for different needs. In academic settings, it is used for teaching programming, analysing data, and doing scientific research.

Field	Example Use Cases
Web Development	Building websites and web applications using frameworks like Django and Flask.
Data Science & Analytics	Analysing large datasets, machine learning, and visualisation with libraries like pandas and matplotlib.
Automation & Scripting	Automating repetitive tasks, such as file management or report generation.
Education	Teaching programming concepts to beginners and students.
Scientific Research	Simulations, statistical analysis, and research computing.

```
print("Hello, world!")
```

This simple example prints 'Hello, world!' to the screen. It demonstrates Python's clear syntax. Even advanced programs in Python often remain easy to read and understand.

Note: Important tip: Learning Python will open doors to many areas of technology. Once you become comfortable, you can use your skills for web development, data analysis, artificial intelligence, or automation. Python's flexibility means you will be able to apply it to real-world problems in many fields.

Install Python and set up essential development tools to begin programming.

Before you can start programming in Python, you need to install Python itself and set up some basic development tools. This ensures you have all the resources and software needed to write and run Python programs on your computer.

Python is a popular and widely used programming language. **To get started, you must download and install Python from the official website.** You will also benefit from using a code editor or integrated development environment (IDE). These tools help you write, edit, and manage your code more effectively.

The installation process is straightforward but can vary slightly depending on your operating system: Windows, macOS, or Linux. Most users start with the official Python distribution, which includes the Python interpreter and standard library modules.

- Download Python from python.org.
- Run the installer and follow the prompts.
- Ensure you tick 'Add Python to PATH' on Windows to make running Python from the command line easier.
- Test your installation by opening a terminal or command prompt and typing 'python --version'.
- Install and set up a code editor or IDE such as Visual Studio Code, PyCharm, or Thonny.
- Familiarise yourself with the terminal or command prompt, which you will use to run scripts and manage files.

```
# Check if Python is installed and view the version
python --version

# Or, on some systems:
python3 --version
```

Tool	Purpose
Python Interpreter	Runs and executes your Python scripts
Code Editor / IDE	Helps you write and manage code files
Terminal or Command Prompt	Allows you to run scripts and commands

There are several code editors available. Visual Studio Code is free and works on all platforms. Thonny is a simple IDE designed especially for beginners. Experienced programmers may prefer tools like PyCharm for more features.

Note: Important tip: Always install the latest stable Python version unless instructed otherwise. If you encounter errors or 'command not recognised', you may need to restart your computer or check your installation settings.

Understand how to run scripts and use the interactive Python mode.

Once you have installed Python on your computer, you are ready to write and run Python code. There are two common ways to execute Python code: by running a script or by using the interactive mode. Understanding both methods will help you experiment with code, develop programs, and troubleshoot problems efficiently.

A **Python script** is simply a text file containing Python code, usually saved with a .py extension. To run a script, you ask Python to read the file and execute the instructions.

In contrast, the interactive Python mode provides a prompt where you can enter Python commands one at a time and see the result immediately. This is useful for testing small pieces of code, learning, or doing calculations.

- Run a Python script to execute a program from start to finish.
- Use interactive mode to test code snippets or explore language features.
- Both methods are useful for different stages of development.

```
# This is a simple Python script saved as hello.py
print('Hello, world!')

# To run this script from the terminal:
# For Windows:
#   python hello.py
# For macOS/Linux:
#   python3 hello.py
```

Method	How to Use
Python script	Write code in a .py file, then run it with <code>python filename.py</code>
Interactive mode	Open a terminal, type <code>python</code> or <code>python3</code> , then enter code directly at the <code>>>></code> prompt
IDLE (Python's editor)	Open IDLE, use File > New File to write scripts, or use the Shell for interactive mode

Note: Tip: The interactive mode displays a '>>>' prompt. After typing a command, press Enter to see the result. Use it to try out small pieces of code before adding them to a script.

Familiarise yourself with Python's basic syntax and typical program structure.

Understanding Python's basic syntax and how a typical program is structured is one of the first steps to becoming a confident Python programmer. Python prides itself on being readable and concise,

which makes it a favourite for beginners. This section introduces you to the essential rules and elements you will use in almost every Python program.

Python syntax refers to the set of rules that define how your code must be written for the interpreter to understand it. **Getting familiar with these rules ensures your scripts will run without errors.** You'll see that Python's structure is clean and straightforward compared to some other programming languages.

Whitespace and indentation are incredibly important in Python. Unlike many other languages, Python uses indentation (spaces at the start of a line) to define blocks of code, such as the body of a function, loop, or conditional. This means you don't use curly brackets to group statements. The standard convention is to use four spaces per level of indentation.

- Statements are written **one per line**.
- **Lines starting with # are comments**—they are not executed.
- **Indentation defines code blocks**. No curly brackets are used.
- **Variables are created by assignment** (no need to declare them first).
- **Python is case sensitive**. For example, myVar and myvar are different.
- **Strings are written** in single (' ') or double (" ") quotes.

```
# This is a comment
name = 'Sam'
print('Hello,', name)

if name == 'Sam':
    print('Welcome!')
else:
    print('Who are you?')
```

A typical Python program starts with comments to describe what the program does. Next, variables are defined, followed by logic using control flow statements (if, for, while, etc.). Functions can be defined to organise code or perform tasks. Finally, you may have one or more lines that actually run the main action of your program.

Element	Description
Variable assignment	Giving a name to a value (e.g., age = 20)
Comment	Line ignored by Python (starts with #)
Indentation	Spaces at the start of a line to define a block (e.g., inside an if statement)
Function	Reusable piece of code, starts with def

Python's syntax is designed to be logical and easy to follow. By sticking to its style rules, your code will not only work as expected but also be easy to read for yourself and others.

Note: Important tip: Always use consistent indentation. Mixing tabs and spaces, or inconsistent levels, will cause errors that are often hard to spot.

Gain hands-on experience with your development environment to practice writing code.

Getting hands-on experience with your development environment is vital when learning to program in Python. The development environment is the combination of tools you use to write, test, and debug code. Using these tools regularly builds your confidence and makes your learning much more effective.

Practising in your environment helps you **move from simply reading about programming to actually doing it**. Typing your own code, running it, and seeing results (or errors) gives you valuable feedback and helps make concepts stick.

Python code can be written using simple text editors or advanced Integrated Development Environments (IDEs). Good options include VS Code, PyCharm, Thonny, or even the built-in IDLE that comes with Python. Each tool can help you write and run code, highlighting mistakes, and offering features like automatic indentation and syntax colouring.

- Open your chosen code editor or IDE and create a new Python file (often ending with .py).
- Type a simple Python program, like printing a message or doing a calculation.
- Save your file, and run it using the 'Run' or 'Execute' command in the tool.
- Try making small changes to the code, saving, and re-running to see how the output changes.
- Use the built-in terminal or command line to run your script if your editor supports it.

```
print('Hello, world!')
number = 7
print('The number is:', number)
```

Action	How to Do It
Create a file	File > New File or Ctrl+N in most editors
Save the file	File > Save or Ctrl+S
Run the code	Press F5, click Run, or use python filename.py in the terminal

Note: Important tip: Don't be afraid to experiment. Breaking things in your own environment helps you learn. Every error message is a chance to understand and fix a problem. The more you practise, the easier writing and running code will become.

RUNNING PYTHON SCRIPTS AND INTERACTIVE MODE

Python programs can be executed either as standalone scripts or interactively using the Python shell.

Python is a flexible language that supports multiple ways of running your code. You can write complete programs and save them as script files, or you can experiment and test ideas using the interactive Python shell. Both methods are useful and serve different needs, especially when you are starting out.

When you write a **standalone script**, you store your Python instructions in a .py file. You then execute the whole file using the Python interpreter. This is ideal for larger programs or anything you want to reuse later.

In contrast, the Python shell provides an interactive environment (also called REPL: Read-Eval-Print Loop). Here, you type Python statements one at a time and see the immediate effect. This helps you learn, experiment, and debug small pieces of code quickly.

- Scripts let you organise and save your code.
- The interactive shell is great for learning and quick tests.
- Both methods use the same Python syntax and interpreter.

```
# Example 1: Running a Python script
print('Hello, world!')

# Save this as hello.py and run with:
# python hello.py

# Example 2: Using the interactive shell
# Type python in your terminal, then:
>>> print('Hello, world!')
Hello, world!
```

Mode	Description
Standalone Script	Python code is saved in a .py file and run all at once.
Interactive Shell	Code is entered line by line and executed immediately.
Use Case	Scripts suit complete programs, while the shell is best for experiments or simple tasks.

Note: If you are practising new concepts or testing small pieces of code, start with the interactive shell. When you want to build or share complete programs, use standalone scripts.

To run a script, save your code in a `.py` file and execute it from the command line using `'python filename.py'`.

Running your Python code as a script is an essential skill for every programmer. Rather than typing instructions one by one, you can write your entire program in a file, save it, and let Python run the whole thing at once. This is the standard way to run your code in real-world scenarios.

When you have finished writing your code, **save it with a `.py` extension** (for example, save as `my_program.py`). This tells your operating system and development tools that it is a Python file.

Once you have your script saved, you can execute it from the command line. Open your terminal (called Command Prompt on Windows, or Terminal on Mac and Linux). Use the `'cd'` command to change into the folder that contains your script. This is necessary so that the terminal knows where to find your file.

- Write your Python code in a text editor (e.g., VS Code, Notepad, Sublime Text).
- Save your file with a `.py` extension, like `hello.py`.
- Open a terminal or command prompt window.
- Change to the directory where you saved your file.
- Run the script by typing: `python hello.py`

```
print('Hello, world!')
```

```
C:\Users\Sam\PythonScripts> python hello.py
Hello, world!
```

Step	Description
1	Write your code and save as <code>filename.py</code>
2	Open terminal and navigate to the file location
3	Run with: <code>python filename.py</code>

It's common to have more than one version of Python installed. If running `'python filename.py'` gives an error, you may need to use `'python3 filename.py'` instead, depending on your system configuration.

Note: Important tip: Make sure you are in the right folder (directory) when you run your script. Use the 'cd' command to change directories. If your script still won't run, check that Python is installed and available in your PATH environment variable.

The interactive mode allows you to enter and execute Python statements one at a time, making it useful for testing and learning.

Python provides an interactive mode that lets you type and run code immediately. This is different from running a whole script, where you write a program in a file and run it all at once. Interactive mode is particularly helpful when you are first learning Python or when you want to quickly experiment with code.

When you start Python in interactive mode, you see something called a **prompt**. The prompt usually looks like '>>>'. After the prompt appears, you can type a statement—such as a calculation or a print command—and see the result immediately.

This approach is valuable for testing single lines of code, learning how functions behave, and quickly trying out ideas. If you make a mistake, you can fix it right away and try again, without rewriting a whole program.

- Enter Python interactive mode by typing 'python' or 'python3' in your terminal.
- Type any command or statement and press Enter to see its result right away.
- Use interactive mode for testing code snippets, calculations, and experimenting with Python features.
- Exit interactive mode at any time by typing 'exit()' or pressing Ctrl+D.

```
>>> 2 + 3
5
>>> print('Hello, world!')
Hello, world!
>>> x = 10
>>> x * 2
20
```

Task	How to do it in interactive mode
Check a calculation	Type the maths expression (e.g., 5 * 4)
See the result of a function	Type the function call (e.g., len('hello'))
Test a variable	Assign a value (e.g., x = 7) and use it

Note: Interactive mode is sometimes called the Python REPL: Read, Evaluate, Print, Loop. Use it as a 'scratch pad'—it is perfect for learning or exploring before writing full programs.

You can start the interactive mode by typing 'python' or 'python3' in your terminal or command prompt.

Python has an interactive mode that allows you to write and run Python code line by line. This mode is a helpful tool for beginners, as it lets you experiment with code, test ideas quickly, and see results immediately. You do not need to create or save a script file to use interactive mode.

To start the interactive mode, open your terminal (Linux or macOS) or command prompt (Windows). Type **'python'** or **'python3'** and press Enter. This will launch Python's interpreter, where you will see a prompt (usually `>>>`) waiting for your input.

You may need to use 'python3' instead of 'python' if you have both Python 2 and Python 3 installed. Many modern computers and environments use 'python3' to specify Python version 3, which is now the standard. Check which version you have by typing 'python --version' or 'python3 --version' before starting the interactive mode.

- Interactive mode lets you test small code snippets instantly.
- All code runs immediately after you press Enter.
- Ideal for learning, debugging, and experimenting with new code.
- You exit the interactive mode by typing 'exit()' or pressing Ctrl + Z (Windows) or Ctrl + D (Linux/macOS).

```
python

# or, on some systems
python3

# Sample session:
>>> print("Hello, world!")
Hello, world!
>>> 2 + 2
4
>>> exit()
```

Command	Description
python	Starts interactive mode (may run Python 2 or 3 depending on system)
python3	Specifically runs Python 3 in interactive mode
exit()	Exits interactive mode

Note: Important tip: If typing 'python' starts Python 2 on your computer, always use 'python3' to make sure you are learning and using the correct version.

Use the interactive mode for quick experimentation, but save scripts in files for more complex or reusable programs.

Python provides two main ways for you to write and run your code: interactive mode and script files. Each method serves a distinct purpose and best suits particular scenarios. Understanding when and how to use each will help you become a more effective programmer.

When you open a terminal and type **python** (or **python3**), you enter the Python interactive mode. You will see the **>>>** prompt. This is like a calculator, where you can type instructions one at a time and see the results immediately.

Interactive mode is very helpful for trying out small bits of code. It is excellent for quick testing, learning, and exploring. However, it is not suitable for storing larger programs, repeating tasks, or building applications or scripts you wish to run again later.

- Use interactive mode for simple calculations or testing ideas.
- Use script files when you want to write programs you can run more than once.
- Saving code in a file allows you to edit, share, and organise your work.

```
# Interactive mode example
>>> print('Hello, world!')
Hello, world!

# Script file example (hello.py)
print('Hello, world!')
```

Interactive Mode	Script File
Immediate feedback	Reusable code
Best for short tests	Best for complete programs
Changes are not saved	Code is saved and editable
Runs line-by-line	Runs entire file at once

Note: Remember: Use interactive mode to explore and learn, but save your valuable or complex code in script files. This makes it easier to organise, debug, and reuse your programs.

UNDERSTANDING PYTHON SYNTAX AND STRUCTURE

Python is a versatile programming language widely used in web development, data analysis, automation, and more.

Python is a high-level programming language known for its clear syntax and readability. It is designed to be easy to learn and use, which makes it especially popular among beginners and professionals alike.

You will find Python in many areas of modern computing. **Web development, data analysis, automation**, scripting, application development, and scientific computing are just some of the key fields where Python excels.

Python stands out because of its simplicity and flexibility. Its syntax is straightforward, allowing you to focus on problem-solving rather than complex code. This reduces the learning curve for those new to programming.

- Web development: Used to build dynamic websites and APIs using frameworks such as Django and Flask.
- Data analysis: Essential for processing and visualising data with tools like pandas and matplotlib.
- Automation: Scripts repetitive tasks, such as file management or data entry, to save time and reduce errors.
- Scientific computing: Powers complex calculations and simulations with libraries like NumPy and SciPy.
- Artificial intelligence and machine learning: Supports models and experimentation through libraries such as TensorFlow and scikit-learn.

```
# Example: Simple automation script in Python
import os

def list_files(folder):
    for file in os.listdir(folder):
        print(file)

list_files('.') # Lists all files in the current directory
```

Field	Python Example
Web Development	Building websites with Django or Flask
Data Analysis	Analysing data using pandas
Automation	Automating reports or system tasks
Scientific Computing	Performing calculations with NumPy
Machine Learning	Training models with scikit-learn

Note: Important tip: Python's large community means you will find plenty of tutorials, examples, and support online—making it even easier to start solving real-world problems straight away.

Set up your development environment by installing Python and any necessary development tools like text editors or IDEs.

Setting up your development environment is the first step on your Python programming journey. A suitable environment makes coding easier and helps you avoid common problems. You need Python itself and tools to write, edit, and run your code comfortably.

To start, you must **install Python**, which is the actual programming language interpreter. This allows your computer to understand and run Python programs. You also need a way to write your code, such as a **text editor** or **Integrated Development Environment (IDE)**. Choosing the right tools depends on your comfort and workflow, but either option is suitable for beginners.

Python is available for all major operating systems, including Windows, macOS, and Linux. Most beginners download Python from the official website. Modern macOS and Linux versions may already include Python, but it's best to install the latest version yourself. Different text editors and IDEs, such as VS Code, PyCharm, Atom, or even simple Notepad, can be used for Python development. IDEs provide extra features like code completion and debugging, which are helpful as you learn.

- **Download Python from the official website:** <https://www.python.org/downloads/>
- **Choose a text editor** (e.g., VS Code, Sublime Text, Notepad++) or an IDE (e.g., PyCharm, Thonny)
- **Install Python** by following the instructions for your operating system
- **Verify your installation** using the terminal (command prompt or shell)
- Optionally, install additional extensions or plugins to improve your coding experience

```
python --version
python3 --version
```


Tool	Purpose
Python Interpreter	Runs your Python code
Text Editor	Edits and saves your code files
IDE (e.g., PyCharm, Thonny)	Combines editing, running, and debugging tools

Note: Important tip: Always tick the box that says 'Add Python to PATH' during installation on Windows. This makes it easy to run Python from the command prompt without extra configuration.

Learn to run Python scripts and use the interactive mode (REPL) for quick code experimentation.

Before you start programming, it is crucial to understand how to run Python code. Python provides two main ways to do this—by running scripts or by using its interactive mode, commonly known as the REPL (Read-Eval-Print Loop). Each has its own advantages and situations where it is most useful. This section will show you the basics of both methods, including practical examples and tips to get you started with confidence.

Python scripts let you write longer programs in a file, while the **interactive mode (REPL)** allows you to quickly test ideas or code snippets and get instant feedback.

To run a Python script, you typically write your code in a plain text file with the .py extension. You then use the Python interpreter to execute the contents of that file. On the other hand, the interactive mode launches a prompt where you can enter Python statements one at a time. This is very helpful for learning, experimenting, or troubleshooting small pieces of code.

- Scripts are best for writing complete programs or saving code to use again.
- Interactive mode is perfect for trying out commands, learning syntax, or testing functions.
- Both methods use the same Python interpreter, so code behaves the same in each.

```
# Example script (hello.py)
print('Hello, world!')
```

To **run a script** from your command line or terminal, type:

```
python hello.py
```

If Python is set up correctly, you will see 'Hello, world!' appear on your screen. If you get an error, check that you are in the directory where your file is saved and that you typed the file name correctly. You may also need to use 'python3' instead of just 'python' on some systems.

To **start the interactive mode**, simply open your terminal or command prompt and type:

```
python
```

You will see three angle brackets (>>>) which means you are now in the Python REPL. Here, you can type and run Python instructions immediately. For example, you can do simple maths, print messages, or try out functions.

```
>>> 2 + 2
4
>>> print('Testing REPL')
Testing REPL
```

Method	Common Uses
Script (.py file)	Writing full programs, saving and sharing code
Interactive mode (REPL)	Experimenting, learning, testing snippets
Integrated Development Environment (IDE)	Combines script editing and REPL in one interface

Note: Interactive mode does not save your work by default. Use scripts for anything you want to keep or reuse. If you make a mistake in the REPL, simply type again—it's safe to experiment!

Familiarise yourself with basic Python syntax such as indentation, comments, and program structure.

Python is designed to be easy to read and simple to write. To achieve this, it uses specific rules known as syntax. These rules determine how you must organise your code so that Python will understand and run it correctly. Some of the most important parts of Python syntax include indentation, comments, and the general structure of a program.

Unlike other programming languages, **indentation in Python is not optional**. Indentation means using spaces at the beginning of a line to show that a block of code belongs together, such as under a loop or function. If you do not indent properly, Python will show an error. Comments let you explain what your code does. Comments start with the hash sign (#), and Python ignores everything after the # on that line.

At the most basic level, a Python program is simply a text file containing instructions that the Python interpreter runs in order. Good structure helps you and others understand, maintain, and extend your code. A typical program will start with import statements, followed by function or class definitions, and then the main code that is executed.

- Indentation groups code and shows its structure.
- Comments help explain code for humans but are ignored by Python.
- The program structure usually starts with imports, then functions, then running code.

```
# This is a comment
import sys

def greet(name):
    print('Hello, ', name)

if __name__ == "__main__":
    greet('world')
```

Concept	Example
Indentation	print('Indented code')
Comment	# This is a comment
Import	import os
Function Definition	def say_hello():

Note: Always use consistent indentation, such as four spaces per level. Mixing spaces and tabs can cause errors that are hard to find. Good comments and clear structure make your programs easier to understand and maintain.

Practice writing and executing simple Python programs to develop a solid understanding of the language's structure.

Learning Python is best done by writing and running your own code. When you practise creating simple programs, you become comfortable with Python's syntax and program structure. This hands-on approach lays the foundation for writing more complex applications later on.

Python programs follow a clear sequence. **Start by opening your text editor or IDE, type some code, then run it to see the results.** Each new program helps you understand how Python reads and executes instructions. The more you experiment, the more natural Python's rules and patterns will feel.

Let's start with very simple examples. Most Python programs are just a series of statements. Each statement tells Python to do something specific, such as print text or do a calculation. Here are some important steps to practise:

- Open your Python IDE or text editor.
- Type a simple Python program, such as one that prints a message.
- Save your file with the .py extension (for example, hello.py).
- Open a terminal or command prompt, navigate to the file's location.
- Run your program with the command `python hello.py`.
- Try changing your code and run again to see different results.

```
print("Hello, world!")

# This is a comment. Comments are ignored by Python and only for
humans to read.
number = 7
print("The number is:", number)
```

Element	Description
<code>print()</code>	A function to display information on the screen.
Variable (number)	Stores a piece of data you can use later.
Comment (# ...)	A note for you or others reading the code.

Note: Important tip: Try creating your own variations of simple programs. Change messages, add numbers, or use different variable names. Practising this way helps you spot errors and understand how Python handles your instructions.

Every time you write and run a program, pay attention to spacing and indentation. Python uses indentation (spaces at the beginning of a line) to show which code belongs together. For now, keep your programs simple—use a new line for each instruction, and do not indent unless you are asked to (indents are important in loops and functions, which you will learn soon).

If you make a mistake, Python will show you an error message. Do not worry—read the message carefully. Often, fixing small spelling mistakes or missing punctuation is enough. Making and fixing mistakes is how you learn.

Practising simple programs helps you see the pattern and structure of Python code. As you become more confident, try combining concepts—store a message in a variable, print it, or add two numbers and show the result. This ongoing practice builds skills you will use in all your future Python projects.

WRITING YOUR FIRST PYTHON PROGRAM

Understand the basic structure of a Python program, including how to write, save, and execute Python scripts.

Python is a simple, versatile programming language. Writing and running your own Python program involves a few basic steps. First, you should learn about the structure of a Python script and how Python code is organised. Then, you can write, save, and execute your own scripts. This process is essential for building programs that solve real problems.

A basic **Python program** usually follows a straightforward layout. Python code is written line by line, with each line representing an instruction. The program is interpreted—from top to bottom—by the Python interpreter. Program files are typically plain text files saved with the extension **.py**. Each statement in your script should be placed on its own line. This makes your code easy to read and understand.

To write a Python script, you should use a text editor or an Integrated Development Environment (IDE). Text editors like Notepad (Windows), TextEdit (macOS in plain text mode), or more advanced editors like VS Code, Atom, or Sublime Text work well. IDEs like PyCharm or Thonny offer features such as syntax highlighting and debugging tools.

- Open your preferred text editor or IDE.
- Type your Python code. Start with a simple instruction like `print("Hello, world!")`.
- Save the file with a `.py` extension, for example, `hello.py`.
- Open a terminal or command prompt window.
- Navigate to the folder where you saved your `.py` file.
- Run the script by typing `python hello.py` and pressing Enter.

```
print("Hello, world!")
print("Welcome to learning Python!")
```

Step	Description
Write code	Type your script in a text editor.
Save file	Use a <code>.py</code> extension, e.g., <code>example.py</code> .
Open terminal	Start the command prompt or terminal in your operating system.
Navigate to location	Use the <code>cd</code> command to go to your file's folder.
Execute script	Run <code>python example.py</code> to execute your program.

Note: Indentation is important in Python. Always use consistent spaces for indentation—typically four spaces per level. Do not mix tabs and spaces, as this may cause errors.

Practice using print statements for displaying output and comments for adding explanations to your code.

In Python programming, two of the simplest yet most important tools you will use are print statements and comments. Print statements let your code communicate by displaying messages or values to the screen. Comments help you and others understand what your code is meant to do. Both are essential for writing clear, beginner-friendly programs.

A **print statement** shows output. You use it to check results, show information, or help debug your code. On the other hand, a **comment** adds notes or explanations to your script. Comments never run as code – they are just for humans reading the program.

To display output, you use the print function. For example, `print('Hello, world!')` will show the text Hello, world! on your screen. You can print numbers, text, or even variables you have created. To add a comment, use the hash symbol (`#`). Everything after `#` on the same line is a comment and will be ignored by Python.

- **Use print statements** to display results or messages.
- **Comments start with #** and explain what sections of code do.
- **Comments help you remember your logic** and make code easier for others to read.
- **Print statements** are vital for testing and debugging.

```
# This is a comment explaining the next line
print('Learning Python is fun!') # This will display a message

number = 10 # Assign 10 to the variable 'number'
print(number) # Show the value of number
```

Element	Purpose
print statement	Displays output on the screen
comment	Explains code, ignored by Python
# symbol	Marks the start of a comment
string	Text shown in output, written in quotes

Note: Important tip: Good habits start early. Add comments to explain why your code does something, not just what it does. Use print statements often to check your progress and results.

Get familiar with using input statements to accept user input in your program.

In Python, programs can interact with users by asking for information through input statements. This allows your program to accept values, such as numbers or words, from the person running it. Learning how to request and use input is one of the first steps in building interactive applications.

The most common way to get input in Python is by using the **input()** function. This function pauses the program and waits for the user to type something and press Enter.

Input provided by users is always treated as text (a string) by default. If you need a different type, such as an integer or a decimal number, you must convert the input to the appropriate type. This is important for calculations or comparing numbers.

- The input() function displays a prompt and waits for input.
- Anything typed by the user is returned as a string.
- You can assign this value to a variable to use it later.
- To work with numbers, use int() or float() to change the type.

```
name = input('What is your name? ')
print('Hello,', name)

age_input = input('How old are you? ')
age = int(age_input)
print('Next year, you will be', age + 1)
```

Statement	What it does
input('Name: ')	Asks the user to enter their name
int(input('Age: '))	Asks for age and changes it to an integer
float(input('Price: '))	Asks for a price and changes it to a decimal

Note: Always check or handle errors if a user enters something unexpected, like text when a number is needed. This prevents program crashes and leads to a better user experience.

Test your first Python script and identify any errors through hands-on practice.

Testing your first Python script is a vital step in the learning process. Running the code you have written allows you to confirm that it behaves as you expect. It is also how you identify and correct errors, which are a normal part of writing code. This hands-on process improves your understanding, boosts your confidence, and helps you become familiar with how Python runs your instructions.

To start, **write a simple Python program in your chosen text editor**. A basic example is a script that prints a greeting to the screen. Save the file with a .py extension, such as hello.py.

Once your script is saved, you can test it by running it with Python. Open your terminal or command prompt, navigate to the folder where your file is saved, and use the command 'python hello.py'. If everything works, you will see your message printed. If there is an error, Python will show you a message explaining what went wrong.

- Check that your file name ends with .py
- Use the python command followed by your script name
- Look for typos or missing punctuation if errors appear
- Read the error message carefully; it often tells you the line and nature of the issue
- Try fixing the error, then run the script again

```
print("Hello, world!")
print("Welcome to Python programming.")
```

Common Error	What it Means
SyntaxError	There is a problem with the structure of your code, such as a missing bracket or quote.
NameError	You tried to use a variable or function name that has not been defined.
IndentationError	Your code is not properly aligned; indentation is important in Python.

Note: Important tip: When you see an error, do not worry. Errors are a normal part of programming. Use the information in the error message to guide your fix, and remember to save your file each time before testing again.

Experiment with editing and rerunning your Python program to observe how changes affect the output.

Once you have written your first Python program, it is important to understand how making changes to your code affects what your program does. Experimenting with editing and rerunning your program is a key step in learning how programming works.

Imagine you have just finished a simple Python script that prints a message. By **editing** the message in your code, then saving the file and running it again, you can immediately see how even small changes produce different outputs. This process helps you build confidence and curiosity in exploring programming.

Let's walk through a typical flow. You write your program. You notice something in the output or think of a way to improve it. You edit the code, then rerun the program to see what happens. This cycle helps you learn both the Python language and how your changes affect program behaviour.

- Open your Python program in your chosen code editor.
- Change part of the code, for example, the text you print or a number in a calculation.
- Save your file after making changes.
- Rerun the program by executing it in your terminal or IDE.
- Observe and compare the new output to the previous one.

```
print("Hello, world!")  
# Now edit the message and rerun:  
print("Hello, Python!")
```

Edit Made	Resulting Output
Original: print('Hello, world!')	Hello, world!
Edited: print('Hello, Python!')	Hello, Python!
Edited: print('Welcome to Python programming.')	Welcome to Python programming.

Note: Important tip: Always save your file before rerunning it. Unsaved changes will not show up in the output. Experimenting helps you understand not just the output, but also the cause-and-effect relationship in your code.

2. Variables, Data Types, and Operators

VARIABLES AND NAMING CONVENTIONS

Use variables to store and manage data for use in your Python programs.

Variables are an essential concept in any programming language, including Python. They allow you to store data in your program so you can refer to it and manipulate it later. Instead of repeating values or computations, you give the data a name and work with that name. This makes your code easier to read and maintain.

In Python, a **variable acts as a labelled box** to hold a piece of information. You can store almost any kind of data—numbers, text, even more complex objects—in a variable. When you want to use the data, you just mention the variable's name.

You create a variable with a single line. Use an equals sign (=) to assign a value. The name goes on the left, and the data goes on the right. For example, `name = 'Alice'`. This stores the word Alice in a variable called name. If you want to use that information again, just use 'name'.

- **Variables** are containers for storing data.
- A **variable name** is how you refer to the stored data.
- Python lets you **change a variable's value at any time**.
- The **data inside a variable** can be a number, text (string), or another type.
- Good variable names help others understand your code.

```
age = 25
name = 'Alice'
is_student = True

print(age)
print(name)
print(is_student)
```

Variable Name	Value Stored
age	25
name	'Alice'
is_student	True

When you assign a new value to a variable, Python forgets the old value. For example, if you write `age = 30` later in your code, `age` now stores 30 instead of 25. This flexibility makes it easy to update information as your program runs.

Choose clear, descriptive names for your variables. Names like `'user_age'` or `'total_cost'` make your program easy to understand. Avoid names that are too short or cryptic, such as `'a'` or `'x'`, unless they have a clear meaning.

Note: Important tip: Variable names in Python are case-sensitive. 'Score' and 'score' are different variables. Stick to lowercase with words separated by underscores, like 'user_name', for consistency.

Follow standard naming conventions for variables, such as using descriptive names and lowercase letters with underscores.

When you write Python programs, it is important to name your variables clearly and consistently. Following standard naming conventions makes your code easier to read and maintain, both for yourself and for others who may work with your code in the future.

Python uses a common style for variable names called **snake_case**. This means all letters are lowercase and words are separated by underscores. For example, you would write `"user_score"` instead of `"UserScore"` or `"userscore"`.

Using descriptive names for variables is highly recommended. A good variable name tells you what kind of data it stores, making your code much more understandable. Avoid short, vague names like `'x'` or `'data'` unless their use is obvious from the context.

- Use only lowercase letters and underscores (e.g., `first_name`, `total_amount`).
- Start variable names with a letter (not a number or symbol).
- Make names descriptive, reflecting the purpose of the variable (e.g., `customer_count`, `max_speed`).
- Avoid using spaces, special characters, or hyphens.
- Do not use reserved Python keywords (like `'if'`, `'for'`, `'while'`) as variable names.

```
# Good examples
user_age = 25
account_balance = 136.50
is_active = True

# Poor examples (do not use)
UserAge = 25 # Uses upper case and camelCase
2nd_place = 'Silver' # Starts with a number
user-age = 35 # Uses a hyphen
data = 10 # Not descriptive
```

Good Name	Poor Name
first_name	FirstName
total_price	tp
email_address	emailAddress
order_count	orderCNT
max_capacity	maxCapacity

Note: Important tip: Following standard naming conventions makes your code more professional and much easier to understand. It also helps when you work in teams or contribute to open source projects, where consistency is key.

Assign appropriate data types to variables, including integers, floats, strings, and booleans.

In Python, you use variables to store values that your program can work with. Each value you assign to a variable belongs to a specific data type. Choosing the right data type makes your program easier to understand, use, and maintain. Four of the most common data types in Python are integers, floats, strings, and booleans.

When you **assign** a value to a variable, Python automatically determines the data type for you. You do not need to tell Python what kind of data it is in advance. However, it's important to use the most suitable data type for your data. This helps your program run correctly and efficiently.

Each data type has its own use and behaviour. Integers are whole numbers. Floats represent numbers with decimal places. Strings are sequences of letters, numbers, or other characters, enclosed in quotes. Booleans are simple truth values – they can be either True or False. Knowing which type to use is an essential programming skill.

- **Integers (int):** Store whole numbers, like 5 or -20.
- **Floats (float):** Store decimal numbers, such as 2.5 or -0.89.
- **Strings (str):** Store text, for example "Hello" or "2024".
- **Booleans (bool):** Store True or False values, useful for decision-making.

```
my_age = 25           # integer
height_metre = 1.75  # float
first_name = "Sam"    # string
is_student = True     # boolean
```

Data Type	Example Value
int	42
float	3.14
str	'apple'
bool	False

To check what data type a variable is, you can use Python's `type()` function. This is helpful for debugging and understanding your code. If you give a variable a new value of a different type, Python will update its data type automatically. Be careful – mixing types can sometimes cause errors.

```
score = 85
print(type(score))
score = "eighty-five"
print(type(score))
```

Assigning the **appropriate data type** helps you avoid bugs and makes your code easier to read. For example, if you store a number as a string, you will not be able to perform maths with it until you convert it back to a number.

- Use integers for counting and whole numbers (e.g., age, quantity).
- Use floats for measurements and precise values (e.g., price, height).
- Use strings for names, labels, and text (e.g., city names, file paths).
- Use booleans to flag yes/no or true/false conditions (e.g., `is_active`, `has_permission`).

Note: Important tip: If you try to use an integer in a place where a string is expected (or the other way around), you may get an error. Check your data types, especially when taking input from users or files.

Make sure variable names do not begin with numbers and do not contain spaces or special characters.

When you create variables in Python, you need to follow some important rules for naming. These rules help keep your code error-free and easy to read. If you break these rules, Python will not understand your code and it will result in errors.

The first rule is that **variable names must not begin with a number**. This means you cannot start a variable name with digits like 1, 2, or 3. For example, '1score' is not allowed, but 'score1' is fine.

Secondly, variable names cannot contain spaces. Instead, use an underscore (_) to separate words, such as 'total_score'. If you use a space, Python will get confused and will not know where the variable name ends.

Another key rule is to **avoid special characters like @, #, \$, %, or ! in your variable names**. Only letters (A-Z, a-z), numbers (but not at the start), and underscores (_) are allowed. This keeps your variable names simple and prevents mistakes.

- Do not start variable names with a number.
- Do not include spaces in variable names. Use underscores instead.
- Do not use special characters such as @, !, or \$ in variable names.

```
# Correct variable names
score1 = 30
total_score = 95
user_name = 'Alex'

# Incorrect variable names
1score = 30          # Error: starts with a number
total score = 95     # Error: contains a space
user@name = 'Alex'   # Error: contains a special character
```

Variable Name	Valid?
age	Yes
2ndPlace	No
first_name	Yes
user name	No
total\$amount	No

Note: Good variable names make your programs easier to read and debug. Choose clear and simple names that follow the rules.

Use consistent naming practices to make your code easier to read and maintain.

Naming your variables well matters a lot in programming. Good names help you and others read, understand, and change your code with ease. Consistency in naming means sticking to a set of simple rules, so that anyone can quickly follow what each part of your code does. This is especially helpful as your projects grow larger or when you work on code in a team.

You should always **choose meaningful variable names that describe their purpose**. For example, use a name like `age_in_years` instead of a vague name like `a` or `x`. Clear names remove confusion and make errors less likely.

Python has common naming methods. You'll often see names written with underscores between words, like `total_price` or `first_name`. This style is called `snake_case` and is standard for variables and functions in Python. Using `snake_case` throughout your code helps keep things familiar and easy to scan.

- Use lowercase letters and words joined by underscores (`snake_case`).
- Describe what the variable holds or represents (e.g., `item_count`).
- Avoid using single-letter names except for short loops (e.g., `for i in range(5)`).
- Be consistent: if you decide on a style, use it everywhere.
- Do not use Python reserved words as variable names (such as `list` or `int`).

```
# Good naming examples
user_age = 30
unit_price = 12.50
is_valid = False

# Poor naming examples
a = 30
prc = 12.50
iv = False
```

Variable Name	Purpose
<code>student_name</code>	Stores the name of a student
<code>total_sales</code>	Tracks the sum of sales
<code>is_active</code>	True if something is active, False if not

Note: Important tip: Stick to the same naming convention across your project. This makes your code faster to read and easier to update, especially when you return to it after some time or share it with others.

BUILT-IN DATA TYPES (INT, FLOAT, STRING, BOOL)

Understand and use built-in data types in Python, such as integers (int), floating-point numbers (float), strings, and boolean values (bool).

Python provides several built-in data types. Some of the most important for beginners are integers, floating-point numbers, strings, and boolean values. When you write Python code, you often need to store and manipulate data using these types. Understanding them helps you make your programs behave correctly and predictably.

An **integer** (int) represents whole numbers without decimal points, such as 10, -5, or 0. A **float** (floating-point number) is a number with a decimal point, like 3.14 or -0.001. **Strings** are sequences of characters, such as words, phrases, or even single letters. They are used to store text. **Boolean values** (bool) are variables that can be only **True** or **False**, often as the result of comparisons or conditions.

You use different data types for different tasks. Numbers are used for calculations. Strings store names or messages. Booleans help you make decisions in your programs. Using the right data type makes your code easier to read and less error-prone. Python can often convert between these types automatically, but it helps to know exactly what type your data is at each step.

- **int:** Whole numbers (e.g., 1, -42, 999)
- **float:** Decimal numbers (e.g., 3.14, -0.5, 2.0)
- **str:** Text or characters (e.g., "Hello, world!", 'A')
- **bool:** True or False values (e.g., True, False)

```
# Integer example
age = 30

# Float example
height = 1.75

# String example
name = "Alice"

# Boolean example
is_student = True
```


Type	Example Value
int	100
float	3.14159
str	"Python is fun!"
bool	False

Note: You can check the type of any variable in Python using the `type()` function. This is useful for debugging or making sure your code works as expected.

Apply variables to store and reference data, using appropriate naming conventions for clarity and readability.

Variables are essential in every programming language, including Python. They are used to store pieces of data, such as numbers or text, so you can use and change them while your program runs. Understanding how to use variables and name them well is a key foundation for writing clear and reliable code.

A **variable** acts as a label that points to a value stored in the computer's memory. You give each variable a name, and this name lets you reference or update the data whenever needed. For example, if you want to store a person's age, you can use a variable such as **age** to hold that value.

Python allows you to create a variable simply by assigning a value to a name. There are no special keywords needed. You pick a meaningful name, use the equals sign (`=`), and provide the value you want to store.

- Variables can store different data types (numbers, text, etc.).
- Names should describe the data (example: `temperature_celsius`, `user_name`).
- Variable names must start with a letter or underscore, not a number.
- Use lowercase letters and underscores between words for readability.
- Avoid vague names like `x`, `y`, or `data`, except for short examples or loops.

```
height_cm = 180
first_name = 'Alice'
is_logged_in = True
balance = 150.75
```

Variable Name	Typical Use
count	Number of items (e.g., count = 5)
total_price	Sum of prices (e.g., total_price = 42.30)
user_email	Stores an email address

Note: Choose descriptive names that make your code easy for you and others to understand. Consistently using clear naming conventions helps prevent mistakes later.

Perform type conversions between different data types to ensure correct program behaviour.

When working with data in Python, different data types are used for different purposes. Sometimes, you need to change a value from one type to another. This process is called 'type conversion.' Proper type conversion ensures your program behaves as expected and avoids errors.

For example, user input from the keyboard is always a string, so you often need to convert it to a number before doing any calculations.

Python provides several built-in functions to convert between common data types. These functions include `int()`, `float()`, `str()`, and `bool()`. You use them to convert values directly. Understanding when and how to apply these conversions is critical in programming.

- `int(x)`: converts `x` to an integer, removing decimal parts.
- `float(x)`: converts `x` to a floating-point number.
- `str(x)`: converts `x` to a string.
- `bool(x)`: converts `x` to a boolean value (True or False).

```
age_input = input('Enter your age: ')
age = int(age_input)  # Convert string to integer

price = 19
price_str = str(price)  # Convert integer to string

value = '3.14'
value_float = float(value)  # Convert string to float
```

Sometimes, type conversion occurs automatically behind the scenes. This is known as 'implicit conversion.' However, explicit conversion is often needed to avoid surprises or errors. For example, adding a string to an integer without conversion will cause a `TypeError`.

Data	Type Conversion Result
'123' (string)	int('123') → 123 (integer)
5.8 (float)	int(5.8) → 5 (integer, decimals removed)
True (boolean)	int(True) → 1
123 (integer)	str(123) → '123' (string)

Note: Important tip: Always check if the value you want to convert makes sense for the target type. For example, int('hello') will cause an error. Use try-except blocks to handle conversion errors gracefully.

Utilise arithmetic, comparison, and logical operators to perform calculations and control program flow.

Operators in Python allow you to perform actions on values and variables. They are essential for calculations, decision making, and controlling how your program behaves. The three common types are arithmetic, comparison, and logical operators.

Arithmetic operators let you work with numbers. **Comparison operators help you check relationships between values.** Logical operators combine conditions so you can create complex decisions.

These operators are used everywhere in Python programming, from simple calculations to making choices. Understanding how and when to use each type will improve your ability to write clear, correct code.

- Arithmetic operators include: + for addition, - for subtraction, * for multiplication, / for division, // for integer division, % for remainder, and ** for exponentiation.
- Comparison operators include: == checks if two values are equal, != checks if they are not equal, > checks if one value is greater than another, < for less than, >= for greater than or equal to, <= for less than or equal to.
- Logical operators include: and, or, not. They combine or reverse conditions, allowing you to make complex decisions.

```
# Arithmetic operators
x = 5
y = 2
addition = x + y          # 7
multiplication = x * y    # 10
division = x / y          # 2.5
```

```
# Comparison operators
is_equal = x == y      # False
is_greater = x > y     # True

# Logical operators
likes_python = True
knows_python = False
can_teach = likes_python and knows_python # False
```

Operator	Description
+	Addition (adds numbers)
==	Equals (checks if values are the same)
and	Logical AND (both conditions must be true)

Note: Remember to use brackets (parentheses) to group operations for clarity—especially with logical operators. For example: if (x > 2) and (y < 5):

Manipulate and process strings using built-in Python methods and techniques.

Strings are sequences of characters used to store and work with text in Python. You will often need to modify, analyse, or format string data. Python provides many built-in methods and features to help you with common string tasks. This makes handling text data simple and efficient.

Whenever you deal with names, addresses, or any other human-readable information, you will work with **strings**. These built-in methods let you change case, find and replace text, split content, join strings, and more. Mastering these skills is essential for reading input, showing output, and processing text files.

Each string in Python is an object, so it comes with dozens of useful methods. You do not need to import any libraries to access these. You simply call the method on your string variable using dot notation.

- Change case (e.g., uppercase, lowercase, titlecase)
- Remove spaces and unwanted characters
- Search for specific text patterns
- Split a string into a list
- Join a list of words into a single string
- Replace parts of a string
- Check if a string starts or ends with certain text

```
# Change case
greeting = 'Hello, World!'
print(greeting.lower())      # hello, world!
print(greeting.upper())      # HELLO, WORLD!
print(greeting.title())      # Hello, World!

# Remove whitespace
text = '    spaced out    '
print(text.strip())          # 'spaced out'

# Split and join
words = 'Python is fun'.split()
print(words)                  # ['Python', 'is', 'fun']
sentence = '-'.join(words)
print(sentence)               # Python-is-fun

# Replace text
new_text = greeting.replace('World', 'Python')
print(new_text)               # Hello, Python!
```

Method	Purpose
lower()	Converts string to lowercase
upper()	Converts string to uppercase
replace(old, new)	Replaces part of the string
strip()	Removes leading and trailing spaces
split(separator)	Splits the string into a list
join(list)	Combines list elements into a string
startswith(text)	Checks if string starts with given text

Note: String operations never change the original string. Each method creates and returns a new string. Remember to assign it to a variable if you want to save the result.

You can also use indexing and slicing to extract or modify specific parts of a string. For example, `name[0]` gives the first character, and `name[1:4]` gives a substring from position 1 up to (but not including) 4.

```
name = 'Alison'  
first_letter = name[0]      # 'A'  
part = name[1:4]           # 'lis'
```

Strings are powerful tools in Python, and you will use them in almost every program. Practise these techniques to become confident working with text. Explore the full list of methods using Python's built-in help system or the official documentation.

TYPE CONVERSION

Understand the purpose of type conversion in Python, such as converting data from one type to another to ensure compatibility in operations.

Type conversion is an essential concept in Python programming. It means changing a value from one data type to another, such as from a string to an integer. This is important because different types of data cannot always be used together in operations. Understanding when and how to convert types ensures your programs run smoothly.

Imagine you have a piece of data, like **"123"**, which looks like a number but is actually stored as text (a string). If you want to add this to another number, you need to first convert it to an integer. Without conversion, Python will throw an error or behave in unexpected ways. Type conversion helps you avoid these issues and combine data correctly.

Automatic type conversion happens when Python changes a data type for you, often called 'implicit conversion.' However, most of the time you'll need to use 'explicit conversion' by calling special functions. These functions are `int()`, `float()`, `str()`, and others. Knowing how to apply them is a core skill in writing reliable programs.

- You need type conversion when input from a user is read as text but needs to be used as a number.
- Many operations (such as maths and comparisons) require data to be of the same type.
- Type conversion lets you prepare data for output or storage in the right format.

```
value = "42"  # This is a string, not a number  
number = int(value)  # Convert from string to integer  
print(number + 8)  # Output: 50
```

Function	Purpose
<code>int()</code>	Convert to integer
<code>float()</code>	Convert to decimal (floating-point) number
<code>str()</code>	Convert to string (text)

Note: Important tip: Always check the original data type before you convert it. Not every value can be turned into every type. For example, 'hello' cannot be converted to an integer and will cause an error.

Use built-in functions like `int()`, `float()`, `str()`, and `bool()` to convert between types as needed for calculations and data processing.

Type conversion is a common and important technique in Python programming. Sometimes, you have data in one type (like a string or an integer) and need to change it to another so you can perform calculations or use it in the right context. Python provides several built-in functions that make it easy to convert between types.

The main type conversion functions in Python are `int()`, `float()`, `str()`, and `bool()`. These functions allow you to switch between integer, floating-point, string, and Boolean types. Using them correctly is key to processing input, handling user data, and writing flexible programs.

For example, if a user provides a number as text (such as '10'), you need to convert it to an integer before adding or multiplying. If you calculate a result and want to display it to the user, you might convert a number to a string. Type conversion is also critical when working with data files, as information is often read as text and must be changed to numbers before you can process it.

- `int()`: Converts a value to an integer. Useful for whole numbers.
- `float()`: Changes a value to a floating-point number. Good for decimals.
- `str()`: Alters a value to a string. Helpful for display or text manipulation.
- `bool()`: Converts a value to True or False. Useful in decision making and testing.

```
# Converting string to integer and adding
user_input = "7"
number = int(user_input)    # Now number is the integer 7

total = number + 5          # total is 12

# Converting integer to string for display
result_str = str(total)     # '12'

# Converting a string to a float
```

```
decimal = float('3.14')    # 3.14

# Converting a value to boolean
is_empty = bool("")        # False (empty string is False)
is_nonzero = bool(123)     # True (non-zero numbers are True)
```

Function	Example Result
int('42')	42 (integer)
float('8.5')	8.5 (float)
str(23)	'23' (string)
bool(0)	False

Note: Some conversions only work if the value is compatible. For example, int('hello') will cause an error. Always check your data before converting.

Be aware that improper type conversions can lead to errors or unexpected results; always check the data before converting.

Type conversion is common in Python programming. You often need to convert data from one type to another, such as changing input text into numbers for calculations. However, converting data incorrectly can cause errors or produce unexpected results.

For example, if you try to **convert a string that does not represent a number (like 'abc') into an integer**, Python will raise an error. Always make sure your data is in the expected format before performing type conversions.

If you assume that user input will always be a number, but someone enters words or symbols, your program could crash. Mistakenly converting data types can also change the meaning of the data. For example, converting a floating-point number to an integer will remove any decimal places, which could lead to inaccurate calculations.

- Wrong type conversions can cause your program to stop working.
- You may get unexpected values if you convert without checking the input.
- Data can lose accuracy or meaning if you convert inappropriately.

```
user_input = '12.5'
try:
    value = int(user_input)
```



```
except ValueError:
    print('Cannot convert to integer. Please enter a whole
number.')
```

Conversion Example	Result
<code>int('25')</code>	25 (integer, works fine)
<code>int('hello')</code>	Error (cannot convert non-numeric string)
<code>int(9.8)</code>	9 (decimal part is removed)

Note: Important tip: Always validate or clean your data before converting types. Use checks, error handling, or built-in Python functions like `isnumeric()` where possible to avoid crashes and surprises.

Practice converting types when reading input, performing calculations, or interacting with various data sources.

In Python, data comes in different types such as strings, integers, and floats. It is common to need to convert between these types, especially when reading user input, performing mathematical calculations, or working with data from files or other sources.

When you read data—like using `input()`—Python always treats it as a string. To perform arithmetic, you must first convert the value to a number type, such as an integer or float.

Type conversion lets you control how Python stores data in memory. Without correct conversions, your code may not work as expected. Python provides built-in functions such as `int()`, `float()`, and `str()` to help you change data from one type to another.

- Use `int()` to convert a string to an integer (e.g., '5' becomes 5)
- Use `float()` to turn a string or integer into a floating-point number (e.g., 5 becomes 5.0)
- Use `str()` to turn numbers into strings for display or text processing
- Always validate or handle errors in case the data cannot be converted

```
# Reading input and converting types
age_str = input('Enter your age: ')
age = int(age_str) # Convert the input from string to integer
print('Next year, you will be', age + 1)

# Reading numbers from a file (as strings)
with open('numbers.txt', 'r') as file:
```

```

    for line in file:
        number = float(line.strip()) # Convert each line to a
float
        print('Half the number is', number / 2)

# Converting numbers to strings
score = 95
message = 'Your score is: ' + str(score)
print(message)

```

Scenario	Type Conversion Example
User Input	<code>int(input('Enter a number: '))</code>
Math Calculation	<code>float('3.14') * 2</code>
File Data	<code>int(line.strip())</code>

Note: Important tip: Always check and handle possible exceptions. Invalid conversions, such as `int('hello')`, will cause an error. Use try-except blocks to write safer code.

Understand the difference between implicit and explicit conversions, and use explicit conversions for clarity and accuracy.

When you work with variables and data in Python, sometimes you need to change a value from one type to another. This is known as type conversion. Python handles type conversions in two ways: implicitly and explicitly. Understanding the difference is essential for writing clear and reliable code.

In **implicit conversion**, Python automatically changes the data type for you when it is safe to do so. In **explicit conversion**, you clearly instruct Python to change the data type by using a built-in function. Knowing when and how to use each type of conversion helps avoid bugs and makes your code easier to read.

Let's take a closer look at how Python performs type conversions, with some examples.

- Implicit conversions happen automatically, often during calculations.
- Explicit conversions use specific functions like `int()`, `float()`, and `str()` to convert values.
- Explicit conversions are clearer and reduce surprises in your code.

```

# Implicit conversion example
x = 5          # x is an integer
y = 2.0        # y is a float
result = x + y  # Python automatically converts x to a float
print(result)   # Output: 7.0

```

```
# Explicit conversion example
x = 5.7
z = int(x)  # You explicitly convert x to an integer
print(z)    # Output: 5
```

Conversion Type	Example
Implicit	Adding int (3) + float (4.0) results in float (7.0)
Explicit	Using str(100) converts integer 100 to string '100'
Explicit	Using float('4.5') converts string '4.5' to float 4.5

Note: Important tip: Always use explicit conversion when you want to control how your data is handled. This makes your code more predictable and easier for others (and your future self) to understand.

OPERATORS (ARITHMETIC, COMPARISON, LOGICAL)

Operators in Python allow you to perform calculations, compare values, and make decisions within your programs.

Operators are symbols and keywords in Python that let you work with data and direct your programs. They allow you to do calculations, compare different values, and control how your code behaves. Learning how operators work is essential to getting started with Python.

There are three main types of operators in Python: **arithmetic** (for math operations), **comparison** (for checking relations between values), and **logical** (to combine conditions and make decisions).

Arithmetic operators let you add, subtract, multiply, and divide numbers. For example, you can use '+' to add two values or '*' to multiply them. These are useful any time you need to do math, like calculating totals or finding averages.

- Arithmetic operators: +, -, *, /, //, %, **
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: and, or, not

```
a = 10
b = 5
sum = a + b          # Arithmetic: 15
is_equal = a == b    # Comparison: False
is_both = (a > 0) and (b > 0)  # Logical: True
```

Operator	Purpose
+	Add two values
==	Check if two values are equal
and	Combine two conditions

Comparison operators allow you to check if values are the same, different, or if one is bigger or smaller than the other. For example, you might want to know if a user's score is higher than another score, or if two names match.

Logical operators combine multiple conditions. For instance, you might want a program to check if two things are true at the same time, or if at least one condition is true. Logical operators give you very powerful tools to control what your program does based on different situations.

Note: Important tip: Using parentheses around your conditions can make your code easier to read and helps avoid errors in complex expressions!

Arithmetic operators include +, -, *, /, %, and **, and are used to perform mathematical operations.

Arithmetic operators in Python are symbols that allow you to perform basic mathematical calculations on numbers. These operators let you add, subtract, multiply, divide, find remainders, and work with exponents. You will use these operators often when writing programs that involve any type of maths or calculations.

The main **arithmetic operators** in Python are easy to use, whether you are working with whole numbers (integers) or decimal values (floats).

Here is an introduction to each arithmetic operator, with examples to show you how they work in practice.

- + (Addition): Adds two numbers together. Example: 3 + 5 gives 8.
- - (Subtraction): Subtracts the right number from the left. Example: 10 - 4 gives 6.
- * (Multiplication): Multiplies two numbers. Example: 6 * 7 gives 42.
- / (Division): Divides the left number by the right, always returns a decimal (float) result. Example: 8 / 2 gives 4.0.
- % (Modulus): Finds the remainder after division. Example: 9 % 4 gives 1.
- ** (Exponentiation): Raises a number to the power of another. Example: 2 ** 3 gives 8.

```
a = 7
b = 3
```

```

sum_result = a + b          # 10
subtraction = a - b         # 4
multiplication = a * b      # 21
division = a / b            # 2.3333333333333335
modulus = a % b             # 1
exponent = a ** b           # 343

print('Sum:', sum_result)
print('Subtraction:', subtraction)
print('Multiplication:', multiplication)
print('Division:', division)
print('Modulus:', modulus)
print('Exponent:', exponent)

```

Operator	Description/Example
+	Addition: 2 + 3 = 5
-	Subtraction: 9 - 2 = 7
*	Multiplication: 4 * 5 = 20
/	Division: 10 / 2 = 5.0
%	Modulus (remainder): 11 % 4 = 3
**	Exponentiation (power): 3 ** 2 = 9

Note: When you use the division operator (/), Python always returns a float result, even if the numbers divide evenly. For example, 8 / 2 gives 4.0, not 4. If you need whole number division, use the double slash operator (//) for floor division.

Comparison operators, such as ==, !=, >, <, >=, and <=, are used to compare two values and return True or False.

Comparison operators are special symbols in Python that allow you to compare two values. These operators check relationships between values, such as if they are equal, not equal, greater, or less than each other. When you use a comparison operator, Python evaluates the comparison and returns a result. This result is always a boolean value: either True or False.

For example, if you want to check if two numbers are the same, you can use the == operator. If you want to check if they are not the same, you use the != operator. Other comparison operators include >, <, >=, and <=.

These operators are often used in conditional statements such as if and while. By comparing values or variables, you can make choices in your programs. For example, you might check if someone's age is above 18 to decide if they are eligible to vote.

- `==` (equal to): Checks if two values are the same.
- `!=` (not equal to): Checks if two values are different.
- `>` (greater than): Checks if the value on the left is larger.
- `<` (less than): Checks if the value on the left is smaller.
- `>=` (greater than or equal to): Checks if the value on the left is larger or the same.
- `<=` (less than or equal to): Checks if the value on the left is smaller or the same.

```
a = 5
b = 10
print(a == b)    # False, because 5 is not equal to 10
print(a != b)    # True, because 5 is not equal to 10
print(a > b)     # False, because 5 is not greater than 10
print(a < b)     # True, because 5 is less than 10
print(a >= 5)    # True, because 5 is equal to 5
print(b <= 15)   # True, because 10 is less than 15
```

Operator	Description
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Note: Remember: Comparison operators always return True or False. You can use them with numbers, strings, and even other data types.

Logical operators like **and**, **or**, and **not** help you build complex conditions and control program flow.

Logical operators are special words in Python that let you combine simple conditions into more detailed decisions. You use them to make your program behave differently based on several requirements at the same time.

The three main logical operators in Python are **and**, **or**, and **not**. These operators match how we talk about decisions in everyday life.

Suppose you want your program to check if a person is old enough and is a citizen before they can vote. Or, you may want to show a warning if someone is not logged in. This is where logical operators are most useful.

- Use **and** to check if several things are true at the same time.
- Use **or** to allow for any one of several conditions to be true.
- Use **not** to reverse the meaning of a condition.

```
# Example: Using logical operators

age = 20
is_citizen = True
if age >= 18 and is_citizen:
    print("You can vote.")

is_logged_in = False
if not is_logged_in:
    print("Please log in to continue.")

password = "Python123"
if password == "Python123" or password == "python123":
    print("Password accepted.")
```

Operator	Description or Example
and	True only if both conditions are true (e.g., <code>age >= 18 and is_citizen</code>)
or	True if at least one condition is true (e.g., <code>user == "Alice" or user == "Bob"</code>)
not	Reverses a condition (e.g., <code>not is_logged_in</code> is True if <code>is_logged_in</code> is False)

Note: Logical operators help you make programs smarter by allowing for more detailed decisions. You can combine several logical operators in one statement, but use brackets (parentheses) to make the order clear.

Understanding when and how to use each type of operator is fundamental to writing accurate and efficient Python code.

In Python, operators let you perform operations on variables and values. They are the building blocks for working with numbers, comparing results, and making decisions. Selecting the right operator is critical for getting the correct result and keeping your code easy to understand.

There are three major types of operators in Python: **arithmetic**, **comparison**, and **logical**. Each serves a different purpose when processing your data.

Arithmetic operators handle maths operations, comparison operators check how values relate to each other, and logical operators allow you to combine conditions. Knowing when and how to use each helps you write code that works as intended and is efficient.

- Arithmetic operators are used for tasks like addition, subtraction, multiplication, and division.
- Comparison operators help you test if two values are equal, different, or if one is bigger or smaller than another.
- Logical operators link different conditions together, like checking if two things are true at once or at least one of them is true.

```
# Arithmetic Example
x = 10
y = 3
sum = x + y          # Addition
product = x * y      # Multiplication

# Comparison Example
is_equal = x == y    # False, because 10 is not equal to 3

# Logical Example
a = True
b = False
result = a and b     # False, because both must be true
```

perator Type	Example and Description
Arithmetic	5 + 2 (Addition: sums two numbers)
Comparison	price > 20 (Checks if 'price' is greater than 20)
Logical	is_active and is_verified (Both conditions must be true)

Note: Always choose the operator that matches your intent. For example, use '==' to compare if two values are the same, not '=' which is for assignment. Using the wrong operator can lead to bugs that are hard to spot.

WORKING WITH STRINGS

Understand how to create and use strings in Python.

Strings are one of the most fundamental data types in Python. A string is simply a sequence of characters enclosed within single (' '), double (" "), or triple quotes (''' ' or "" "" """). Strings are used to represent text such as words, sentences, or even paragraphs.

To **create** a string, use quotes around your text. For example, 'Hello', "Python", and "Good Morning" are all valid Python strings.

You can use strings in Python to store names, messages, file paths, or any text-based information. Strings are assigned to variables in the usual way, and you can print, join, or split them, as well as perform many common text manipulations.

- Strings can be empty ("") or contain any characters.
- Use single, double, or triple quotes for creating string literals.
- Triple quotes allow multi-line strings.
- Escape sequences like \n (new line) and \t (tab) add special characters.
- You can combine (concatenate) strings using the + operator.
- Use len() to find the length of a string.

```
# Creating strings
name = 'Alice'
greeting = "Hello, world!"
multi_line = '''This is
a multi-line
string.'''

# Printing strings
print(name)
print(greeting)
print(multi_line)

# Concatenating strings
full_greeting = greeting + ' I am ' + name
print(full_greeting)

# Finding length of a string
length = len(greeting)
print('Greeting has', length, 'characters')
```

Example	Result
'Python'	Python
"123abc"	123abc
"""Line1\nLine2"""	Line1 (new line) Line2
'Hi' + ' there'	Hi there
len('apple')	5

Note: Strings in Python are immutable. This means you cannot change a character within an existing string directly. Instead, you need to create a new string if you want to modify it.

Explore different string methods for manipulating text, such as concatenation, slicing, and formatting.

Strings are essential for working with text in Python. You will often need to join, split, or modify pieces of text in your programs. Python provides many ways to manipulate strings, making it easier to handle user input, display information, or transform data. In this section, you'll learn about string concatenation, slicing, and formatting—three fundamental methods for working with text.

The most basic string manipulation is to **join strings together**. This is called concatenation. Slicing lets you extract parts of a string, while formatting helps you build messages that include variables or dynamic information.

Learning how to combine and modify strings will help you build user-friendly programs. For example, you may need to display a personalised message, extract a headline from a paragraph, or format numbers for output.

- Concatenation: Join two or more strings using the + operator.
- Slicing: Extract a part of a string by specifying start and end positions.
- Formatting: Insert variables or values into strings using special methods.

```
# Concatenation
greeting = 'Hello, ' + 'world!'
print(greeting)  # Output: Hello, world!

# Slicing
text = 'Python programming'
word = text[0:6]  # Extracts 'Python'
print(word)
```

```
# Formatting with f-strings
name = 'Mia'
age = 25
message = f'My name is {name} and I am {age} years old.'
print(message)

# Formatting with str.format()
template = 'The temperature is {} degrees.'
print(template.format(22))
```

Method	Description
Concatenation (+)	Combines two or more strings into one.
Slicing ([start:end])	Extracts a portion of the string.
f-strings (f'...')	Inserts variables into a string using curly braces.
str.format()	Formats strings by placing values in placeholders.

Note: Remember, strings in Python are immutable. This means once a string is created, it cannot be changed. Any operation like concatenation or slicing creates a new string.

Practice common string operations, including searching, replacing, and splitting text.

Strings are a common data type used to store text in Python. Many programs rely on string operations for processing data, cleaning input, or extracting information. Mastering string operations like searching, replacing, and splitting will help you handle a wide range of text-processing tasks.

You can **search for text** in a string using functions like `find()` or by using operators. When you want to **replace text**, Python offers the `replace()` method. To **split a string into pieces**, you can use `split()`. Each of these operations is simple and powerful.

To find out where a substring appears in a string, use the `find()` method. It returns the position as a number (called an index), or -1 if it does not find the substring. The `replace()` method lets you swap out parts of a string with new text. The `split()` method breaks a string into a list by splitting it at a specified character or pattern.

- `find()`: Locate the position of a substring within a string.
- `replace()`: Swap out part of a string for something else.
- `split()`: Break a string into a list based on a separator.
- `count()`: Tally how many times a substring occurs in a string.
- `startswith()` and `endswith()`: Check if a string starts or ends with certain text.

```
# Example string
sentence = "The quick brown fox jumps over the lazy dog."

# 1. Search for a word
index = sentence.find("fox")      # Returns 16
not_found = sentence.find("cat") # Returns -1

# 2. Replace a word
new_sentence = sentence.replace("dog", "cat")

# 3. Split sentence into words
words = sentence.split()

print(index)      # 16
print(not_found)  # -1
print(new_sentence) # 'The quick brown fox jumps over the lazy cat.'
print(words)      # ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog.']
```

Method	Description
find(sub)	Returns index of first occurrence of sub, or -1.
replace(old, new)	Replaces all occurrences of old with new.
split(sep)	Splits string into a list using sep as separator.

Note: String methods like `find()`, `replace()`, and `split()` do not change the original string. They return a new string or a list. In Python, strings are immutable, so any changes create a new string.

Learn how to handle user input as strings and convert data types when needed.

When building Python programs, you will often need users to provide information. This could be a name, a number, or some other value. In Python, any input you get from a user is received as a string, even if the user types a number. Because of this, you'll need to know how to work with input, treat it as a string, and convert it to other data types when necessary.

The basic way to get input from a user in Python is with the **input()** function. This function displays a prompt, waits for the user to type something, and returns what they enter as a string.

For example, if you want to ask the user for their age, you might use `input()`. However, `input()` does not try to guess what the user meant. Whether the user types 25 or Hello, you will get a string each time, like '25' or 'Hello'.

- User input from `input()` is always a string, even if it looks like a number.
- You may need to convert input to another type (such as integer or float) for calculations.
- Trying to use a string as a number before converting it will cause errors.

```
user_name = input('What is your name? ')
user_age = input('How old are you? ')

# user_age is a string! Let's convert it:
age_number = int(user_age)
print('You will be', age_number + 1, 'next year!')
```

Function	Purpose
<code>input()</code>	Get user input as a string
<code>int()</code>	Convert a string (e.g. '5') to an integer (5)
<code>float()</code>	Convert a string (e.g. '3.14') to a floating-point number (3.14)
<code>str()</code>	Convert a value to its string form (e.g. 5 to '5')

Note: Important tip: Always check or handle the possibility that the user enters invalid data. If you try to convert 'abc' to an integer, your program will crash. Use try-except blocks for safer conversions.

Apply string techniques to real-world scenarios, such as formatting output or filtering data.

Strings are a fundamental part of Python programming. You use them whenever you deal with text—whether that's displaying messages for users, getting input, or processing information. Real-world programs spend a lot of time working with strings. Key scenarios include formatting output to users, preparing reports, extracting relevant information, searching within files, and cleaning up messy data.

Suppose you want to display a user's name and account balance. **String formatting** lets you combine text and data in useful ways. Python offers several options for this, including f-strings (formatted string literals), the `format()` method, and old-style `%` formatting.

Using formatting improves the readability and appearance of your program's output. It also helps ensure that numbers and dates use consistent styles, which makes your application more professional. For example, you can present currency with two decimal places, pad numbers with zeros, or align columns in a table.

- You can filter and clean data by removing unwanted spaces or characters with `strip()`, `replace()`, or regular expressions.
- Split large blocks of text into lines or words using `split()`. This is useful for parsing logs or preparing data for analysis.
- Check if a string matches a certain pattern with `startswith()`, `endswith()`, or `in`.
- Count the number of times a word appears using `count()`.
- Convert text to uppercase or lowercase to standardise input with `upper()` and `lower()`.

```
name = 'Fatima'
balance = 1545.9387

# Using an f-string for readable output
print(f'Hello {name}, your account balance is ${balance:.2f}')
```

```
# Filtering data: remove extra whitespace and replace unwanted
characters
raw_email = '  Contact: info@example.com;  '
clean_email = raw_email.strip().replace('; ', ' ')
print(clean_email)
```

```
# Splitting a string and filtering lines that contain a keyword
log = 'INFO: All systems go\nERROR: Disk full\nINFO: Backup
completed'
for line in log.split('\n'):
    if 'ERROR' in line:
        print('Problem found:', line)
```

Technique	Example Use
<code>strip()</code>	Remove leading or trailing spaces from user input
<code>replace()</code>	Change hyphens to spaces in a product name
<code>split()</code>	Break up a CSV line for processing columns
f-string	Format output with labels and variables

Note: When handling user input, always clean and check your text before using it. This helps prevent bugs and makes your programs more reliable and secure. Using string methods effectively will give you more control over your application's behaviour.

3. Control Flow – Conditionals and Loops

CONDITIONAL STATEMENTS (IF, ELIF, ELSE)

Conditional statements in Python are used to execute code blocks based on specific conditions using **if**, **elif**, and **else**.

Conditional statements allow your Python programs to make decisions. They help your code do different things depending on input, data, or other situations. You use these statements to control the flow of your program, so it can act in an intelligent way rather than doing the same thing every time.

In Python, the main keywords for conditional statements are **if**, **elif**, and **else**.

The **if** statement checks a condition and runs a block of code if the condition is true. If you have more conditions to check, you use **elif** (short for 'else if'). If none of the conditions is true, **else** lets you provide fallback code to run.

- **if**: Checks a condition. Runs code if the condition is true.
- **elif**: Checks another condition if the previous ones are false.
- **else**: Runs code if none of the conditions above are true.

```
temperature = 30
if temperature > 25:
    print('It is a hot day.')
elif temperature > 15:
    print('It is a nice day.')
else:
    print('It is a cold day.')
```

Keyword	Purpose
if	Run code when a condition is true
elif	Check another condition if previous ones were false
else	Run default code if all previous conditions were false

Note: Indentation matters in Python. Always indent your code blocks under **if**, **elif**, and **else** statements—usually by four spaces.

You can use any type of logical comparison in a condition, such as equal to (==), not equal (!=), greater than (>), less than (<), and even combine these with and, or, and not.

```
age = 18
if age >= 18 and age < 65:
    print('You are an adult.')
else:
    print('You are either under 18 or a senior.')
```

Remember, you do not have to use elif or else every time. An if statement can stand alone if you only need to check and act on a single condition.

Note: Conditional statements are essential for writing programs that adapt to user input or data. They form the backbone of logical thinking in Python and are used in almost every application.

The if statement checks a condition and runs the associated block if the condition is true.

Conditional logic is a central part of any program. The if statement in Python allows your code to make decisions. When the computer reaches an if statement, it checks whether a specific condition is true or false. If the condition is true, the code underneath the if statement (the associated block) will run. If the condition is false, this code is skipped.

An **if statement** lets your program choose what to do based on the situation. This is similar to asking, 'If it is raining, do I need an umbrella?' Your program follows this logic: If the condition matches, run the required steps.

To create an if statement in Python, you use the if keyword, followed by a condition and a colon. The condition is usually a comparison (for example: is x greater than 10?). The block of code to run if the condition is true is indented underneath the if statement. Indentation is critical in Python as it defines which statements are part of the if block.

- The condition after if must produce True or False (a Boolean value).
- If the condition is True, the indented code underneath runs.
- If the condition is False, Python skips the indented code.
- You can check variables, perform comparisons, or call functions in conditions.

```
temperature = 18
if temperature < 20:
    print("It is a bit chilly. Wear a jumper.")
print("Have a good day!")
```


Condition	Will the print statement run?
temperature < 20 is True	Yes
temperature < 20 is False	No
temperature == 18	If the temperature is 18, code will run

Note: Indentation matters in Python. Forgetting to indent the code after an if statement or using the wrong number of spaces will cause an error. The standard is four spaces per indentation level.

The elif (else if) statement allows you to test multiple conditions after an initial if.

In Python, conditional statements help your program make decisions. The if statement checks if a condition is true. If it is, it runs a block of code. But what if you have more than two outcomes? This is where elif comes in. The elif keyword stands for 'else if'. It lets you check more conditions after the first if.

When you use an if statement, only one block of code runs if the condition is true. If you want to check another possibility, you can use an **elif** clause. You can have as many elif clauses as you need. If none of the if or elif conditions are true, you can finish with an else clause.

This makes your program flexible. You can test as many different situations as you want, without repeating lots of if statements. It also makes your code easier to read and maintain.

- Use if for the first condition you want to check.
- Use elif for each extra condition.
- Use else for anything not covered by if or elif.

```
temperature = 18

if temperature > 25:
    print("It's hot today.")
elif temperature > 15:
    print("It's mild today.")
elif temperature > 5:
    print("It's a bit chilly.")
else:
    print("It's cold today.")
```

Statement	Purpose
if	Checks the first condition
elif	Adds more conditions to check if the first is false
else	Runs if none of the above conditions are true

Note: Remember, Python checks each condition in order. As soon as one is true, it runs that code and skips the rest. This order is important.

The else statement provides a block of code that runs if none of the preceding conditions are true.

In Python, conditional statements let you control which parts of your program run, depending on certain conditions. The most common conditionals use the keywords `if`, `elif`, and `else`. Each checks whether something is true, then decides which code block to run. The `else` statement has a special role: it runs only if all the previous conditions fail.

Let's say you ask a user to enter their age. You want your program to print a message based on the age they enter. First, `if` checks for the main situation you want to handle. Then, you might use `elif` to check for other possibilities. Finally, you use the `else` block to cover all remaining cases—the 'catch-all' for anything not already handled by your `if` or `elif` branches.

The `else` statement is always attached to an `if` block, and it must come last. Python reads your conditions from top to bottom. If none of the `if` or any `elif` statements are true, the `else` block runs. This is useful for handling exceptions, errors, or simply providing a default response in a program.

- `else` must come after all `if` and `elif` branches.
- Only one `else` block can be attached to an `if/elif` chain.
- If a condition in `if` or `elif` is true, `else` is skipped.
- `else` often handles 'all other cases' for clarity and completeness.

```
age = int(input('Enter your age: '))
if age < 18:
    print('You are under 18.')
elif age < 65:
    print('You are an adult.')
else:
    print('You are a senior.')
```

Condition	Message Displayed
age < 18	You are under 18.
18 <= age < 65	You are an adult.
age >= 65	You are a senior.

Note: Important tip: Use else to make your programs more reliable. It ensures every possible input or situation has a response, even if you haven't thought of every single condition.

Correct indentation is essential for defining code blocks under each conditional statement in Python.

Indentation is one of the most important rules in Python programming. Unlike many other languages that use braces or keywords to indicate blocks of code, Python relies on the position of code lines to group commands together. Every time you write a conditional statement like 'if', 'elif', or 'else', you must indent the code that should be executed as part of that condition.

For example, when you write an **if** statement, any code that should run only when the condition is true must be indented by the same number of spaces. The same rule applies for **elif**, and **else** blocks. This ensures that Python knows exactly which lines belong together as a unit.

By convention, Python uses four spaces per level of indentation. However, it is possible to choose a different number of spaces, as long as you are consistent within the same code block. Mixing tabs and spaces is not allowed and will cause errors.

- Indentation groups related lines of code under a condition.
- All lines within the same block must be indented equally.
- Incorrect indentation leads to syntax errors in Python.
- Indentation visually shows the structure and flow of your program.
- Nested blocks require additional indentation.

```
temperature = 21
if temperature > 20:
    print('It is warm.')
    print('Wear light clothing.')
else:
    print('It is cool.')
    print('Wear a jacket.')
```

Line	Indentation Level
if temperature > 20:	0 (no indentation)
print('It is warm.')	1 (4 spaces)
print('Wear light clothing.')	1 (4 spaces)
else:	0 (no indentation)
print('It is cool.')	1 (4 spaces)
print('Wear a jacket.')	1 (4 spaces)

Note: Important tip: Always use spaces rather than tabs for indentation in Python. Most code editors offer a setting to insert spaces automatically when you press the Tab key. Sticking to four spaces is the standard and helps avoid problems with code readability and compatibility.

NESTED CONDITIONS

[Learn to implement decision-making in Python using if, elif, and else statements.](#)

Decision-making is a core concept in every programming language. In Python, you use `if`, `elif`, and `else` statements to control how your program responds to different situations. These statements help your code make choices, such as what to do when a user enters a certain value or when some condition is met.

In Python, the `if` statement checks if a condition is true and runs a block of code if it is. The `elif` statement, short for 'else if', checks the next condition if the first one is not true. Finally, the `else` statement runs a block of code if none of the previous conditions are true.

This control flow lets your program take different paths based on input or internal logic. Each decision is made in order, from top to bottom. Once a matching condition is found and its code block is run, the rest are skipped. Proper indentation is essential in Python, as it makes blocks of code clear and prevents errors.

- **Use `if`** to check the first condition.
- **Use `elif`** to check another condition if the previous ones are false.
- **Use `else`** when none of the tested conditions match.
- **Indent the blocks** under each statement.
- You can have multiple `elif` statements but only one `if` and one `else` per block.

```
age = 18
if age < 13:
    print('You are a child.')
elif age < 18:
    print('You are a teenager.')
else:
    print('You are an adult.')
```

Condition	Output
age = 10	You are a child.
age = 15	You are a teenager.
age = 25	You are an adult.

Note: Use a colon (:) after if, elif, and else statements. Always indent the statements inside each block. Python relies on indentation to separate code blocks.

Understand and apply nested conditions to handle more complex logic scenarios.

In programming, sometimes you need to make decisions based on more than one condition. Nested conditions allow you to structure your code so that different checks or decisions happen inside other conditional logic. This is helpful when dealing with more complicated decision-making situations.

A **nested condition** occurs when you place an if, elif, or else statement inside another if, elif, or else block. This means the inner condition will only be checked if the outer one is true. Nesting conditions lets you handle scenarios where several related checks must be performed, each depending on the result of the previous ones.

Imagine a scenario in which you want to check whether a person is eligible for a driving licence. You might first check if the person is over 18. If true, you want to check whether they have passed a driving test. Only if both conditions are satisfied do you approve the licence application. This is a perfect case for nested conditions.

- **Nesting** helps organise multi-step decisions
- You can **nest if, elif, or else** blocks inside each other
- **Nested logic** can be as deep as required, but too much nesting can get confusing
- **Indentation** is important to show which conditions are nested
- **Use nested conditions** when one check should only happen after another

```
age = 20
has_passed_test = True
```

```

if age >= 18:
    print("You are old enough.")
    if has_passed_test:
        print("Congratulations, you are eligible for a driving
licence.")
    else:
        print("You need to pass the driving test.")
else:
    print("You are not old enough for a driving licence.")
  
```

Age	Passed Test	Eligibility Output
17	True	You are not old enough for a driving licence.
20	False	You are old enough. You need to pass the driving test.
25	True	You are old enough. Congratulations, you are eligible for a driving licence.

Note: Indentation is vital with nested conditions. Each nested block must be indented further to show its structure. Poor indentation can cause errors or make your code hard to understand.

Use for and while loops to repeat tasks and process data efficiently.

Loops are important in Python for repeating actions and processing data. They help you write code that handles repetitive tasks without having to type the same instructions over and over. Python provides two main types of loops: for loops and while loops. These are reliable tools for tasks like processing lists, reading files, or performing calculations multiple times.

A **for loop** is best used when you know exactly how many times you want to repeat a task. If you have a collection of items such as a list, you can use a for loop to go through each item one by one. A **while loop** is ideal when you need to keep repeating an action until a certain condition is no longer true.

Understanding when to use each type of loop helps you write efficient, simple code. For example, reading every line in a file or processing each number in a list is best with a for loop. If you want to keep prompting a user until they provide a valid answer, a while loop is suitable.

- **Use for loops** to process each element in a sequence like a list or string.
- **Choose while loops** when the number of repetitions is not known beforehand.
- Loops can reduce errors by preventing copy-and-paste coding.

```
# Using a for loop to process each item in a list
```

```

numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print('Number:', num)

# Using a while loop to repeat until a condition is met
count = 0
while count < 5:
    print('Count:', count)
    count += 1
  
```

Type of Loop	When to Use
for	Fixed number of repeats, iterating over a collection
while	Repeat until a condition changes (unknown repetitions)

Note: Always check that your loop's ending condition is achievable. If not, your loop might run forever, causing your program to freeze.

Control how your loops execute using break and continue statements.

When working with loops in Python, you sometimes need to take more control over how they run. Two statements help you do this: break and continue. These statements let you stop a loop early or skip parts of it, based on conditions in your code.

The **break** statement immediately ends the loop it is in, even if the loop has not finished all its cycles. The **continue** statement skips the rest of the current iteration and moves on to the next loop cycle. Understanding how and when to use these statements gives you more flexibility in solving many programming problems.

Picture a scenario where you are searching a list of names for a particular entry. Once you find the entry, you don't need to keep searching. The break statement lets you exit the search loop as soon as you find what you need. Similarly, if you want to ignore certain items, such as skipping blank entries or filtering out special values, continue helps move past them without running the rest of your loop commands for those cases.

- **Use break** to exit a for or while loop before it naturally ends.
- **Use continue** to skip the rest of the current repetition and go to the next one.
- Both statements only affect the loop in which they are written.

```

# Example using break
names = ["Alice", "Bob", "Charlie", "Dana"]
for name in names:
    if name == "Charlie":
  
```

```

        print("Found Charlie! Stopping search.")
        break
    print(f"Checked {name}")

# Example using continue
scores = [55, 0, 45, 70]
for score in scores:
    if score == 0:
        print("Zero score found, skipping.")
        continue
    print(f"Processing score: {score}")

```

Statement	Effect on Loop
break	Ends the current loop immediately.
continue	Goes straight to the next loop cycle, skipping any code after continue inside the loop.

Note: Important tip: Use break and continue carefully. Overusing them may make your code harder to follow. They are most helpful when you need to respond to specific conditions as you process data in a loop.

Apply control flow concepts to build flexible and responsive programs.

Control flow is a critical part of any program. It describes how your code makes decisions and repeats actions. In Python, control flow allows you to guide your program through different paths based on data, user input, or other conditions. When you apply control flow concepts well, you create programs that can adapt, react, and handle unexpected situations gracefully.

When building **flexible and responsive programs**, you use conditionals and loops to decide what your program does next. For example, you might check if a user's input is valid, repeat a process until it is finished, or skip certain steps based on settings. By combining if/elif/else statements with for and while loops, your Python code can adapt to a wide variety of scenarios.

Imagine a program that asks for a user's age and responds differently depending on the value. You want your code to say 'Too young' for ages under 18, 'Welcome' for adults, and flag anything invalid. This is a common use for conditionals. If you also want to keep asking until the user gives a valid age, you need a loop. Combining these approaches leads to code that really interacts with its users and reacts to their choices.

- Conditionals (if/elif/else): Let your program choose different actions based on information.
- Loops (for/while): Let your program repeat actions until a condition changes.
- Nesting and combining: Use conditionals inside loops, and vice versa, for complex logic.
- break and continue: Control the flow inside loops for more flexible behaviour.

- User input and error checking: Use control flow to validate data and handle unexpected situations.

```
while True:
    age_input = input('Please enter your age: ')
    if not age_input.isdigit():
        print('Invalid entry. Please enter a number.')
        continue
    age = int(age_input)
    if age < 0:
        print('Age cannot be negative.')
    elif age < 18:
        print('Too young for this activity.')
    elif age >= 18:
        print('Welcome!')
        break
```

Concept	Purpose
if/elif/else	Make decisions in code
for loop	Repeat actions for a series/grid
while loop	Repeat while a condition remains true
break	Exit a loop early
continue	Skip to the next loop iteration

Note: Important tip: Always check for invalid input or unexpected situations. Proper use of control flow makes your Python programs more robust, interactive, and user-friendly.

LOOPING CONSTRUCTS (FOR, WHILE)

Learn how to use for and while loops to repeat actions in your Python programs.

Loops let you repeat actions in a Python program. This is useful when you want to process multiple items, perform repeated calculations, or automate routine tasks. Python provides two main types of loops: for loops and while loops. Understanding how to use these will help you write more efficient and flexible code.

A **for loop** is best when you know in advance how many times you want to repeat something, such as working through items in a list or a sequence of numbers. A **while loop** is more suited to situations where you want to repeat actions until a certain condition changes, and you might not know beforehand exactly how many repetitions are needed.

Both for and while loops contain a block of code that runs repeatedly. The difference lies in how repetition is controlled. Let's look at each loop type step by step, with clear examples.

- Use a for loop when stepping through a set of items (like a list or a range of numbers).
- Use a while loop when you want to keep going until a condition is no longer true.
- Both types can help avoid duplicating code and reduce errors.

```
print("For loop example:")
for number in range(1, 6):
    print("Number:", number)

print("\nWhile loop example:")
count = 1
while count <= 5:
    print("Count:", count)
    count += 1
```

Loop Type	Typical Use Case
for	Iterate through a list of items
for	Repeat a fixed number of times using range()
while	Repeat until a condition becomes False
while	Pause and wait for user input or event

Note: Important tip: Always make sure your while loops will eventually end, otherwise your program could get stuck in an infinite loop. Increasing or changing the loop variable inside the while loop is essential.

Understand the syntax and structure of both for loops (for iterating over sequences) and while loops (for repeating until a condition is met).

Loops are a fundamental concept in Python. They allow you to repeat actions in your programs. Python provides two main types of loops: for loops and while loops. Each is suited to different situations. Understanding when and how to use them will make your code clearer and more flexible.

A **for loop** is used for iterating over a sequence, such as a list, tuple, or string. It steps through each item in the sequence in order. A **while loop** is used to repeat an action as long as a specific condition is true.

For loops are great when you know exactly what you want to step through—such as every item in a list of fruits, or every number in a range. While loops are useful when the number of repetitions is not known in advance, but depends on a condition that you check each time.

- Use for loops to cycle through sequences like lists, dictionaries, strings, or ranges.
- Use while loops when you want to keep repeating as long as a condition remains true.
- Both loops need a block of code to run each time (called the loop body).

```
# Example of a for loop
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Example of a while loop
count = 0
while count < 3:
    print('Counting:', count)
    count += 1
```

Loop Type	Typical Use Case
for loop	Iterating over known sequences
while loop	Repeating until condition is false
for loop	Counting with range (e.g. for i in range(5))

Note: Indentation is important in Python. The statements inside a loop must be indented to show what belongs to the loop body. If you forget this, Python will give you an error.

Practice controlling loop execution with **break** and **continue** statements to manage flow within loops.

When writing programs with loops, you may want to control how and when a loop should stop or skip its standard path. Python provides two handy statements—**break** and **continue**—for this task. These help you make your loops smarter and more flexible in responding to different situations.

The **break** statement is used when you want to completely exit a loop before it reaches its natural end. On the other hand, the **continue** statement skips the rest of the code inside the current loop block and moves straight to the next iteration. They help to refine how loops behave in your program.

Let's look at an example where **break** and **continue** are useful. Suppose you are searching through a list of numbers for a specific value. If you find it, you want to stop (**break**) the search. Or, if you only want even numbers, you might want to skip the odd ones using **continue**.

- **break** exits a loop early when a condition is met.
- **continue** skips to the next round of the loop, missing the rest of the code below it.
- Both statements can be used with **for** or **while** loops.

```
# Using break
numbers = [1, 3, 5, 7, 9, 11]
for num in numbers:
    if num == 7:
        print('Found 7! Stopping the loop.')
        break
    print(f'Checked number: {num}')

# Using continue
for num in numbers:
    if num % 2 != 0:
        continue # Skip odd numbers
    print(f'Even number: {num}')
```

Statement	Effect
break	Ends the loop immediately.
continue	Skips to the next loop cycle.
No statement	Loop runs as normal until done.

Note: Important tip: Overusing break and continue may make code harder to read. Use them to simplify logic or handle edge cases, not as shortcuts for unclear logic.

Develop skills to apply loops in practical situations, such as processing lists or automating repetitive tasks.

Loops in Python let you repeat actions automatically. This is useful when you need to process many items, perform calculations several times, or handle repetitive jobs like checking files or responding to user input. Instead of writing the same code again and again, you write a loop to do the hard work for you.

In Python, two main types of loops allow you to automate repetition: the **for loop** and the **while loop**. Both are powerful, but they have different strengths and are suited to different problems.

A for loop is ideal when you want to go through each item in a list, tuple, or other sequence. For example, suppose you have a list of numbers and want to print each one. Using a for loop lets you process every item without writing extra code.

- for loops are best for processing each item in a sequence (like a list or string).
- while loops are best when you need to repeat something until a condition is met.
- You can use loops to automate repetitive jobs, reducing manual mistakes and saving time.

```
# Example: Print every item in a list using a for loop
items = ["apple", "banana", "cherry"]
for fruit in items:
    print("I like", fruit)

# Example: Keep asking the user until they guess correctly
answer = "python"
guess = ""
while guess != answer:
    guess = input("Guess the word: ")
print("Correct!")
```

Loop Type	Typical Practical Use
for	Processing every item in a list (e.g., summing numbers, updating records)
while	Repeating an action until a condition is false (e.g., getting valid user input)
for & while	Automating repetitive tasks like file processing or monitoring a process

Loops help you handle practical programming tasks efficiently. Suppose you want to send a reminder email to every person in your contacts list. You would use a for loop to go through the list and send the email to each contact automatically. Or imagine cleaning up data from a file: you might use a while loop to read lines one by one until you reach the end.

Note: Important tip: Always double-check your loop conditions. An incorrect condition can lead to infinite loops or skip items. For for loops, use clear and predictable sequences. For while loops, make sure something changes within the loop, so it eventually finishes.

Recognise common pitfalls in loop construction, such as infinite loops and off-by-one errors, and learn how to avoid them.

Loops are a core part of programming, allowing you to repeat actions efficiently. However, it is easy to make mistakes when writing loops. Some common issues include infinite loops and off-by-one errors. Identifying and avoiding these problems will help you write reliable Python code.

An **infinite loop** runs endlessly because the condition to stop the loop is never met. This can cause your program to freeze or use up all your computer's resources. An **off-by-one error** happens when a loop repeats one time too many or one time too few, often because of incorrect start or end values.

The two main types of loops in Python are for loops and while loops. A for loop is typically used when you want to repeat an action a specific number of times. A while loop is used when you want to repeat an action until a certain condition is met. Both types can suffer from common looping mistakes if you are not careful.

- Infinite loops keep running without stopping
- Off-by-one errors can miss or double-count a loop iteration
- Forgetting to update variables in a while loop causes bugs
- Using the wrong range values in for loops can skip or repeat steps
- Breaking out of loops incorrectly may halt execution before finishing

```
# Infinite loop example
count = 0
while count < 5:
    print('Counting:', count)
    # count is never increased! This loop never ends.

# Off-by-one error example (should print 0 to 4, but prints 0 to 5)
for i in range(0, 6):
    print(i)
```

Pitfall	How to Avoid
Infinite loop	Always check your condition will eventually become false. Update variables that change the condition inside the loop.
Off-by-one error	Understand how Python's range() works: range(0, 5) runs 0, 1, 2, 3, 4, but not 5. Test your start and stop values.
Missed update	In while loops, update the variable used in the condition (e.g., count = count + 1) every time the loop runs.

Note: If your program seems to 'hang' or never finish, it may be stuck in an infinite loop. You can often stop an infinite script in the command line by pressing Ctrl+C.

BREAK AND CONTINUE STATEMENTS

Break and continue statements are used within loops to modify the flow of execution.

When writing loops in Python, sometimes you may need to change how the loop behaves while it runs. Two important statements that help you do this are break and continue. Both are used inside for and while loops. They help you stop a loop early or skip certain parts, giving you more control over your program's flow.

The **break** statement will immediately end the loop it is in, even if the loop did not finish all its cycles. On the other hand, the **continue** statement skips the rest of the code in the loop for the current turn, but lets the loop carry on with the next cycle.

Imagine searching for a number in a list. As soon as you find it, you do not want to loop through the rest of the list. The break statement lets you exit the loop straightaway. Similarly, if you want to ignore certain numbers (for example, all odd numbers), you can use continue to skip over them and move to the next number.

- break leaves the loop completely.
- continue skips to the next cycle of the loop.
- Both must be inside a loop (for or while).
- Can make your code more efficient and readable.

```
# Using 'break' to stop a loop early
numbers = [4, 8, 15, 16, 23, 42]
for number in numbers:
    if number == 15:
        print('Found 15! Stopping loop.')
        break
    print('Current number:', number)
```

```
# Using 'continue' to skip certain values
for number in numbers:
    if number % 2 == 0:
        continue # Skip even numbers
    print('Odd number:', number)
```

Statement	Effect
break	Ends the loop instantly, even if the loop condition is still true.
continue	Jumps to the next cycle of the loop, skipping the rest of the code for this turn.

Note: Important tip: Only use break and continue when needed. Too many can make your loops hard to read. Before using them, check if there is a clearer way to write your logic.

The break statement immediately exits the enclosing loop, skipping the rest of its code.

The break statement is used in Python to stop a loop before it would naturally finish. When Python executes a break statement inside a loop, it exits that loop straight away and jumps to the first line of code after the loop. This is helpful when you want to stop the loop early, for example if you have already found what you are looking for.

Suppose you are searching through a list of items. If you find the item you want, there is no reason to keep looking. The **break statement** lets you exit the loop the moment your search is successful, instead of checking every remaining item.

The break statement only applies to the loop it is directly inside. If you have several nested loops, break will exit the innermost one. The rest of the loop code is skipped, and the program continues on with what follows the loop.

- break can be used in for and while loops.
- break helps improve efficiency by stopping loops early.
- After break, no more statements in that loop will run.

```
colours = ['red', 'green', 'blue', 'yellow']
for colour in colours:
    if colour == 'blue':
        print('Found blue! Exiting loop.')
        break
    print('Checking:', colour)
print('Loop is finished.')
```


Loop Step	Output
First	Checking: red
Second	Checking: green
Third	Found blue! Exiting loop.
After Loop	Loop is finished.

Note: Important tip: Use break to avoid unnecessary work in loops, especially when you can finish a task as soon as a condition is met.

The continue statement skips the current iteration and jumps to the next iteration of the loop.

The continue statement is used inside loops in Python. When Python executes a continue statement, it immediately ends the current loop iteration. The program then moves on to the next iteration of the loop. The rest of the code inside the loop, after the continue statement, is skipped during that iteration.

You can use **continue** to create more efficient loops. This statement helps you avoid running unnecessary code. For example, you might want to skip certain values or outputs in your loop while still finishing all other iterations.

Imagine you are looping through a list of numbers and you want to print all numbers except the negative ones. The continue statement allows you to check for a condition (like being negative). If the condition is true, you skip to the next value in the list. This means you do not need extra nesting or complicated logic.

- continue can only be used inside loops, such as for or while.
- When continue is reached, any remaining code in the loop's body for that iteration will not run.
- The loop then moves to its next value or checks its condition again.

```
numbers = [3, -1, 7, 0, -5, 8]

for number in numbers:
    if number < 0:
        continue # Skip negative numbers
    print('Number:', number)
```

Iteration	Action
number = 3	Prints 'Number: 3'
number = -1	continue triggers, prints nothing
number = 7	Prints 'Number: 7'
number = 0	Prints 'Number: 0'
number = -5	continue triggers, prints nothing
number = 8	Prints 'Number: 8'

Note: Important tip: Avoid overusing continue. Too many continue statements in one loop can make code hard to read and debug. Use it when it makes your logic clearer and the loop easier to follow.

Both statements can help manage complex conditions and improve readability.

When writing programs, you often face situations where you need more control over how your loops behave. The `break` and `continue` statements are tools in Python that allow you to do just that. They help you handle special conditions inside loops, making your code neater and easier to follow.

The **`break`** statement will immediately stop the current loop and move on to the next statement after the loop. The **`continue`** statement, on the other hand, skips the rest of the code in the current loop iteration and starts the next round of the loop. Used well, these statements help you manage tricky situations, especially when the logic gets more complicated.

As your programs grow, you might find yourself checking for several different conditions within one loop. Adding extra `if` and `else` statements can quickly make things hard to read. The `break` and `continue` statements help you escape or skip unneeded steps, keeping your code shorter and more readable.

- **Avoid** deeply nested `if` statements.
- **Exit a loop** early when the needed condition is found.
- Skip unnecessary processing for some items in a list or range.
- Make your intentions clearer to someone else reading your code.

```
numbers = [1, 5, -3, 10, 0, 7]

for num in numbers:
    if num < 0:
        print('Found a negative number, stopping!')
        break
    elif num == 0:
```

```
print('Zero found, skipping this number.')
continue
print('Processing', num)
```

Statement	What it Does
break	Leaves the loop as soon as it runs
continue	Skips the rest of the current loop turn

Note: Important tip: Overusing break and continue may make your code harder to follow. Use them thoughtfully to simplify complex logic or reduce repeated conditions.

Use break and continue carefully to avoid creating confusing or infinite loops.

When writing loops in Python, you may want to change how the loop behaves based on certain conditions. Two statements, **break** and **continue**, can help control your loop's flow. However, using them without careful planning can make your program hard to understand or cause it to loop forever.

The **break** statement stops the loop immediately and moves to the next part of your code. The **continue** statement skips the rest of the loop for the current cycle and starts the next one. Using these statements is sometimes necessary, but too many—or placing them badly—can cause problems and confusion.

Using **break** and **continue** without clear reasoning can lead to unclear code. When others read your code (or when you revisit it later), you may struggle to follow how the loop moves and when it might finish. In the worst case, careless use can create infinite loops—loops that never end.

- **Use break and continue** to keep your loops simple and clear.
- Always check your loop has a way to finish (a stopping condition).
- Be wary of using these statements inside loops with many conditions, as it gets hard to see how the loop behaves.
- Try to use **meaningful variable** names and comments to make your intention obvious.
- Review code after adding **break** or **continue** to make sure the loop behaves as expected.

```
numbers = [5, 8, 0, -2, 9]
for n in numbers:
    if n == 0:
        print('Found zero! Stopping search.')
        break # Stops the loop if zero is found
    if n < 0:
        continue # Skips negative numbers
    print('Processing:', n)
```

Statement	Effect
break	Ends the loop immediately, even if there are items left.
continue	Skips the rest of the loop body for this cycle and moves to the next iteration.
No break or continue	Loop runs until the normal end or a condition is met.

Note: Important tip: After adding break or continue, test your code with different inputs. Make sure your loop always finishes and behaves as you expect. Avoid using these statements unless they make your code clearer or solve a real problem—they are helpful tools, not shortcuts.

PRACTICAL DECISION-MAKING LOGIC

Understand and implement decision-making in Python using if, elif, and else statements.

In programming, decision-making allows your code to choose what actions to take based on certain conditions. Python uses if, elif, and else statements to help you express these choices. These statements are essential for building programs that respond differently depending on input or data.

Consider the situation where you want to check a student's test score and provide feedback: **you need to decide which message to display based on the score.** This is a perfect time to use if, elif, and else statements.

In Python, an if statement tests a condition. If the condition is true, the indented code under if runs. elif (short for 'else if') provides additional conditions to test if the first if is false. else is used for all other cases when none of the previous conditions are true.

- if: Checks if a condition is true. Runs code only if it is true.
- elif: Checks another condition if the first if is false. You can have multiple elifs.
- else: Runs only if none of the above conditions are true.

```
score = 73

if score >= 85:
    print('Excellent! You scored an A.')
elif score >= 70:
    print('Well done! You scored a B.')
elif score >= 50:
    print('Good effort! You passed.')
else:
    print('Keep trying. You can improve next time!')
```

Statement	Purpose
if	Tests the first condition (score >= 85). Runs if true.
elif	Tests the next condition (score >= 70 or score >= 50) if previous tests fail.
else	Catches all other cases. Runs if all conditions above are false.

Indentation is vital in Python. All lines of code under if, elif, or else must be indented by at least one level, usually four spaces. This visually links the condition to the block of code it controls.

Note: Always use a colon (:) at the end of if, elif, and else lines. This tells Python a decision block is starting.

You can combine conditions using logical operators like and and or. For example, you might want to check if a number is between two values.

```
temperature = 22
if temperature > 20 and temperature < 30:
    print('The weather is pleasant.')
```

With this structure, your Python programs can react to many different scenarios and inputs. As you practise, you will see how if, elif, and else help your code make clear, logical decisions.

- **Use as many elifs** as needed.
- **else** is always optional.
- Order your conditions from most specific to most general.

Note: Test your decision-making logic with different values. This helps confirm that every branch behaves as you expect.

Work with nested conditionals to handle complex situations.

Nested conditionals allow you to express complex decision-making in your Python programs. When you nest conditionals, you place one if or elif or else statement inside another. This helps when you need to check multiple conditions in sequence, or when decisions depend on the outcomes of previous checks. By structuring your code in this way, you can handle situations where there are several layers of logic.

For example, imagine you are writing a program to check if a student has passed an exam. **First, you might check if the student sat the exam.** If they have, you then check their score to see if they passed or failed. If they did not sit the exam, you might decide to display a different message altogether. This situation is well-suited to nested conditional statements.

Nested conditionals should be used when you have a sequence of dependent decisions to make. Be mindful that nesting too deeply can make your code harder to read or maintain. It's best to nest only when each level of the decision is clearly necessary.

- Allows handling of scenarios with several related conditions.
- Each level of nesting represents a new branch in your logic.
- Nesting is typically used inside functions or within loops.
- Careless nesting can make code confusing to read.
- Indentation is important—each nested block needs its own indentation.

```
exam_sitting = input('Did the student sit the exam? (yes/no): ')
if exam_sitting == 'yes':
    score = int(input('Enter the student score: '))
    if score >= 50:
        print('The student passed!')
    else:
        print('The student did not pass.')
else:
    print('No exam record for this student.')
```

Condition	Outcome
Exam sat AND score >= 50	Pass message
Exam sat AND score < 50	Fail message
Exam not sat	No record message

Note: Important tip: Keep your nesting to a minimum to maintain clarity. If you notice your code going more than two or three levels deep, consider breaking it into functions or rethinking your logic.

Use for and while loops to repeat actions and process data collections.

Loops let you repeat actions in your Python programs. This is essential when you want to carry out similar tasks many times or process each item in a group of data. Python provides two main types of loops: for loops and while loops. Each suits different situations, and together they are powerful tools for controlling how your code runs.

A **for loop** is used when you want to repeat an action for every item in a collection, such as a list or a range of numbers. You usually know in advance how many times the loop will run.

For example, if you have a list of student names and want to print each name, you can use a for loop. This approach is straightforward and avoids repeating similar code many times.

- Use a for loop to process all items in a list or range.
- A for loop is ideal when you know the number of repeats.
- Common for loop targets include lists, strings, and other collections.
- You can access each item directly inside the loop.

```
students = ['Alice', 'Bob', 'Charlie']
for name in students:
    print('Hello,', name)
```

A **while loop** repeats actions as long as a certain condition remains true. It is most useful when you do not know in advance how many times a task needs to repeat, such as collecting user input until they type 'exit'.

With while loops, you must make sure there is a way for the loop to end. If the condition never changes to false, the loop will run forever, causing your program to get stuck.

- Use a while loop when you do not know how many repeats are needed.
- A while loop runs until its test condition becomes false.
- It is often used for input validation and waiting for a task to complete.
- Changing the loop condition inside the loop is important to avoid infinite loops.

```
user_input = ''
while user_input != 'exit':
    user_input = input('Type something (or "exit" to stop): ')
    print('You typed:', user_input)
```

Loop Type	Best Use Case
for loop	Repeating actions for every item in a known collection
while loop	Repeating an action until a condition becomes false
for loop with range()	Repeating a set number of times

Loops are not just for printing data. They are commonly used to calculate totals, find items, filter data, or update values in a list. For instance, you may use a for loop to count words in texts or a while loop to keep trying until you get the right answer.

Note: Important tip: Always check that your loops can end. For for loops, this happens automatically. With while loops, make sure the condition will eventually become false, or your program could hang.

Control loop execution with break and continue statements for efficient coding.

In Python programming, loops allow you to repeat sections of code until a condition is met. Sometimes, you need extra control over how a loop executes. This is where the break and continue statements come in. They help you manage the flow of your loops, making your code more efficient and easier to understand.

Suppose you are searching through a list of numbers. If you find the number you want, there's no point searching further. This is when the **break** statement is useful. On the other hand, if you want to skip over certain values and continue with the next loop cycle, you can use the **continue** statement. Both statements can make loops shorter and save time by avoiding unnecessary work.

The break statement lets you exit a loop as soon as a given condition is true. This avoids running through the rest of the items and can greatly improve program speed in some situations. The continue statement, by contrast, skips the current loop cycle and moves on to the next one. This is helpful when you want to ignore certain cases, such as invalid input values, and process the rest.

- **Use break to exit** loops early when a task is complete.
- **Use continue to skip** over specific loop cycles.
- **Break is often used for searching tasks** and stopping once found.
- **Continue** is useful for ignoring unusable, unwanted, or invalid data.
- Both help to write clean, readable, and efficient code.

```
# Example using break
numbers = [2, 4, 6, 8, 10]
for num in numbers:
    if num == 6:
        print("Found", num)
        break # Stops the loop when number 6 is found
    print("Checked", num)

# Example using continue
for num in numbers:
    if num % 4 == 0:
        continue # Skips numbers that are multiples of 4
    print("Processing", num)
```


Statement	Effect
break	Immediately exits the loop; no further cycles run
continue	Skips to the next loop cycle without running rest of the code in that cycle
No statement	Loop keeps running until the condition is false or the list ends

Note: Important tip: Use break and continue thoughtfully. Overusing them can make your code harder to read. Always test your loops to check they behave as expected and do not skip important steps.

Practice building programs that use control flow to solve real-world problems.

Control flow is at the heart of how programs make decisions and repeat actions. In Python, you use control flow statements like if, elif, else, for loops, and while loops to guide your program's path. This lets your code respond to different situations, repeat tasks, and solve practical problems.

Let's look at how control flow applies to **real-world programming tasks**. By combining conditions and loops, you can design solutions for everyday challenges, such as checking temperatures, managing shopping lists, or processing user data.

For example, you could write a program that asks a user for their age and then tells them if they are eligible to vote. You could also loop through a list of temperatures from a weather station and print a warning if any are too high.

- Use if, elif, and else to compare values and decide what happens next
- Use for loops to go through items in a list, set, or dictionary
- Use while loops when you want a process to repeat until a certain condition is false
- Use break and continue to control loops more precisely, such as exiting early or skipping some steps

```
# Example 1: Simple conditional logic
user_age = int(input('Enter your age: '))
if user_age >= 18:
    print('You are eligible to vote.')
else:
    print('Sorry, you cannot vote yet.')

# Example 2: Using a loop for a shopping list
shopping_list = ['milk', 'bread', 'eggs']
for item in shopping_list:
    if item == 'milk':
        print('Remember to check the use-by date for milk!')
    print('Buy:', item)
```

Control Flow Statement	Typical Usage Example
if / elif / else	Check which action to take depending on conditions
for loop	Repeat actions for each item in a collection
while loop	Keep repeating as long as a test is true
break	Exit the current loop early
continue	Skip to the next loop cycle

Note: Start simple. Build small programs using just a few conditions or loops. Test often. Then try combining them to solve bigger problems—like filtering names from a file or building a basic menu system.

4. Program Design with Pseudocode and Flowcharts

PROBLEM-SOLVING STRATEGIES

Structured problem-solving is essential before starting to write code.

Before you write any code, it is vital to understand the problem you want to solve. Jumping straight into programming can lead to confusion and mistakes. Adopting a structured approach helps you break complex issues into smaller, manageable parts. This ensures you create clear, effective solutions.

Structured problem-solving means following a logical process. You start by **analysing the problem**, considering possible solutions, planning your steps, then writing and testing your code. This method saves time and reduces errors later.

For beginners, it is easy to feel eager and start coding right away. However, skipping the problem-solving stage can make it harder to finish your project or debug issues. Carefully planning your approach makes the coding process smoother and leads to better results.

- Clarify what the problem is really asking.
- Break the task into smaller steps.
- Consider various ways to solve each step.
- Plan your solution before touching the keyboard.
- Test your thinking with diagrams or pseudocode.

```
# Bad approach: Jumping straight in
number = input('Enter a number: ')
if number % 2 == 0:
    print('Even')
else:
    print('Odd')

# Better approach: Think, plan, then code
# 1. Ask: What am I trying to do?
# 2. What steps do I need?
# 3. How do I check evenness?
number = int(input('Enter a number: '))
if number % 2 == 0:
    print('Even')
else:
    print('Odd')
```

Action	Purpose
Define the problem	Understand what needs to be solved
Break into steps	Manage complexity in smaller chunks
Write pseudocode or draw a flowchart	Visualise your thinking

Note: Important tip: Taking time to plan before you write any code will help you spot mistakes early, saving you frustration and effort later on. Use structured problem-solving every time you start a new project, no matter how small.

Break complex problems into logical steps using techniques like pseudocode and flowcharts.

Solving a programming problem can sometimes feel overwhelming, especially when the task is complex or unfamiliar. One of the best ways to approach such problems is to break them into smaller, logical steps. This strategy will help you to understand the problem clearly, avoid mistakes, and write code that is both accurate and maintainable.

Pseudocode and flowcharts are two popular techniques to assist in this process. **Pseudocode** is a way to describe the steps of your solution using plain English. It focuses on logic, not programming syntax. **Flowcharts** use diagrams with symbols and arrows to represent the flow of your solution visually. Both methods are valuable planning tools before you start writing real code.

For example, imagine you are asked to write a program that calculates the average mark from a list of student scores. Rather than jumping straight into code, you can use pseudocode to outline the required steps. Similarly, drawing a flowchart lets you visualise how the program will handle tasks such as input, calculation, and output.

- Identify the overall goal of the program.
- Divide the task into logical stages (e.g., input, processing, output).
- Write the major steps in pseudocode first.
- Draw a flowchart to map out the sequence of steps and decisions.
- Refine each step with more detail as needed before coding.

Pseudocode example:

```
1. Get the list of student scores
2. Set total to 0
3. For each score in the list
   a. Add score to total
4. Calculate average = total / number of scores
5. Display the average
```

Technique	Description
Pseudocode	Write the solution's steps in plain language to clarify logic.
Flowchart	A diagram to show the order of steps and decisions visually.
Decomposition	Divide the problem into smaller parts to simplify each step.

Note: Important tip: If you are stuck on a coding problem, step away from your keyboard and try writing pseudocode or drawing a flowchart first. This can help you spot gaps in your thinking and prevent errors when you begin actual programming.

Practice writing pseudocode to outline the main logic of your solution before implementation.

Writing pseudocode is an important step before jumping straight into coding. Pseudocode is a way to plan your program by describing its steps using simple, plain language. This helps you focus on the logic and structure of your solution, rather than worrying about the exact Python syntax from the start.

Pseudocode acts as a **blueprint** for your code. It allows you to organise your thoughts, check your approach, and refine your solution before you start writing real code. This practice is especially valuable for beginners, as it makes it easier to catch mistakes early and breaks complex problems into smaller, manageable steps.

When writing pseudocode, you do not need to follow strict rules. The main goal is to describe what your program will do in a step-by-step manner that you and others can easily follow. Use everyday language, but try to keep it structured and organised, much like you would with real code.

- **Write each major step** of your solution on a new line.
- **Use indentation to show actions** that happen within loops or if statements.
- **Keep your language** simple and clear.
- **Focus on the logic**, not the syntax.
- Use keywords like **IF, ELSE, FOR, WHILE** to describe control flow.

Example problem: Find the largest number in a list

Pseudocode:

```
SET largest to first item in list
FOR each item in list
    IF item is greater than largest
        SET largest to item
    END IF
END FOR
PRINT largest
```

Pseudocode Phrase	What It Means
SET x to y	Assigns value y to variable x
IF condition	Check if condition is true
FOR each item in list	Repeat actions for every item

Note: Important tip: Try reading your pseudocode aloud. If it sounds confusing, rewrite it! Clear pseudocode usually leads to clear code.

Use flowcharts to visually map the flow and decisions in your program.

Flowcharts are a visual tool used to outline the basic steps and decisions in a program before any code is written. They help you see the order of actions, possible choices, and results. By sketching out your logic, you can spot mistakes or missing steps early. This makes development smoother and helps others understand your thinking.

Flowcharts use standard shapes to represent steps—rectangles for actions and diamonds for decisions. **Arrows connect these shapes** to show the flow of your program. By following these lines, you trace how your program moves from one step to another, branching where choices exist.

For beginners, flowcharts make it easier to break problems into smaller chunks. You can test your understanding of a process without worrying about syntax. If a step seems unclear, you can adjust your flowchart until it describes your desired behaviour.

- **Clarifies** each action your program needs to take
- **Highlights** where decisions or branches happen
- Gives you a '**big picture**' overview of your logic
- **Makes it easier** to explain or discuss your approach with others
- **Helps you plan code** before you start typing

```

Example: Deciding if a number is even or odd
-----
[Start]
  |
[Input number]
  |
[Is number divisible by 2?] ---Yes---> [Print 'Even']
                                |
                                No
                                |
                                [Print 'Odd']
  |
[End]
  
```

Shape	Meaning
Oval	Start or end of the process
Rectangle	An action or instruction (e.g., 'Input number')
Diamond	A decision or question (e.g., 'Is number > 10?')
Arrow	Shows direction of flow between steps

Note: Flowcharts are great for planning, but do not need to be perfect works of art. Simple sketches can be just as effective as fancy diagrams. The goal is to clarify your process and make your code easier to write and understand.

Translate your pseudocode and flowchart designs directly into Python code to streamline the programming process.

When you design a program, you often start with a plan. This plan may take the form of pseudocode or a flowchart. Both help you lay out the steps of your solution before you write any real code. By translating your pseudocode and flowchart designs directly into Python, you make the development process smoother and less error-prone.

Think of **pseudocode as the written instructions** for your program, using plain English or a mix of English and programming terms. A flowchart shows the same ideas as a diagram, with arrows and shapes to illustrate the flow of logic. These tools help you break down complex problems into easy-to-understand pieces.

Translating your design directly into Python involves following each step in your pseudocode or each arrow in your flowchart and writing the matching Python code underneath. This method ensures that you do not forget any steps and that your code matches your intended logic. It also makes it easier to spot mistakes early, rather than after you have written lots of code.

- **Start with your completed pseudocode** or flowchart.
- **Read each step** and think about how it looks in Python.
- **Write Python code for each part**, keeping the logic clear.
- **Test small sections** as you go to catch errors early.
- **Check your final program** against your original design.

```
# Pseudocode:
# 1. Ask the user for a number
# 2. Double the number
# 3. Print the result

number = int(input('Enter a number: '))
doubled = number * 2
```

```
print('Double is', doubled)
```

Pseudocode Step	Python Code Example
Ask the user for a number	<code>number = int(input('Enter a number: '))</code>
Double the number	<code>doubled = number * 2</code>
Print the result	<code>print('Double is', doubled)</code>

Note: Important tip: Break each step of your design into simple lines of code. This will make both writing and debugging your programs easier.

BREAKING PROBLEMS INTO STEPS

Start by analysing the problem and dividing it into manageable steps.

Before you begin writing any Python code, it is essential to carefully analyse the problem you are trying to solve. This first step ensures that you fully understand what needs to be achieved, reduces mistakes, and saves you time later on. Good problem analysis also helps you organise your thoughts and approach complex tasks with confidence.

Start by reading the problem description thoroughly and **identifying the main goal**. Ask yourself: What must the program achieve? Then, look for key details—all inputs, outputs, and any rules the solution must follow.

The next step is to divide the problem into smaller, easier steps. This technique is called decomposition. By breaking the task up, each piece becomes easier to tackle on its own. It also makes your solution clearer, easier to test, and less likely to contain hidden errors.

- Carefully read the problem statement.
- Identify what data will be used (inputs) and what results are needed (outputs).
- List the main tasks required to solve the problem.
- Divide each main task into sub-tasks, if needed.
- Arrange the steps in a logical order.

Example Problem: Calculate the average of three exam scores.

1. Ask the user for three scores.
2. Add the three scores together.
3. Divide the sum by three.
4. Display the average.

Step	Description
1	Get the scores from the user
2	Add the scores together
3	Calculate the average
4	Show the result

Note: Important tip: Take your time analysing the problem at the start. Rushing this step can result in wasted effort or a program that does not solve the right problem. Treat each part as a small puzzle to solve, and do not move on until you are confident about every step.

Use pseudocode to outline the logic of each step before writing any actual code.

Pseudocode is a simple way to plan your program before you start writing any actual Python code. It helps you break down a problem into clear, logical steps, making the later coding process much easier. Pseudocode is not specific to any programming language and does not use precise syntax. Instead, it uses plain English to describe what needs to happen in your program.

When you use pseudocode, **you focus on the logical sequence of tasks** instead of worrying about the specific details of Python right away. This can help prevent mistakes and confusion, especially when you're learning.

For example, imagine you want to write a program that asks the user for their name, then prints a personalised greeting. Before writing any code, you can use pseudocode to plan the steps you need. Think about what needs to happen, one piece at a time, and write down those actions in a clear, everyday language.

- Clarifies what the program should do, step by step
- Makes it easy to discuss your approach or ask for feedback
- Helps spot missing or unclear requirements before coding
- Reduces errors that come from jumping straight into coding
- Can be reused as comments when you write actual code

Pseudocode example for a greeting program:

```
1. Display a message asking for the user's name
2. Read the user's input and save it
3. Display a greeting message that includes the user's name
```

Action	Pseudocode Step
Get user input	Ask for the user's name
Store input	Save the name in a variable
Display output	Show a personalised greeting

Note: Important tip: Pseudocode should be clear and detailed enough so that anyone can read it and understand the planned logic, even if they have never seen Python before.

Create flowcharts to visually represent the process flow and decision points.

Flowcharts are diagrams that use shapes and arrows to show the steps and decisions in a process. They offer a clear, visual way to map how a program or system should work before any code is written. By seeing each step laid out, you can spot problems early and ensure every part of your program is considered.

When you use **flowcharts**, you make the logic of your program easy to follow. Anyone reading the diagram can understand the overall process and where decisions are made. This is especially useful when working in a team or explaining your ideas to others.

A flowchart contains different symbols. Each symbol has a meaning. For example, ovals represent starting or ending points, rectangles show actions or processes, and diamonds mark decision points. Arrows link these symbols, showing the direction in which the process flows.

- Oval: Begin or end the process (Start/End)
- Rectangle: Action or instruction (e.g., calculate a sum)
- Diamond: Decision point (yes/no questions or tests)
- Arrows: Show the flow from one step to the next

```

Start
|
v
Input number
|
v
Is number even?
/      \
Yes      No
|          |
Print 'Even'  Print 'Odd'
|          |
\          /
v
End
  
```

Symbol	Purpose
Oval	Marks start or end of the flowchart
Rectangle	Shows a task or action step
Diamond	Represents a decision or yes/no question
Arrow	Indicates direction and sequence of flow

For example, say you want to decide if a number entered by a user is even or odd. The flowchart starts with an oval, moves to an input step, checks the number with a diamond (Is number even?), and then splits: print 'Even' if yes, or print 'Odd' if no. Finally, both paths end at a shared terminating point.

Flowcharts help prevent missing steps. They also clarify where choices must be made and how different outcomes are handled. If you need to update or improve your process later, you can adjust the flowchart first, making changes before touching your code.

Note: Important tip: Keep flowcharts simple and tidy. Use clear labels and be consistent with your symbols. This makes the diagrams easy for anyone to read, check, and use for coding later.

Ensure each step logically follows from the previous one to create a clear solution path.

When tackling a programming problem, it is important to break it into smaller steps. However, those steps must fit together in a logical sequence. Each action should naturally lead to the next. This helps create a solution path that is easy to follow, understand, and implement.

Think of designing a program as writing a set of instructions for someone else. If you jump between steps or miss important connections, the reader (or the computer) might become confused. **Each step should depend on the outcome of the previous one.** This creates a flow that is straightforward to interpret.

In programming, a clear solution path avoids mistakes and makes debugging much easier. Illogical or out-of-order steps can produce errors or unexpected results. Logical sequencing also makes your code easier to improve or update in the future, since anyone can trace the reasoning from start to finish.

- Begin with a clear definition of the problem.
- Identify the main tasks that must be performed.
- Order the tasks so each one prepares for the next.
- Double-check that no steps are skipped or repeated.
- Test your sequence with different scenarios.

```

1. Start
2. Input the user's name
3. Greet the user by name
4. Ask the user for a number
5. Double the number
6. Output the result
7. End
  
```

Step	Purpose
Input user's name	Collect information needed for greeting.
Greet user	Use the name from previous step.
Ask for number	Start calculation process.
Double number	Process the user's input.
Output result	Show final answer based on previous step.

Note: Always review your step sequence. Try explaining your plan aloud or to a friend. If the logic does not flow smoothly from one step to the next, adjust your design before you start writing code.

Translate your pseudocode and flowchart designs into Python programs systematically.

After you design your solution using pseudocode and flowcharts, the next step is to write your Python program. This process involves carefully following your design and implementing each step using Python syntax and programming techniques.

Start by **reviewing your pseudocode and flowchart thoroughly**. Make sure you understand each part of your design. The goal is to translate each logical step into actual code, one piece at a time.

Think of your pseudocode or flowchart as a map. Use it to guide your programming. Do not try to write all the code at once. Work through your design line by line, or shape by shape, turning each one into Python code. This method makes your work organised and reduces mistakes.

- Start coding the steps in order, just as in your pseudocode or flowchart.
- Test your code often to make sure each part works before moving to the next.
- If you get stuck, go back and check your design for errors or missing details.
- Keep your code tidy and use comments to show where each step from your design happens.
- Once all the steps are coded, run your program to make sure it matches your design outcomes.

```
# Pseudocode Step: Get two numbers from user
num1 = input('Enter the first number: ')
num2 = input('Enter the second number: ')

# Pseudocode Step: Convert to integers
num1 = int(num1)
num2 = int(num2)

# Pseudocode Step: Add numbers
sum_result = num1 + num2

# Pseudocode Step: Show the result
print('The sum is:', sum_result)
```

Design Step	Python Example
Get input	name = input('What is your name?')
Decision	if age >= 18: print('Adult')
Repeat	for i in range(5): print(i)

Note: Important tip: Translating your design step-by-step reduces errors and makes your code easier to check and improve. Never skip the planning phase, even for small problems.

WRITING PSEUDOCODE

Pseudocode is a way to plan your program using simple, English-like statements before writing actual Python code.

Pseudocode helps you plan your program before you start writing any code. Instead of using a programming language like Python, you write simple statements in plain English. This lets you focus on the logic of your solution, rather than worrying about Python syntax at the start.

When you use pseudocode, you describe what your program should do, step by step, using easy-to-understand language. This approach helps you **break down complex problems into smaller pieces**, making it much easier to organise your thoughts and plan an effective solution.

Pseudocode does not have strict rules. You can use words and phrases you are comfortable with, as long as each step is clear and easy to follow. Think of pseudocode as a rough draft. It allows you to experiment and try different solutions quickly, without worrying about errors caused by incorrect syntax.

- Pseudocode is written in plain English and is easy to read.
- It helps you map out the logical steps of your program.

- You do not need to follow a fixed structure, but clarity is important.
- It saves time by letting you spot problems before you start coding.
- Pseudocode is especially useful for beginners to organise ideas.

Example: Pseudocode for finding the largest number in a list

```
Set largest to first number in the list
For each number in the list:
    If number is bigger than largest:
        Set largest to number
Print largest
```

Pseudocode Statement	Description
Set total to 0	Create a variable named total and set it to zero
For each item in the list:	Repeat the following steps for each item
If value is greater than 10:	Check if a value is bigger than 10

Note: Important tip: Write pseudocode as if you are giving instructions to someone else. If your steps are clear to others, they will be easy to translate into Python code later.

Writing pseudocode helps you organise your thoughts and clarify each step needed to solve a problem.

Pseudocode is a simple way to plan your program before you start writing actual code. It uses plain language to describe the steps your program will take, without following strict programming syntax. Writing pseudocode guides you through your problem-solving process one step at a time.

When you write pseudocode, you focus on the **logic and sequence** of your solution instead of worrying about programming details like brackets or exact keywords. This makes it easier to organise your ideas clearly.

Many new programmers try to start directly with code. This often leads to confusion or getting stuck on syntax errors. By using pseudocode, you create a clear plan first, breaking down a problem into logical steps. Each step is written simply, focusing on what needs to be done, not how Python or any other language would write it.

- Helps you stay focused on the problem, not the language.
- Makes it easier to spot mistakes in your logic early.
- Saves time when you start coding because you already have a plan.
- Works as a communication tool if you're working with others.
- Helps you see if a step is missing or out of order.

Example task: Find the largest number in a list

Pseudocode:

```
START
SET largest to first number in the list
FOR each number in the list
  IF number > largest
    SET largest to number
  END IF
END FOR
PRINT largest
END
```

Pseudocode Step	Explanation
SET largest to first number in the list	Initialise the largest variable as the first item.
FOR each number in the list	Check every number, one at a time.
IF number > largest	Compare current number to the largest value.
SET largest to number	Update largest value if you found a bigger number.
PRINT largest	Show the final result after checking all numbers.

Note: Important tip: Pseudocode does not have a fixed format. Use simple and clear instructions, and don't worry if it looks similar to English or a mix of basic programming ideas.

Use pseudocode to describe input, processing, and output clearly, focusing on what the program should do.

Pseudocode is a simple way to plan your programs before you start writing actual Python code. It uses plain language to describe the steps your program will take. Good pseudocode focuses on what needs to happen, not how it will be done in code.

In any program, you usually need to deal with three key things: **input** (what data your program receives), **processing** (the actions your program takes), and **output** (the results your program produces).

Start your pseudocode by describing, in clear steps, what input you will need. Next, outline the processing that must be done to the input. Finally, describe what your program should output. This approach helps you organise your thoughts and makes your code easier to write.

- List the data your program will receive (input).
- Describe the steps your program will perform (processing).
- Specify what information your program will show or return (output).

Example: Find the largest of three numbers

```
START
Get first number from user
Get second number from user
Get third number from user
IF first number is greater than both second and third number
    Largest is first number
ELSE IF second number is greater than third number
    Largest is second number
ELSE
    Largest is third number
Display the largest number
END
```

Section	Description
Input	Three numbers from the user
Processing	Compare the numbers to find which is largest
Output	Print the largest number

Note: Write your pseudocode as if you are giving clear instructions to another person. Focus on the logic, not Python details. You can use phrases like GET, IF, ELSE, and DISPLAY to explain what happens.

Pseudocode should be detailed enough for you (or someone else) to easily translate it into Python code.

Pseudocode is a way to plan your program before writing the actual Python code. It uses plain English (or your preferred language) to describe the steps your program should follow. Good pseudocode is clear and detailed, making it straightforward for yourself or someone else to write working Python code from your plan.

The main goal of pseudocode is to **describe the logic of your solution in enough detail that translating it into Python becomes a simple mechanical process**. If your pseudocode is too brief, you might forget important steps or struggle during coding. If it is too vague, it may lead to uncertainty or mistakes.

Well-written pseudocode does not use real code syntax. Instead, it explains what needs to happen, step by step, using clear statements. Think of it as telling another person how to solve the problem, one action at a time. Each line should capture a single operation or decision.

- Use clear, simple language—avoid technical jargon.
- Describe every important operation, including calculations and decisions.
- Make sure each step leads naturally to the next—there should be no large logical jumps.
- Include any inputs, outputs, and major variables.
- Use indentation to show structure for loops and conditionals.

Example Task: Find the average of three numbers

Pseudocode:

1. Set total to 0.
2. Get first number from user; add to total.
3. Get second number from user; add to total.
4. Get third number from user; add to total.
5. Set average to total divided by 3.
6. Display average.

Pseudocode	Equivalent Python
Get number from user	<code>number = float(input("Enter a number: "))</code>
Add to total	<code>total += number</code>
Display result	<code>print(average)</code>

Note: Important tip: Imagine handing your pseudocode to someone else. Could they write the Python code without asking questions or making guesses? If not, add more detail.

Practice writing pseudocode regularly to develop strong problem-solving and programming habits.

Pseudocode is a simple way to plan your programs using plain English. It is not proper code, but a description of your logic, written step by step. By practising pseudocode, you develop a habit of thinking clearly about problems before you start coding. This makes you better at solving problems and helps prevent mistakes in your code.

Regular practice with pseudocode trains your mind to **break complex tasks into smaller, manageable steps**. This skill is crucial for any programmer, whether you are just starting or have years of experience.

Consider pseudocode as a map for your program. If you skip this stage, it is easy to get lost or make errors that are hard to find. When you make it part of your routine, it becomes second nature to outline your solution before opening your code editor.

- Improves your ability to organise and visualise solutions before you write code
- Makes it easier to explain your logic to others, like teammates or instructors
- Helps you debug programs by making your intentions clear
- Speeds up coding as you have a plan to follow
- Encourages careful thinking which reduces simple mistakes
- Builds confidence, especially if you are new to programming

```
START
Set total to 0
For each number in the list
    Add the number to total
END FOR
Print the total
END
```

Pseudocode Habit	Benefit
Daily practice	Builds routine and confidence
Reviewing others' pseudocode	Improves understanding and teamwork
Writing before coding	Reduces errors and confusion

Note: Important tip: Try to write pseudocode for every coding task, no matter how small. Even simple outlines help you clarify your approach and identify any gaps before you start coding.

DESIGNING FLOWCHARTS

Use flowcharts to visually map out the steps of a program before coding.

Flowcharts are diagrams that visually represent the steps and decisions involved in a process. In programming, you use flowcharts to organise your thinking and plan out your program before you begin writing code. This technique is especially helpful for beginners because it makes complex logic easier to understand at a glance.

By using a flowchart, **you create a clear, step-by-step map of how your program will work.** This helps you identify problems early, avoid mistakes, and communicate your ideas to others. Flowcharts use standard shapes and arrows to show the flow and decisions in a process.

Before you write any Python code, it's a good idea to 'draw' your program with a flowchart. Doing so shows you the big picture. It lets you see what needs to happen, in what order, and where choices or loops occur. If you spot issues or find better ways to structure your program at this stage, it's much easier—and less time-consuming—to make changes here than after you've written lots of code.

- Flowcharts use shapes like ovals, rectangles, diamonds, and arrows.
- Ovals usually represent Start or End points.
- Rectangles show actions or steps (such as calculations).
- Diamonds are for decisions (yes/no or true/false questions).
- Arrows point from one step to the next, showing the sequence.

Example: Imagine you want to design a simple program that checks if a person's age allows them to vote.

Here is how you might map it out as steps before coding:

1. Start
2. Ask for age
3. If age $\geq$ 18?
 Yes: Print 'You can vote.'
 No: Print 'You cannot vote.'
4. End

Flowchart Shape	Purpose
Oval	Start or End the program
Rectangle	Process step or action
Diamond	Decision or condition
Arrow	Shows flow or direction

Note: Important tip: Even a rough, hand-drawn flowchart can help you stay organised and spot issues early. Don't worry about making it perfect. Aim for clear, logical steps.

Include input, output, decision, and processing symbols to represent different actions in your flowchart.

Flowcharts use symbols to show each step or action in a process. These symbols help you quickly understand what happens at each stage, making your plans easy to read and follow. By choosing the correct symbols for input, output, decisions, and processing, you create a clear map for your Python programs.

There are four core symbols in most flowcharts: input, output, decision, and processing. **Input** symbols show where your program gets information. **Output** symbols display your results or feedback. **Decision** symbols represent yes/no or true/false choices. **Processing** symbols are used for calculations, actions, or steps that change data.

Using the right symbol for each step allows anyone to follow your logic. This is especially important when working in a team or when you want to turn your flowchart into code later. Each symbol type highlights what kind of action is taking place, reducing confusion and mistakes.

- **Input/Output:** Parallelogram symbol. Use for reading data or displaying results.
- **Processing:** Rectangle symbol. Use for calculations, assignments, or changing values.
- **Decision:** Diamond symbol. Use for branching or making choices based on conditions.
- **Start/End:** Oval symbol. Mark the beginning and end of your process.

```
Oval:           Start/End
Parallelogram:  Input/Output (e.g., get user input, print result)
Rectangle:      Processing (e.g., add numbers, assign values)
Diamond:        Decision (e.g., Is score > 50?)
```

Symbol	Purpose
Parallelogram	Input or Output
Rectangle	Processing Step
Diamond	Decision Point
Oval	Start or End

Note: Choose your symbols carefully. A clear flowchart makes coding in Python much simpler and helps you spot logical errors early.

Follow standard flowchart symbols and connect steps logically with arrows.

Flowcharts are visual diagrams that help you plan how a program or process works before you start coding. To make flowcharts clear and useful, you need to use recognised symbols and connect them in a logical order. This makes it easy for anyone reading the flowchart to understand your thinking and follow the steps you plan to implement.

Each symbol in a flowchart has a specific purpose. **Using standard symbols is essential** because it avoids confusion and keeps communication clear across teams. The main symbols are used for starting or ending the process, carrying out actions, making decisions, and displaying input or output.

Once you have chosen the correct symbols, you must connect them so that each step flows logically into the next. This is where arrows come in. Arrows indicate the direction of the flow, guiding the reader from the beginning of the process to the end. Each decision or action should make sense in the sequence shown.

- **Oval (Terminator):** Shows the start or end of a flowchart.
- **Rectangle (Process):** Represents an action, such as a calculation or instruction.
- **Diamond (Decision):** Used whenever a yes/no or true/false choice is made.
- **Parallelogram (Input/Output):** Indicates data entering or leaving the system.
- **Arrows:** Connect symbols in the correct order to show the path taken.

```

Start (Oval)
  |
Enter Number (Parallelogram)
  |
Is Number > 0? (Diamond)
  |
Yes          No
  |          |
Print 'Positive'  Print 'Not positive' (Rectangle)
  |          |
  -----
          |
          End (Oval)
  
```

Symbol	Meaning
Oval	Start or End point
Rectangle	Action or process
Diamond	Decision point
Parallelogram	Input or output
Arrow	Flow direction

Note: It helps to keep your flowchart tidy with symbols the same size and arrows neatly connecting each step in order. Make sure every path leads to a clear end point.

Refine your flowchart to ensure it accurately reflects the logic and sequence of your solution.

After creating your initial flowchart, it is important to revisit and refine it. Refinement is the process of reviewing your diagram to ensure it truly matches the steps you want a computer program to follow. This means checking that actions are in the right order, decisions are clear, and no steps are missing or repeated. A clear and accurate flowchart saves time when coding, as it uncovers problems before you start programming.

Start by walking through your flowchart **step by step, imagining you are the computer**. Check if each action and decision leads correctly to the next. Think about what would happen if each branch is followed. This helps you spot missing paths, loops, or misplaced steps.

Imagine you are designing a flowchart for a program that checks whether a number is even or odd. During refinement, you might notice that you forgot to handle what happens if the input is not a valid number. Adding that check improves your logic, ensuring the sequence covers all real-world scenarios.

- **Look for any missing steps** or unclear decisions.
- **Ensure each shape follows the correct flowchart** conventions.
- Make sure **every possible case leads to an outcome**.
- **Check for errors** like unreachable steps or unnecessary loops.
- **Confirm** that the starting point and end point are clear.

```

Start → Input number → Is input a valid number? →
    |Yes                |No
    v                  v
Is number even?      Display error
    |Yes    |No      |
    v      v        |
  
```

```

  Display 'Even'   Display 'Odd'
    |
  End
  
```

Common Mistake	How to Refine
Skipping input validation	Add checks for valid user input
Confusing step order	Reposition actions logically
Loops with no exit	Add a condition to stop repeating
Duplicate paths	Combine similar branches or steps
Unclear outcomes	Label all outputs clearly and end paths properly

Note: Important tip: Ask someone else to review your flowchart. A fresh pair of eyes often spots unclear logic or missing steps that you might overlook.

Use your completed flowchart as a reference when translating your design into Python code.

Once you have finished designing a flowchart for your program, it becomes a valuable tool for writing your actual Python code. The flowchart visually represents the logical steps of your solution and helps you keep track of how information moves and changes. By referring to the flowchart while you write your code, you can follow your plan closely, avoid skipping important steps, and reduce the chance of introducing errors.

A flowchart **shows the order and structure** of your solution at a glance. Decision points, loops, and sequence are clear in the diagram, making it easier to convert these parts into code. As you work, move from one step of your flowchart to the next, and translate each action or decision directly into Python syntax.

For example, if your flowchart starts with input from the user, you would start your code by collecting that input. If the next step is a decision (such as checking if a number is positive or negative), you write an if statement. By moving step by step, you ensure that your code follows your planned logic.

- **Start by identifying each step** in your flowchart.
- **Write Python code** for each operation or action box.
- **Convert decision points** (diamonds) to if, elif, or else statements.
- **Represent loops** as while or for loops in code.
- **Align your code's structure** with the flowchart's flow.

```
# Example: Flowchart for checking if a number is even or odd
number = int(input("Enter a number: "))
if number % 2 == 0:
    print("This number is even.")
else:
    print("This number is odd.")
```

Flowchart Shape	Python Equivalent
Oval (Start/End)	Start/stop program
Parallelogram (Input/Output)	input(), print()
Rectangle (Process)	Assignment, calculations, function calls
Diamond (Decision)	if, elif, else
Loop Symbol / Arrow	for, while

Note: Important tip: If you get stuck when coding, look back at your flowchart. It reminds you of the logic and helps you find where you might have gone off track. Always update your flowchart if you change your approach, so your design and code stay in sync.

TRANSLATING DESIGN INTO PYTHON CODE

Break down programming problems into logical, manageable steps before writing code.

Before you start typing Python code, it's vital to organise your thinking. Taking time to break a programming problem into logical, manageable steps will help you plan a clear path from the problem statement to the solution. This habit reduces errors, makes coding easier, and often saves time.

Imagine you need to write a program that asks a user for their age and then tells them if they are eligible to vote. Jumping straight into code might seem tempting. However, it's best to **divide the problem into smaller steps** first. These steps act as your roadmap for development.

When you break problems into steps, you clarify what the program must do. Each step describes one part of the process, making it easier to write and test code for that part. This approach is called 'decomposition'. It is especially helpful for large or complex tasks, but even small programs benefit from it.

- **Understand the problem:** Read the requirements carefully.
- **Identify inputs:** What information does your program need from the user or other sources?
- **List outputs:** What results does your program need to produce?

- **Decide on processing steps:** How will you transform inputs into outputs?
- **Arrange the steps logically:** What needs to happen first, second, and so on?

Problem: Given a list of numbers, print the largest number.

Breakdown:

1. Get the list of numbers from the user
2. Compare each number to find the largest
3. Print the largest number

Step	Example (Find Largest Number)
Input	Ask for a list of numbers
Process	Go through each number, keep track of the largest
Output	Display the largest number found

Note: Important tip: Use pen and paper or a text editor to list the steps and sketch out your plan before you code. This helps you spot missing pieces and avoid mistakes early.

Develop solution outlines by writing pseudocode to map out your algorithm.

Before you write any Python code, it helps to have a clear plan. Writing pseudocode is one of the best ways to outline your solution to a problem. Pseudocode uses short sentences and phrases—sometimes mixed with a little bit of programming-style structure—to describe what your program should do, step by step, without following any strict programming language rules.

Writing pseudocode is **like creating a skeleton of your program**. You break down your ideas into smaller, logical steps. This lets you focus on the problem-solving process instead of worrying about Python's exact syntax early on.

This approach is helpful for beginners because it makes the path between 'problem' and 'code' much clearer. You can spot logical errors, missing steps, or unclear instructions while working in pseudocode. It is also a great way to discuss your approach with teachers, classmates, or colleagues, since pseudocode is meant to be easy to read.

- **Pseudocode** is written in plain English (or your native language if preferred).
- You can use simple instructions like **'IF', 'FOR EACH', 'PRINT', or 'READ'** to make intentions clear.
- You do not need to follow any programming language rules—**just focus on the algorithm's flow.**

Example Problem: Find the average of five numbers entered by the user

Pseudocode:

1. Set total to 0
2. Repeat 5 times:
 - a. Ask the user for a number
 - b. Add the number to total
3. Divide total by 5 to get the average
4. Display the average

Pseudocode Instruction	Purpose
Set total to 0	Initialise the running total
Ask the user for a number	Get input from the user
Add the number to total	Update the total with each new input
Divide total by 5	Calculate the average
Display the average	Show the result to the user

Note: Important tip: You do not have to be perfect with your pseudocode. As long as it clearly shows the main steps, it will help you organise your thoughts and write better Python code later.

Create flowcharts to visually represent the flow of your program's logic.

Before you start coding, it is helpful to map out your program's logic using flowcharts. Flowcharts are diagrams that show the sequence of actions, decisions, and processes within a program. These diagrams use standard shapes and arrows to represent the flow of control and data. Using flowcharts makes it easier to plan your solution, communicate ideas, and identify potential mistakes early.

A flowchart begins at a **Start** point and ends at a **End** point. In between, you can show steps such as processing actions, decisions, input/output operations, and loops. Each shape has a standard meaning that makes your chart easy to understand by others and by yourself later on.

There are a few basic shapes you will use in flowcharts. An oval is used for Start and End. Rectangles represent process steps, like calculations or assignments. Diamonds are used for decisions, where you branch depending on a true/false condition. Parallelograms show input and output actions, such as displaying results or reading user input.

- **Ovals** – Start and End points.

- **Rectangles** – Processing steps, such as calculations.
- **Diamonds** – Decision points; branching based on conditions.
- **Parallelograms** – Input and output operations.
- **Arrows** – Show direction of flow from one step to the next.

Example: Flowchart for checking if a number is odd or even

```

[Start]
  |
[Input Number]
  |
[Is Number % 2 == 0?]
  |       |
[Yes]     [No]
  |       |
[Print "Even"] [Print "Odd"]
  |       |
[End]     [End]
  
```

Shape	Used For
Oval	Start and End points
Rectangle	Process steps (e.g., assign a value)
Diamond	Decision points (e.g., if/else)
Parallelogram	Input/output (e.g., print, ask user)
Arrow	Direction of flow

Note: You do not need any special software to draw a flowchart. Paper and pencil work well for quick designs. For neater diagrams, use free tools like draw.io or Lucidchart.

Flowcharts are especially helpful when working on larger problems or with others. You can walk through each step, make changes easily, and spot issues without rewriting code. If you practise creating flowcharts before coding, your final Python programs will be more organised and easier to debug.

Learning to use flowcharts effectively develops your logical thinking. It also helps you communicate your ideas to teammates or instructors, even if they are not familiar with Python. As a beginner, mapping out your ideas visually takes the guesswork out of planning and lays a strong foundation for smooth coding.

Use your pseudocode and flowcharts as guides to write clear and accurate Python code.

Pseudocode and flowcharts are powerful planning tools. They help you organise your thoughts and map out the steps needed to solve a problem, before you begin writing any code. By creating these designs first, you can avoid confusion and reduce errors when you move onto actual coding. This makes programming easier and helps you feel more confident as you build solutions.

When you start writing Python code, always have your pseudocode or flowchart open nearby. **Use them as reference points** for each step you need to implement. This ensures your code closely matches your design and follows a logical sequence.

Start by translating each step from your pseudocode or flowchart into Python, one at a time. Focus on making your code simple and easy to read. This will help others (and your future self) understand how your program works. Always check back to your design to make sure you are not missing any key steps.

- Begin with a clear design using pseudocode or a flowchart
- Write code for one step at a time, following your plan
- Test as you go to ensure your code matches your design
- Return to your design if you get stuck or confused
- Edit your design or code if you notice improvements

```
# Example pseudocode:
# 1. Ask user for a number
# 2. Add 10 to the number
# 3. Print the result

number = int(input('Enter a number: '))
result = number + 10
print('The result is', result)
```

Pseudocode/Step	Python Code Statement
Ask user for a number	number = int(input('Enter a number: '))
Add 10 to the number	result = number + 10
Print the result	print('The result is', result)

Note: Important tip: If your code does not work as expected, double-check your pseudocode or flowchart. These designs can help you spot missing steps or mistakes.

Review and test your Python code to ensure it correctly implements your original design.

Once you have written Python code based on your pseudocode or flowchart, you must review and test it carefully. This helps you check that your program behaves as expected and matches your original design. Regular reviews and testing are essential for finding mistakes early and making sure your solution solves the problem correctly.

Begin by **reading your code closely to compare it with your design**. Look for missing features, misplaced logic, or errors. Does each part of your code match the steps in your pseudocode or flowchart? If anything is out of order, incomplete, or missing, make notes and adjust the code until it aligns with your design.

Testing your code means running it with different inputs to see what happens. Try using values you expect as well as those you do not expect. This helps you find and fix errors such as logic mistakes or incorrect outputs. The aim is to catch mistakes on the screen before anyone else sees them. Testing also builds your confidence that your code works as it should.

- Check each code section matches your design steps.
- Test normal (expected) inputs and edge cases.
- Watch for error messages or unexpected behaviour.
- Compare your program output with what you planned.
- Adjust your code when you find differences or mistakes.

```
# Example: Program to calculate total price with a discount
# Original design: Get price, apply discount if price > 100, show
# final amount

price = float(input('Enter price: '))
if price > 100:
    price = price * 0.9 # 10% discount
print('Final price:', price)
```

Test Input	Expected Output
150	Final price: 135.0
100	Final price: 100.0
50	Final price: 50.0

Note: Important tip: Always compare your program output with your design or test plan. This helps you spot small mistakes before they become big problems. Testing regularly leads to more reliable and maintainable code.

5. Functions and Modular Programming

DEFINING AND CALLING FUNCTIONS

A function is a reusable block of code designed to perform a specific task.

In Python, a function is a named group of instructions that operate together to achieve a particular result. Functions allow you to structure your code by separating tasks into manageable sections, making your programs clearer and easier to maintain.

Functions help you avoid repeating the same code in different places. When you notice a task that will be needed more than once, it's a good idea to **create a function** for that task. This technique is called 'code re-use', which makes your programs shorter, easier to read, and less prone to errors.

To define a function in Python, you use the `def` keyword, followed by the function's name and a pair of brackets. Inside the brackets, you can list any information the function needs, known as parameters. The code block inside the function runs whenever you 'call' or 'invoke' the function.

- Functions let you name a behaviour you need more than once.
- Each function can take inputs (called arguments) and return outputs (known as return values).
- Dividing your code into functions helps with problem-solving, collaboration, and future improvements.

```
def greet(name):
    print("Hello, " + name + "!")

greet("Alice")
greet("Bob")
```

Benefit	Description
Reusability	Write the code once, use it many times
Clarity	Each function has a clear, single purpose
Maintainability	Easier to update just one function when behaviour needs to change

Note: Think of a function like a recipe: you give it some ingredients (inputs), it follows the steps, and it produces a result. You can use the same recipe as many times as you like, just by giving it different ingredients.

Functions are defined using the 'def' keyword, followed by the function name and parentheses.

In Python, functions allow you to organise your code into reusable blocks that perform specific tasks. To define a function, you use the 'def' keyword, a unique name for your function, and a pair of parentheses. This helps make your programs more organised, readable, and easier to maintain.

The **def** keyword tells Python you are creating a new function. After **def**, give your function a name without spaces, then add parentheses. For example:

Function names should be descriptive and use lowercase letters, sometimes with underscores to separate words. This helps others understand your code and makes it easier to read.

- Use **'def'** to start your function definition.
- Choose a name that **describes** what the function does.
- Add **parentheses** after the function name (these can contain parameters).
- End the line with a **colon** to start the function block.
- Write the **function code** indented underneath.

```
def greet():  
    print("Hello, welcome to Python!")
```

Component	Description
def	Starts the function definition
greet	Name of the function
()	Parentheses (for parameters, if any)
:	Colon marks the start of the function body

Note: Function names must follow the same rules as variable names in Python. They cannot start with a number or contain spaces, and they should not use reserved keywords.

Parameters can be listed in the parentheses to accept input values.

When you define a function in Python, you can allow it to receive information. This is useful because it helps your function work with different data each time it runs. You do this by including parameters in the parentheses after the function name.

A **parameter** is a variable inside the function definition. This variable acts as a placeholder. Each time you call the function, you can pass a value to it. That value is called an argument.

For example, you might create a function that greets a person by name. You want the function to work for any name, not just one. By using a parameter, you allow the user to supply a name each time the function is called.

- Parameters sit in the parentheses after the function name.
- You can define one or more parameters separated by commas.
- The function can use these parameters like normal variables.
- When calling the function, supply arguments to match the parameters.

```
def greet(name):
    print("Hello, " + name + "!")

greet("Amina")
greet("Liam")
```

Definition	Call
def add(a, b):	add(10, 5)
def square(number):	square(7)
def print_age(age):	print_age(26)

Note: Remember, parameters act as labels for the input values a function expects. Arguments are the actual values you pass to these labels when you run the function.

To execute (call) a function, write its name followed by parentheses, providing any necessary arguments.

In Python, functions allow you to organise your code into sections that each complete a specific task. After you define a function, you need to call it to make it run. Calling a function tells Python to execute the instructions inside that function.

To call a function, simply write the function's name followed by parentheses. **If the function requires inputs—these are called arguments—you provide them inside the parentheses.** Otherwise, you leave the parentheses empty.

Using parentheses is essential, as it distinguishes calling the function from just referring to it. Omitting the parentheses would not execute the function, but just give a reference to it.

- **A function call always uses parentheses:** my_function()
- **Arguments are separated by commas:** sum_numbers(5, 10)
- **Calling a function triggers the code** inside it to run
- You can call functions **as often as you need**

```
def greet(name):
    print("Hello, " + name + "!")

greet('Asha') # Calls the function with 'Asha' as the argument
```



```
greet('Liam') # Calls the function with 'Liam' as the argument
```

Example	Description
say_hello()	Calls the function with no arguments
add_numbers(3, 4)	Calls with two arguments: 3 and 4
show_help()	Calls a function meant to display instructions

Note: Always include parentheses when calling a function, even if you are not passing any arguments. Without parentheses, Python will not execute the function.

Using functions improves code organisation, readability, and helps avoid repetition.

Functions are essential building blocks in any programming language, including Python. When you use functions, you break your code into small, manageable pieces. Each piece performs a specific task. This makes your code easier to read, understand, and maintain.

One of the main reasons to use functions is to **avoid repetition**. When you have code that does the same thing in several places, putting it into a function means you only have to write and update it once. This reduces errors and saves time.

Organising your code into functions helps you describe the intent of each section. When you give your functions meaningful names, anyone reading your code can quickly see what each part is meant to do. This improves clarity and makes it simpler to troubleshoot.

- **Functions** allow you to focus on one problem at a time.
- They make your code **easier for others to read** and understand.
- **Reusing functions** saves time and reduces the chance of making mistakes.
- **Updating or fixing logic** is easier when it's in one place.
- **Well-organised code** is more maintainable and less overwhelming.

```
def greet_user(name):
    print(f"Hello, {name}!")

greet_user("Sam")
greet_user("Alex")
```

Without Functions	With Functions
<pre>print("Hello, Sam!") print("Hello, Alex!")</pre>	<pre>def greet_user(name): print(f"Hello, {name}!") greet_user("Sam") greet_user("Alex")</pre>
Copy/paste similar code many times	Reuse a single function with different inputs
Hard to update if greeting changes	Change message in one place

Note: When your code is split into well-named functions, you and others can quickly locate and fix issues. It also encourages testing and reusing your work in other projects.

PARAMETERS AND RETURN VALUES

Parameters allow you to pass information into a function, enabling it to perform tasks with different inputs.

When you write a function in Python, you often want it to work with different bits of information. Rather than rewriting the function every time something changes, you use parameters. Parameters act like placeholders that allow you to give the function new data each time it runs. This makes your code far more flexible and powerful.

When defining a function, you list one or more **parameters** inside the parentheses. When calling the function, you provide **arguments**, which are the actual values passed in. Parameters receive the arguments so your function can use them.

This approach lets a single block of code do many jobs. For example, you can write a function to greet a user and provide their name as a parameter. Each time you call the function with a different name, it greets a different person.

- Parameters are like local variable names declared in the function's definition.
- They receive their initial value from the arguments you supply in the function call.
- You must match the number of arguments to the number of parameters unless a default is set.
- Parameters make functions reusable with varying data.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Ava")    # Output: Hello, Ava!
greet("Rahul") # Output: Hello, Rahul!
```

Term	Description
Parameter	The variable name used in the function definition to accept input (e.g., name)
Argument	The actual value you pass into the function when you call it (e.g., "Ava")
Function Call	The act of running the function and supplying arguments

Note: Using parameters makes your functions more general and useful. Always plan for what data a function really needs so you can design its parameter list clearly.

Return values let a function provide an output or result back to the code that called it.

When you write a function in Python, you often want it to give back a result. This is called the function's return value. The return value lets other parts of your code get something useful from the function after it finishes its work.

You can use the **return** statement in a function to choose what is sent back. Any code that calls the function will receive this value, and you can then use it, store it in a variable, or pass it on to another function.

For example, if you write a function that adds two numbers, you want the sum to be returned to where you called the function. If a function does not use return, it gives back a special value called None, which means there is no result.

- A **return value** lets a function give an answer or result.
- You decide what gets returned using the **return statement**.
- You can **return numbers, text, lists, or even other functions**.
- If you do not use return, the function **returns None**.
- Returned values can be captured **in variables** and used later.

```
def add_two_numbers(a, b):  
    sum = a + b  
    return sum  
  
result = add_two_numbers(3, 5)  
print(result)  # Output: 8
```

Function Example	Return Value
<code>def greet(): return 'Hello'</code>	'Hello'
<code>def double(x): return x * 2</code>	x multiplied by 2
<code>def nothing(): pass</code>	None

Note: Important tip: Use return when you want your function to provide information to the rest of your program. If you forget to include return, or you do not need to return anything, Python will use None as the default return value.

You can define functions with multiple parameters, separated by commas.

When you write a function in Python, you often want it to accept input values. These input values are called parameters. Parameters allow your function to do its job using different information each time it runs.

You can give a function more than one parameter by including them in the function definition, separated by **commas**. This lets your function accept several pieces of information at once. Each parameter acts as a variable inside the function.

For example, imagine you want to write a function that adds two numbers together. You can define it with two parameters—one for each number you want to add. When you call the function, you provide two values, and the function uses them.

- Parameters are placed inside the brackets after the function name.
- Each parameter is separated by a comma.
- You can use as many parameters as you need (within reason).
- You must provide values for each parameter when you call the function, unless you have set default values.

```
def add_numbers(num1, num2):  
    result = num1 + num2  
    return result  
  
# Calling the function with two values  
print(add_numbers(4, 7))    # Output: 11
```

Function Definition	Description
<code>def greet(name, message):</code>	Defines a function with two parameters: name and message.
<code>def multiply(x, y, z):</code>	Defines a function with three parameters.
<code>def distance(x1, y1, x2, y2):</code>	Defines a function with four parameters.

Note: The names you use for parameters should be clear and describe the information each one holds. Good naming makes your code much easier to read and maintain.

Functions may return one or more values using the return statement.

In Python, functions can send data back to where they were called by using the return statement. This is a key feature that makes functions flexible and useful. When a function completes its task, it can provide a result to the rest of your program.

The **return** statement marks the end of a function and specifies what value, or values, should be delivered back to the caller. This lets you pass results or calculations out of your function, so you can use them elsewhere in your code.

If you do not include a return statement, Python supplies a special value called None. None indicates that the function did not return any particular data.

You can use return in several ways. Sometimes you return one value (such as a number or a string). Python also allows you to return multiple values in a single statement. If you do this, the values are bundled together into a tuple. You can unpack this tuple into separate variables when you call the function.

- Use return to send a result from your function.
- Omit return to finish a function without sending data—Python returns None automatically.
- Return one value (like an integer), or several (as a tuple, such as a pair of numbers).
- Unpack returned tuples when you call the function for clarity.

```
def add(x, y):
    # Returns the sum of x and y
    return x + y

result = add(3, 5)
print(result)  # Output: 8

# Example of multiple return values
def min_max(numbers):
```

```

lowest = min(numbers)
highest = max(numbers)
return lowest, highest

low, high = min_max([6, 3, 9, 2, 5])
print('Lowest:', low)
print('Highest:', high)

```

Return Style	Example
Single value	return x * 2
No explicit return	No return line (returns None)
Multiple values	return x, y, z

Note: You can only reach the first return statement in a function. After a return command, the function exits immediately—no further lines are run.

Understanding parameters and return values is essential for writing modular and maintainable code.

Functions are the building blocks of modular programming. They allow you to group code into logical units, making programs easier to read and maintain. Parameters and return values play a central role in how functions communicate with the rest of your program.

When you define a function, **parameters** are placeholders for values that the function expects when it is called. These values, known as arguments, are provided by the calling code. **Return values** are the outputs that a function produces and sends back to the caller. Clear use of parameters and return values helps make code reusable and easier to debug.

Imagine you are writing a program to calculate the area of a rectangle. Rather than hard-coding specific values, you can write a function that takes the width and height as parameters. The function can then return the calculated area to the caller. This approach allows you to reuse the same function with different values every time.

- **Parameters** make functions flexible and reusable.
- **Return values** let functions communicate results back to the caller.
- Using parameters and return values makes your code easier to **read, test, and maintain**.
- You can use **multiple parameters** and return complex data types.

```
def add_numbers(a, b):  
    result = a + b  
    return result  
  
sum = add_numbers(3, 5)  
print(sum)  # Output: 8
```

Term	Description
Parameter	A named input in a function definition; a placeholder for a value.
Argument	The actual value supplied to a function parameter when called.
Return Value	The output that a function provides back to the caller.

Note: Important tip: Always use descriptive parameter names and keep functions focused on a single task. This makes code easier to understand and reduces errors.

SCOPE OF VARIABLES

Variables in Python have scope, which determines where they can be accessed in a program.

In Python, every variable has a scope. Scope defines where you can use or access a variable within your code. Understanding scope helps you organise your programs, prevent errors, and avoid unexpected results. Scope plays a key role in how Python manages memory and keeps variables separate.

There are two main types of scope in Python: **local scope** and **global scope**. Variables declared inside functions usually have local scope, while those declared outside functions have global scope.

A variable's scope determines how long it exists and where you can use it. If you try to use a variable outside its scope, you will get an error. This helps you design cleaner code and avoid mistakes like accidentally overwriting variables.

- **Local scope:** Variables defined inside a function. These can only be used within that function.
- **Global scope:** Variables defined outside any function. These can be used anywhere in your program, including inside functions (unless shadowed).
- **Variables with the same name** can exist in both scopes, but they will not interfere with each other.

```
# Example: Local and Global Variable Scope

global_var = 'Hello, world!' # Global scope

def greet():
    local_var = 'Hi there!' # Local scope
    print(local_var)        # Can access local_var inside the
                             # function
    print(global_var)        # Can access global_var from inside
                             # the function

print(global_var)           # Can access global_var outside the
                             # function
greet()
# print(local_var)          # This will cause an error: local_var
                             # is not defined here
```

Scope Type	Where Accessible
Global	Anywhere: inside or outside functions (unless overridden locally)
Local	Only inside the function where the variable is defined
Nonlocal/Enclosing	Nested functions (advanced topic)

Note: Important tip: Use local variables whenever possible to avoid naming conflicts and minimise errors. This makes your code more modular and easier to debug.

Local variables are defined within a function and can only be used inside that function.

When you write a function in Python, you often create variables just for use inside that function. These are known as local variables. Understanding local variables helps you write code that is organised, safer, and easier to troubleshoot.

A **local variable** is one that is created and used only inside a specific function. As soon as the function finishes running, any local variables inside it are removed from memory. This means you cannot access them from outside the function, and their values cannot affect code elsewhere.

This behaviour helps prevent accidental changes to your program from variables in other places. It also means that functions can have variables with the same name without causing conflicts. Let's see how it works with an example.

- **Local variables** are created when a function is called.
- They only exist while the **function is running**.

- They **cannot be used or changed** outside their function.
- Each call to the function creates a **brand-new set of local variables**.
- Using local variables helps keep your **code neat and reduces mistakes**.

```
def calculate_area(radius):
    pi = 3.14159 # 'pi' is a local variable
    area = pi * radius * radius # 'area' is also a local variable
    return area

print(calculate_area(5))
# print(pi) # This will cause an error: 'NameError: name 'pi' is not defined'
```

Scope Type	Where Defined	Where Accessible
Local Variable	Inside a function	Only inside that function
Global Variable	Outside all functions	Everywhere in the script
Parameter	Inside function definition	Only inside that function

Note: Important tip: Giving your variables clear, unique names inside functions makes your code easier to understand. Don't rely on local variables outside the function in which they are defined.

Global variables are defined outside of functions and can be accessed from anywhere in the code.

In Python, a variable is called global when you create it outside of any function. Global variables are accessible throughout your program, meaning any function or block of code can use them. They remain available for as long as your program is running.

Understanding the difference between local and global variables is crucial. **Global variables are usually created at the top of your script, outside any function.** This makes them easy to find and use from anywhere in your code.

Because global variables can be accessed everywhere, they are useful for values or settings your entire program needs to share. However, using too many global variables can make your code harder to understand and maintain. A good approach is to keep global variables for things that genuinely need to be available everywhere, like configuration options.

- **Global variables** are declared outside all functions or classes.
- Any part of the program can access and read their value.

- You can **change a global variable within a function** by using the global keyword.

```
# This is a global variable
app_name = "MyCalculator"

def show_app_name():
    print("Application Name:", app_name)

def change_app_name():
    global app_name
    app_name = "SuperCalculator"

show_app_name()      # Output: Application Name: MyCalculator
change_app_name()
show_app_name()      # Output: Application Name: SuperCalculator
```

Variable Type	Where It Is Accessible
Global	Anywhere in the script, including inside functions (with some restrictions)
Local	Only inside the function where it is defined
Nonlocal	In nested functions, refers to variables in an enclosing function

Note: Important tip: Use global variables sparingly and only when truly necessary. If every function changes the same global variable, it can become difficult to track changes and find bugs.

To modify a global variable inside a function, use the 'global' keyword.

When you write Python code, you often create variables in different places. Some variables exist only inside a function, while others are global. If you want a function to change or update a variable that was created outside the function (a global variable), you need to use the 'global' keyword.

Normally, assigning a value to a variable inside a function creates a new local variable. However, if you want to **change the value of a global variable from inside a function**, you must declare it as 'global' within that function. This tells Python that you are referring to the variable outside of the function, not creating a new one.

This behaviour is important to understand because it affects how your programs work. Without using 'global', changes you make inside a function won't affect the global variable. This can be confusing if you're expecting the function to update a variable used elsewhere in your program.

- The 'global' keyword is used inside functions only.
- It connects the inside of the function to the variable defined outside.
- Use it when you need to change or update a global variable from within a function.

```
counter = 0

def increase_counter():
    global counter
    counter += 1

increase_counter()
print(counter)  # Output: 1
```

Action	Effect
Assign to variable in function without 'global'	Creates a new local variable
Use 'global' and assign in function	Updates the global variable
Change global variable outside function	Updates variable for the whole program

Note: Important tip: Use the 'global' keyword only when really needed. Too many global variables can make your code harder to understand and debug. Aim to limit their use where possible.

Understanding variable scope helps prevent bugs and makes your code more predictable.

Variable scope refers to where in your program a variable can be accessed or changed.

Understanding how scope works in Python is a key skill for building reliable and readable programs. When you organise your code with clear scopes, it becomes easier to predict what will happen, and you reduce the chance of bugs caused by accidental changes to variables.

In Python, there are two main types of variable scope: **local** and **global**. Local variables only exist inside the function or block where you define them. Global variables are available throughout the file after they are created.

If you do not think about variable scope, you might accidentally change a value in one part of your program and affect another part unexpectedly. This can cause bugs that are hard to track down. Making your variable scopes clear helps you understand what each part of your program is doing.

- Local variables only exist in the block or function where you create them.
- Global variables are available everywhere in your file after you define them.

- Variables inside functions do not affect variables outside, unless you explicitly tell Python to use the global version.
- Well-organised variable scope makes your code easier to debug and maintain.

```
x = 10  # This is a global variable

def my_function():
    x = 5  # This is a local variable, different from the global x
    print('Inside the function, x =', x)

my_function()  # Prints: Inside the function, x = 5
print('Outside the function, x =', x)  # Prints: Outside the
function, x = 10
```

Variable Type	Where Is It Accessible?
Local	Inside the function or block only
Global	Anywhere in the file after it is defined
Built-in	Always available (e.g., print, len)

Note: Important tip: Keep your variables as local as possible. This practice reduces the risk of errors and makes your code easier to understand.

REUSABILITY AND MODULAR DESIGN

Learn how to define and call Python functions to organise code into logical units.

When you write your own programs, it is important to keep your code organised and easy to understand. One of the best ways to achieve this is by dividing your code into small, logical sections. In Python, these sections are called functions. Functions help you group related instructions together, so your program is cleaner, easier to read, and simpler to manage.

A **function** is a named block of code that can be run whenever you need it. Functions let you write a set of instructions once, and then reuse them as many times as you want throughout your program.

Imagine you want to perform a task—like displaying a welcome message—several times in your application. Rather than copying and pasting the instructions over and over, you can put them in a function. This not only saves time but also avoids mistakes. You can also update the function in one place if your requirements change, instead of updating many copies.

- **Functions** help break your code into manageable parts.

- They make your programs **easier to read and maintain**.
- Functions allow **code reuse**—you only need to write logic once.
- **Testing is easier** because you can check each function separately.
- Functions help you **avoid repeating yourself** (which is good practice).

```
def greet():
    print('Hello, welcome to the program!')

greet()
greet()
```

In the example above, 'def' is a keyword that tells Python you are defining a new function. The function is named 'greet'. The code inside the function prints a welcome message. You call this function by writing its name with brackets, like greet(). You can call it as many times as you like.

Part	Purpose
def	Starts a function definition
function name	Gives the function an identifier
()	Brackets that may hold parameters
:	Ends the function header and starts the code block

You can also design functions to do more than display messages. For example, you might write a function to calculate totals, check passwords, or process data. Functions can take input (parameters) and sometimes give you a result (return value) to use elsewhere in your code.

```
def square(number):
    return number * number

result = square(5)
print('5 squared is:', result)
```

Note: It is good practice to give functions meaningful names that describe what they do. This makes your code easier for others (and yourself) to read later.

Understand the use of parameters and return values to enable flexible and reusable functions.

When you write a function in Python, you often want the function to solve more than one specific problem. You can achieve this by using parameters and return values. Parameters let you pass information to the function, while return values let you send results back from the function. This makes your function flexible and reusable.

Let's break it down: **Parameters** are like variable names you define in the function's header. When you call the function, you provide arguments (concrete values) for these parameters. **Return values** come from the function after it runs. You use a return statement to decide what goes back to the caller. Functions that do not specify a return value will return None by default.

Without parameters, a function can only use values hardcoded inside it or available globally. Without return values, your function can only produce side effects, such as printing to the screen. With parameters and return values, you can write functions that act like tools—they can be used repeatedly in different scenarios, just by changing the information you give them.

- **Parameters make your functions accept input**, so you don't need to rewrite similar code for every task.
- **Return values help you get results from your functions**, so you can use these results in other parts of your code.
- Writing functions with parameters and return values is **key to modular, maintainable, and testable programs**.

```
def calculate_area(width, height):
    area = width * height
    return area

result = calculate_area(5, 10)
print('Area:', result)
```

Concept	Description
Parameter	A named value listed in the function definition. Used to accept input.
Argument	The actual value supplied to the function when called.
Return value	The result produced by the function and sent back to the caller using return.

Note: Whenever you find yourself repeating similar blocks of code with only small differences, consider writing a reusable function with parameters. This keeps your code clean and avoids duplication.

Identify and apply variable scope rules to prevent naming conflicts in your programs.

When writing Python programs, you will often use many variables to store information and control your code. It's essential to understand how variable scope works so you can avoid naming conflicts, unexpected results, and bugs. Scope refers to where a variable is visible and accessible in your program. Using scope rules correctly helps keep your programs organised, safe, and easy to understand.

In Python, variables can exist in different scopes. **Local scope** means a variable is only available inside a function or block of code. **Global scope** means a variable can be accessed from anywhere in the file after it is defined. Understanding which is which will help you control how variables behave and interact.

When you define a variable inside a function, it is usually local. Only code inside that function can see or change the variable. If you create a variable outside any function, it is global. All functions in the file can use or change it unless they make their own local variable with the same name.

- Local variables: Defined inside functions, only used within that function.
- Global variables: Defined outside functions, can be used throughout the program.
- Variables with the same name in different scopes do not affect each other directly.
- Python will use the closest matching variable in the innermost scope first.
- Overlapping variable names can cause confusion, so choose names carefully.

```
x = 10  # Global variable
def print_number():
    x = 5  # Local variable
    print('Inside function, x =', x)

print_number()
print('Outside function, x =', x)
```

Scope Type	Where It's Used
Local	Inside a function or code block
Global	Outside all functions, at the top level
Enclosing	In enclosing functions (nested functions)

Note: Important tip: Avoid using the same variable name for both local and global variables unless absolutely necessary. This will help you prevent unexpected behaviour and make your code easier to read and maintain.

Design and structure code to be modular and maintainable for easier updates and debugging.

Writing code is not just about getting the program to work. Good code should also be easy to understand, update, and fix later. This is where modularity and maintainability become important. By designing your code to be modular, you break it into smaller pieces, or modules, that each handle a specific task. Maintaining code means organising it so changes can be made easily without causing new problems.

You achieve modular design by putting related code together—for example, all code for mathematical calculations goes in one function or module. **This practice makes your code easier to read, test, and reuse.** If you need to fix a bug or add a feature, you usually only update a small part, not the entire program.

Consider how you might build a house. You would not try to construct it all in one piece. Instead, you would work on separate rooms and utilities (like plumbing and wiring). In coding, modular design works the same way. You build programs as a collection of smaller, independent parts that work together. This structure improves both teamwork and long-term code health.

- **Each function should do one thing** and do it well.
- **Group related functions or data together** in modules.
- **Use clear and consistent names** for functions and variables.
- **Write comments to explain why code does something**, not just what it does.
- **Avoid repeating code:** reuse functions and modules whenever possible.

```
def calculate_area(width, height):
    """Calculate the area of a rectangle."""
    return width * height

def print_area(width, height):
    area = calculate_area(width, height)
    print(f"Area is {area} square metres.")

# Main program
print_area(5, 7)
```

Concept	Benefit
Modularity	Update parts of your code without affecting others
Reusability	Write functions once and use them many times
Maintainability	Easier to understand and change code later

Note: Important tip: When your program grows larger, good modular design saves you time and frustration. If you split your code into well-defined parts, it's much simpler to find bugs and add new features.

Practice creating reusable Python components to efficiently solve complex problems.

Reusability is a core principle of good software design. In Python, building reusable components allows you to tackle complex problems more effectively. Reusable code means you can solve similar tasks without having to rewrite code every time. This saves time and reduces errors.

Creating **reusable components** in Python often involves writing functions, classes, or modules. When code is modular and reusable, it becomes easier to manage, test, and update in the future.

To achieve reusability, break larger problems into smaller, well-defined tasks. Each task should be handled by a separate function or module. This approach is called modular programming. For example, if you are building a calculator, each operation—addition, subtraction, multiplication, and division—should have its own function.

- Keeps your code **organised and readable**
- Makes **code easier to test and debug**
- **Allows you to use the same code** in multiple projects
- **Reduces risk of bugs** by centralising logic
- **Helps teams collaborate** by dividing work into clear parts

```
def square(number):
    return number * number

def cube(number):
    return number * number * number

# These functions can be reused in any calculation involving
squares or cubes
print(square(4))      # Outputs: 16
print(cube(3))        # Outputs: 27
```

Strategy	Description
Function	Encapsulate a repeatable task that can be called many times.
Class	Define a blueprint for objects, useful for complex behaviours.
Module	Group related functions or classes in a separate file for reuse.

Note: When creating reusable components, always give them clear names and document what they do. This makes it easier for you and others to use your code in different contexts.

6. Data Structures

LISTS (INDEXING, SLICING, METHODS)

Lists allow you to store and organize multiple items in a single variable.

A list in Python is a flexible way to keep an ordered collection of items in one place. Unlike single variables that hold just one value, lists can store many values together. You can include numbers, words, or even other lists inside a list.

Lists are enclosed in square brackets, like `[ ]`. Each item in a list is separated by a comma. This design makes them easy to create and use.

Think of a list as a row of labelled boxes. Each box can hold a value, and you can find any box quickly using its position number, called an index. Lists are perfect when you need to collect items together, such as names, scores, or tasks.

- Lists keep things in order—your first item stays first, and so on.
- You can change (or 'mutate') lists by adding, removing, or updating items.
- Lists can hold different types of items, such as numbers and text, at the same time.
- You can loop through a list to examine or change its contents.

```
# Creating a list of colours
colours = ['red', 'green', 'blue']

# Adding an item
colours.append('yellow')

# Accessing the first item
first_colour = colours[0]

# Changing an item
colours[2] = 'purple'

# Resulting list: ['red', 'green', 'purple', 'yellow']
```

Action	Example code
Create a list	<code>fruits = ['apple', 'banana', 'cherry']</code>
Get first item	<code>fruits[0]</code>
Change an item	<code>fruits[2] = 'pear'</code>
Add new item	<code>fruits.append('plum')</code>

Note: The order of items in a list matters. Python lists start indexing from zero, not one. For example, the first item is at index 0.

Individual elements within a list can be accessed using zero-based indexing.

In Python, lists are used to store collections of items in a specific order. Each item in a list is called an element, and every element has a position in the list. To work with individual elements, you'll need to understand how Python counts positions within a list. Python uses zero-based indexing, which means the first element has an index of 0, the second has an index of 1, and so on.

Zero-based indexing is **standard in Python** and many other programming languages. If your list has five elements, their indexes will be 0, 1, 2, 3, and 4. This simple rule makes it easy to remember that the position and the index are not always the same number.

To access an element, you use its index inside square brackets after the list name. For example, if you have a list called `colours` and want the first colour, you use index 0. Trying to use an index outside the range of the list will cause an error.

- Indexes start at 0. The first element is accessed with index 0.
- You can use negative indexes to count from the end of a list, with -1 giving you the last item.
- Trying to use an index that does not exist will cause an `IndexError`.

```
colours = ['red', 'green', 'blue', 'yellow']
print(colours[0])    # Outputs 'red'
print(colours[2])    # Outputs 'blue'
print(colours[-1])   # Outputs 'yellow'
```

Index	Element
0	red
1	green
2	blue
3	yellow

Note: Remember, always start counting at zero when accessing list elements in Python. If you try to use an index equal to the length of the list (for example, `colours[4]` in a four-item list), you will get an `IndexError`.

Slicing lets you obtain a subset of a list using a specified range of indices.

When working with lists in Python, you will often need just a part of the data instead of the entire list. Slicing is a powerful feature that allows you to extract a section of a list by specifying where the slice should start and end using indices. This makes it easy to work with subsets of data without changing the original list.

A **slice** refers to a part of a list selected by indicating a start index, a stop index, and sometimes a step value. The syntax for slicing is `list[start:stop]`. Python includes all items from the start index up to, but not including, the stop index.

Let's look at an example. Suppose you have a list of numbers: `numbers = [10, 20, 30, 40, 50, 60, 70]`. If you want the second, third, and fourth items, you can use slicing: `numbers[1:4]` returns `[20, 30, 40]`. This is because indices start at 0, so 1 is the second item, and 4 is just after the one you want.

- Start index is included in the result.
- Stop index is excluded.
- You can omit the start to begin from the start of the list.
- You can omit the stop to go to the end of the list.
- Negative indices count from the end of the list.

```
# Sample list
data = ['a', 'b', 'c', 'd', 'e', 'f']

# Slicing examples
first_three = data[0:3]      # ['a', 'b', 'c']
last_two = data[-2:]        # ['e', 'f']
every_second = data[::2]    # ['a', 'c', 'e']
except_first = data[1:]     # ['b', 'c', 'd', 'e', 'f']
middle = data[2:4]          # ['c', 'd']
```

Slice Example	Result
<code>data[1:4]</code>	<code>['b', 'c', 'd']</code>
<code>data[:3]</code>	<code>['a', 'b', 'c']</code>
<code>data[3:]</code>	<code>['d', 'e', 'f']</code>
<code>data[-3:]</code>	<code>['d', 'e', 'f']</code>
<code>data[::-1]</code>	<code>['f', 'e', 'd', 'c', 'b', 'a']</code>

Note: Remember, slicing does not alter the original list. It creates a new list containing just the selected elements. This is handy for trying things out or copying parts of your data.

Lists come with built-in methods such as `append()`, `remove()`, and `sort()` for effective manipulation.

Python lists are not just simple containers for values—they provide a set of built-in methods that make working with data much easier. By using these methods, you can add, remove, rearrange, or update list items efficiently. Understanding these methods will help you write cleaner and more effective code.

Three of the most commonly used list methods are **`append()`**, **`remove()`**, and **`sort()`**. Each of these serves a different purpose for list manipulation.

The `append()` method adds an item to the end of your list. This is useful when you want to grow your collection one item at a time. For example, building up a shopping list as you think of new items.

- `append(item)`: Adds an item to the end of the list.
- `remove(item)`: Removes the first occurrence of the item from the list.
- `sort()`: Arranges the list items in ascending order by default.

```
# Example: Using built-in list methods
fruits = ['apple', 'banana', 'cherry']
fruits.append('orange') # List becomes ['apple', 'banana', 'cherry', 'orange']
fruits.remove('banana') # List becomes ['apple', 'cherry', 'orange']
fruits.sort()           # List becomes ['apple', 'cherry', 'orange']
```

Method	Description
<code>append()</code>	Adds an item to the end
<code>remove()</code>	Deletes the first matching item
<code>sort()</code>	Sorts items in ascending order

If you try to remove an item that is not in the list, Python will give you an error. The `sort()` method, by default, arranges everything from smallest to largest (or alphabetically for strings), and you can also tell it to sort in reverse order if needed.

Note: Important tip: These methods change the actual list they are called on. If you want to keep the original list unchanged, make a copy before manipulating.

Lists in Python are mutable, meaning their content can be changed after creation.

In Python, a list is a flexible data structure used to store ordered collections of items. One of the most important features of lists is their mutability. This means you can change the contents of a list after it has been created, whether by adding, removing, or modifying existing elements.

Mutability is an essential concept in programming. When a data structure is **mutable**, you can alter its contents without having to create a whole new list. In contrast, **immutable** objects (like strings or tuples) cannot be changed after creation.

Because lists are mutable, you have many ways to work with them. You can append new items to the end, insert them at a specific position, change the value of an existing item, or even remove items. This flexibility makes lists one of the most widely used data structures in Python.

- Change values of elements after creation
- Add new elements to the list
- Remove or replace existing elements
- Grow and shrink in size as needed

```
# Creating a list
fruits = ['apple', 'banana', 'cherry']

# Lists are mutable
# Change an item
fruits[1] = 'blueberry' # Now: ['apple', 'blueberry', 'cherry']

# Add a new item
fruits.append('orange') # Now: ['apple', 'blueberry', 'cherry', 'orange']

# Remove an item
del fruits[0] # Now: ['blueberry', 'cherry', 'orange']
```

Operation	Example
Modify element	my_list[2] = 10
Append item	my_list.append(5)
Remove item	del my_list[0]

Note: Important tip: Because lists are mutable, changes made to a list within a function will affect the original list. Always be mindful when passing lists around in your code.

TUPLES AND WHEN TO USE THEM

Tuples are immutable sequences in Python, meaning their values cannot be changed after creation.

In Python, a tuple is a type of sequence that stores an ordered collection of values. Unlike lists, which you can change after you create them, tuples are immutable. This means once you create a tuple, you cannot alter, add, or remove any of its items.

You might wonder what 'immutable' means in practice. **Immutable means unchangeable.** When you create a tuple, its contents are fixed. For example, if you create a tuple with three numbers, you cannot replace, remove, or add a number to it later.

Immutability is a key difference between tuples and lists. It affects how you use them in your programs. Many Python programmers use tuples to represent collections of related information that should not be changed. For example, you might use a tuple to store the coordinates of a point on a map, a date, or configuration settings that need to stay the same.

- You cannot change, add, or remove elements from a tuple after creating it.
- Tuples are defined using round brackets: ()
- Tuples can hold different types of data, like numbers, strings, or even other tuples.
- If you try to change a tuple, Python will give you an error.
- Because they are immutable, tuples are safe to use as keys in dictionaries.

```
# Creating a tuple
point = (3, 5)
# Accessing tuple elements
x = point[0] # x gets 3
# Trying to change a tuple value (will cause an error)
point[0] = 8 # This line will cause a TypeError
```

Feature	Tuple	List
Mutable	No	Yes
Syntax	(1, 2, 3)	[1, 2, 3]
Can change size?	No	Yes

Note: Important tip: Use tuples when you want to protect data from accidental changes, like storing fixed coordinates or days of the week. If you need to modify the collection, use a list instead.

Use tuples when you need to store a fixed collection of items that should not be modified.

In Python, a tuple is a way to store several values together in a single variable. Unlike lists, tuples are immutable. This means you cannot change their contents after you create them. Tuples are useful when you have a group of values that belong together and you want to make sure they are not accidentally changed.

Imagine you need to keep your program's release version as a combination of three numbers, like **1.0.5**. You want to ensure that these numbers stay the same throughout the code. Using a tuple, you guard against unwanted changes. This provides extra safety for important data.

Tuples are quick to create and read, which makes them efficient for fixed collections of values. Since their contents cannot be altered, they are also more secure in situations where your program must not let data be edited later by mistake.

- Use a tuple for fixed data, such as days of the week.
- Tuples are ideal for returning several values from a function.
- They can be used as keys in dictionaries because they do not change.
- Protects your data from accidental changes in large programs.

```
version_info = (1, 0, 5)
# version_info[0] = 2 # This will cause an error, as tuples can't
# be modified
person = ('Alex', 'Brown', 30)
# Storing a fixed list of user details that shouldn't be changed
```

Feature	Tuples	Lists
Can be changed after creation?	No	Yes
Syntax	(1, 2, 3)	[1, 2, 3]
Can be used as dictionary keys?	Yes	No
Performance	Faster for fixed data	Slightly slower

Note: Important tip: If your data might need updating (like a shopping list), use a list. If the values should stay the same, choose a tuple.

Tuples can store multiple data types, similar to lists, but provide data integrity due to their immutability.

Tuples are a fundamental data structure in Python. Like lists, they let you group together multiple pieces of data in a single container. However, tuples have a special feature—they are immutable. This means, once you create a tuple, its contents cannot be changed. This property helps protect the data you store.

You can use tuples to store **different types of data in one place**. For example, you might keep a name as a string, an age as an integer, and a grade as a float all together in one tuple. This flexibility is similar to lists, but with the added safety of immutability.

Immutability means you cannot change, add, or remove items after creating the tuple. This helps ensure that the data stays the same throughout your program. In situations where you want to guarantee data is not changed accidentally, tuples offer a reliable solution.

- Tuples can include any data type—numbers, strings, even other collections.
- Immutability helps maintain data integrity and trustworthiness.
- Using tuples can make your code safer and clearer, especially when you want fixed data.

```
# Creating a tuple with multiple data types
student = ("Alice Smith", 22, 88.5)
print(student)
# Trying to change a value will cause an error
# student[1] = 23 # This line would result in a TypeError
```

Lists	Tuples
Mutable (changeable)	Immutable (fixed)
Add/remove items after creation	Cannot add or remove items
Good for data that might change	Best for data that should not change

Note: Important tip: Use tuples for data that must stay the same, such as settings, coordinates, or database keys. This avoids accidental changes in your program.

They are commonly used to represent structured data, such as coordinates or database records.

Tuples are a core Python data structure used to group together multiple pieces of related information. When you need to represent a fixed set of values that belong together, such as a point in space or a simple record from a database, tuples are ideal. Their main feature is that, after creation, their contents cannot be changed.

A tuple is ordered and **immutable**, which means the position of items and their values stay the same after creation. This makes tuples very useful for structured data where each position has a specific meaning.

For example, consider representing the coordinates of a location. You might use a tuple like (x, y) or (latitude, longitude), where each value has a particular meaning. Tuples are also helpful when fetching records from a database, as one tuple can hold multiple columns of data for a single row.

- Tuples are best for grouping related, unchanging data together.
- Each element in a tuple usually represents a specific property, such as age, name or coordinates.
- Because tuples are immutable, they help prevent accidental modification of your data.

```
# Representing a point (x, y) using a tuple
point = (10, 20)

# Representing a database record (id, name, email)
user_record = (101, "Sam Lee", "sam.lee@example.com")

# Accessing values
x_coord = point[0]
y_coord = point[1]

print(f"X: {x_coord}, Y: {y_coord}")
print(f"User ID: {user_record[0]}, Name: {user_record[1]}")
```

Use Case	Example Tuple
Geographic Coordinates	(-33.8688, 151.2093)
RGB Colour	(255, 127, 80)
Database Row	(1, "Alex Smith", "alex@example.com")

Note: Important tip: Use tuples when the structure and size of your grouped data should not change throughout your program. They make your intention clear and help protect data from being accidentally altered.

Tuples can be used as dictionary keys or set elements, unlike lists, because they are hashable.

In Python, different types of data structures have different rules about how they can be stored and used. One especially useful property of tuples is that they are hashable. This means you can use tuples as keys in dictionaries or as elements in sets. In contrast, lists cannot be used in this way.

A value is considered **hashable** if it does not change (is immutable) and has a unique hash value that stays the same throughout its lifetime. Tuples are immutable—once created, their contents cannot be changed. Lists are mutable, meaning their items can be altered, which makes them unfit for use as dictionary keys or set elements.

Using tuples for dictionary keys or as elements in sets is common when you need to represent a fixed combination of values, such as coordinates or pairs. Because Python dictionaries and sets use hashing to check for data quickly, only hashable (immutable) objects can be used in these roles.

- Tuples are immutable, so their hash value does not change.
- Lists are mutable, so they cannot be hashed and cannot be used as dictionary keys or set elements.
- Tuples can store related pieces of data together and be used in collections that require unique, hashable items.

```
coordinates = (10, 20)
my_dict = {coordinates: "This is a point on a grid"}
print(my_dict[(10, 20)]) # Outputs: This is a point on a grid

my_set = set()
my_set.add((1, 2, 3))      # Allowed
your_list = [1, 2, 3]
# my_set.add(your_list)    # Not allowed, will raise an error
```

Data Type	Can be dictionary key / set element?
Tuple	Yes
List	No
Integer	Yes

Note: Important tip: Only tuples that contain hashable (immutable) objects can be used as dictionary keys or set elements. For example, a tuple containing a list is not hashable.

SETS AND UNIQUENESS

Sets are unordered collections of unique elements, meaning duplicate values are automatically removed.

In Python, a set is a built-in data structure that stores a collection of items. What sets it apart is that each item must be unique. You cannot have two identical items in a set. Sets are also unordered, which means the order you add items does not matter and is not preserved when you access or display the set.

If you try to add a value to a set that already exists in that set, Python will simply ignore the duplicate. **Sets automatically remove duplicate values.** This property is very useful when you need to identify all unique items in a group or remove repeated entries from a list.

Because sets are unordered, you cannot rely on them to keep items in any particular sequence. When you print or loop through a set, the items may appear in a different order each time. This makes sets suitable for tasks where uniqueness is more important than order.

- A set cannot contain duplicate values. Only one instance of each unique value is stored.
- The items in a set have no specific order and their position can change.
- Sets are useful for finding unique entries or removing duplicates from a collection.

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers) # Output might be: {1, 2, 3, 4, 5}
```

Operation	Result
Create a set from a list with duplicates	{1, 2, 3}
Add a duplicate to a set	No change; the set stays the same
Sets keep order?	No; order is random and may change

Note: Important tip: When you convert a list with repeated values to a set, all duplicates will be automatically filtered out. Use sets when you only care about unique values, not about the order in which they appear.

Use sets in Python when you need to store a collection of items without duplicates.

A set in Python is a collection of items where each item is unique. That means no value can appear more than once. Sets are very useful if you need to store or check for unique items, and you do not care about their order. Python's set type makes it simple to remove duplicates, perform mathematical set operations, and quickly check if something is present in the collection.

Sets are vital in programming tasks that involve **uniqueness**. If you have a list with repeating values and want to find only unique ones, using a set is the easiest way. Sets help eliminate duplicate data automatically, which is often necessary when cleaning data or verifying membership.

In Python, you create a set using curly braces or the `set()` function. Once created, a set cannot contain repeated values. Sets are also very fast when checking whether an item exists in the collection, making them practical for membership tests.

- **Sets do not allow duplicates** – every element is unique.
- **Order is not maintained**: sets are unordered collections.
- **You can add and remove items but not update existing values.**
- **Sets cannot contain lists or other sets**, but can include numbers, strings, and tuples.
- **Use sets** to compare data, find differences, or remove duplicate entries.

```
# Create a set from a list with duplicates
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers)  # Output might be: {1, 2, 3, 4, 5}

# Membership test
if 3 in unique_numbers:
    print('3 is in the set!')

# Add a new element
unique_numbers.add(6)
print(unique_numbers)

# Try to add a duplicate
unique_numbers.add(4)  # No change, 4 is already present
```

Operation	Example
Create an empty set	<code>my_set = set()</code>
Remove duplicates from a list	<code>set([1, 1, 2, 3])</code>
Check if element exists	<code>'a' in my_set</code>
Add an element	<code>my_set.add('b')</code>
Remove an element	<code>my_set.remove('b')</code>

Note: Important tip: Use sets any time you need to store a group of values and want to guarantee each element appears only once. Sets are perfect for tasks like finding unique values, checking for membership, or comparing different collections efficiently.

Common set operations include union, intersection, difference, and symmetric difference.

Sets in Python are collections that store unique elements. They are useful when you want to remove duplicates or perform operations that rely on uniqueness. Understanding set operations helps you compare and process groups of items efficiently.

The main set operations are **union**, **intersection**, **difference**, and **symmetric difference**. These operations let you combine or compare sets in various ways.

Union brings together all unique elements from two sets. Intersection finds elements common to both. Difference shows what is present in one set but not the other. Symmetric difference returns items that are in one set or the other, but not both. These operations help you solve problems such as finding shared interests or unique attributes.

- **Union:** Combines all items from two or more sets, keeping only one copy of each unique value.
- **Intersection:** Finds items that appear in both sets.
- **Difference:** Shows items in the first set that are not in the second.
- **Symmetric Difference:** Finds items present in either set, but not in both.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Union
a_union_b = set_a | set_b # {1, 2, 3, 4, 5, 6}

# Intersection
a_intersection_b = set_a & set_b # {3, 4}
```

```
# Difference
a_diff_b = set_a - set_b  # {1, 2}
b_diff_a = set_b - set_a  # {5, 6}

# Symmetric Difference
a_sym_diff_b = set_a ^ set_b  # {1, 2, 5, 6}
```

Operation	Description
Union	All unique items from both sets.
Intersection	Only items in both sets.
Difference	Items in the first set but not the second.
Symmetric Difference	Items in either set, but not both.

Note: Sets ignore duplicate elements automatically. If you add the same value twice, only one copy is stored in the set.

Sets are created using curly braces {} or the set() function.

In Python, a set is a built-in data structure used to store an unordered collection of unique items. Sets are very useful when you want to keep track of a group of things, but do not care about the order or duplicate values.

You can create a set using **curly braces {}**, similar to how lists and dictionaries have their own symbols. Alternatively, if you prefer or need to create an empty set, use **the set() function**.

Using curly braces {}, you can define a set and list its values immediately inside the braces. For example, writing {'apple', 'banana', 'cherry'} creates a set with three fruit names. It is important to use commas to separate items, much like a list. However, unlike lists, you cannot have duplicate items. Python automatically removes any duplicates.

- **Sets are unordered**, so items have no fixed position.
- **Every item in a set must be unique**. Duplicates are removed.
- You can **create a set with values using curly braces**.
- To make an **empty set**, always use **the set() function**, not just {}.


```
# Creating a set with curly braces
fruits = {'apple', 'banana', 'cherry'}
print(fruits)

# Attempting to make an empty set
empty_dict = {}
print(type(empty_dict)) # This is a dict, not a set

empty_set = set()
print(type(empty_set)) # This is a set
```

Expression	Result
{'red', 'blue', 'green'}	Set with three colours
set([1, 2, 2, 3])	Set with 1, 2, 3 (duplicates removed)
{}	Creates an empty dictionary, not a set

Note: To create an empty set, always use `set()` rather than `{}`. Using curly braces without values makes an empty dictionary, which behaves very differently from a set.

Sets are useful for data validation, removing duplicates, and membership testing.

Python sets are a built-in data structure that lets you store collections of unique items. They are particularly useful when you need to manage lists of values without duplicates, check if a value exists in a group, or validate data. The set data type is simple yet powerful and is a fundamental concept in programming with Python.

A set is created using curly braces `{ }` or with the built-in `set()` function. Sets can only contain unique, immutable elements, meaning you cannot have duplicate values, and each item must be of a type that does not change, like numbers or strings.

One key benefit of sets is that they automatically remove duplicate items. If you add repeated values, only one copy is kept. This makes sets handy for cleaning up lists where duplicates could cause problems, such as in email lists, user IDs, or any collection where uniqueness matters.

- **Validating data:** Check if an input appears in a predefined set of valid options.
- **Removing duplicates:** Convert a list to a set to eliminate repeated elements.
- **Membership testing:** Quickly check if an item is present in a collection.

```
# Example: Removing duplicates from a list
emails = ['alice@example.com', 'bob@example.com',
          'alice@example.com']
unique_emails = set(emails)
print(unique_emails)  # Output: {'alice@example.com',
                        'bob@example.com'}

# Example: Membership testing
allowed_users = {'alice', 'bob', 'carol'}
user = 'bob'
if user in allowed_users:
    print('Access granted.')
```

Operation	Benefit
Data Validation	Check input against allowed values.
Removing Duplicates	Ensures only unique items are kept.
Membership Testing	Faster checks compared to lists for large datasets.

Note: Important tip: Sets are unordered collections. This means items have no fixed position, and you cannot access them by index like you can with lists.

DICTIONARIES (KEY-VALUE PAIRS)

A dictionary in Python is a mutable data structure used to store key-value pairs.

A dictionary is one of the core data structures in Python. It allows you to store and organise data using a system of keys and values. Instead of accessing items by their position, as in a list, you access them using unique keys. This makes dictionaries very useful when you need to quickly look up a value based on some identifier.

You create a dictionary by placing pairs of keys and values inside curly braces. **Each key must be unique**, but values can be duplicated or of any type. Keys are commonly strings or numbers, while values can be any Python object.

Dictionaries are mutable, which means you can change their contents after they are created. You can add new key-value pairs, update existing values, or remove items. This flexibility makes them ideal for storing information that may change during your program's execution.

- **Dictionaries** use curly braces: {}.
- **Each item** is a key-value pair, separated by a colon.
- **Keys must be unique and immutable** (e.g., strings, numbers, tuples).

- Values can be of any data type and may be duplicated.
- Dictionaries are unordered (prior to Python 3.7—since then, they maintain insertion order).
- Dictionaries are mutable—you can add, modify, or remove items.

```
# Creating a dictionary
student = {
    'name': 'Aditi',
    'age': 20,
    'major': 'Computer Science'
}

# Accessing a value
print(student['name']) # Output: Aditi

# Adding a new key-value pair
student['year'] = 2

# Updating a value
student['age'] = 21

# Removing a key-value pair
del student['major']
```

Operation	Example / Explanation
Create dictionary	<code>data = { 'key1': 'value1', 'key2': 'value2' }</code>
Access value by key	<code>data['key1']</code> # returns 'value1'
Add / update value	<code>data['key3'] = 'new value'</code>
Delete key-value pair	<code>del data['key2']</code>
Get all keys	<code>data.keys()</code>
Get all values	<code>data.values()</code>

Note: Important tip: If you try to access a key that does not exist in the dictionary using square brackets, Python will give you a `KeyError`. To avoid this, you can use the `get()` method, which allows you to specify a default value if the key is missing—like `data.get('nonexistent', 'Not found')`.

Each key in a dictionary must be unique and immutable (such as a string, number, or tuple).

In Python, a dictionary is a collection that stores data using key-value pairs. One important rule is that each key in the dictionary must be both unique and immutable. This means that you cannot have two pairs with the same key, and you can only use objects as keys if they do not change during the dictionary's life.

A dictionary's **key** acts as a label to access its associated value. For example, you can use a person's name to look up their phone number. Because a dictionary uses keys to find values quickly, each key must always be unique. If you try to use the same key more than once, the last value given to that key will replace any previous value.

Keys must also be immutable. In Python, an immutable object is one that cannot be changed after it is created. Common immutable types include strings, numbers (integers and floats), and tuples (but only when their contents are also immutable). Lists and dictionaries themselves cannot be used as dictionary keys because they can change.

- Keys are case-sensitive. "Name" and "name" are treated as different keys.
- You can use numbers, strings, or tuples as dictionary keys.
- Lists, dictionaries, and other mutable types cannot be used as keys.
- If you assign a value to an existing key, the old value is overwritten.

```
# Valid keys
phonebook = {
    "Alice": "0432 123 456",
    2024: "This is a year",
    ("Sydney", "NSW"): "City and state"
}

# Invalid key (will cause error)
# phonebook[[1, 2, 3]] = "Cannot use a list as a key"
```

Key Example	Valid as Dictionary Key?
"hello"	Yes
42	Yes
(4, 5)	Yes
[1, 2]	No
{"x": 1}	No

Note: Important tip: If you need to use a collection as a key in a dictionary, make sure it is a tuple and all its elements are also immutable. This helps ensure your programs are reliable and free from subtle bugs.

Dictionaries provide fast, efficient lookups for retrieving values associated with a specific key.

Dictionaries are a built-in data structure in Python used to store data as key-value pairs. Instead of accessing data by its position (like with a list), you access values by using a unique key. This way of storing and looking up data is very efficient, making dictionaries a popular choice whenever you need to associate one piece of information with another.

For example, if you want to store information about countries and their capitals, you can use the country name as the **key** and the capital city as the **value**. This means that looking up the capital city for a given country is both quick and simple.

When you use a dictionary, you are able to get the information you want directly by providing the key. Python takes care of finding the value for you, no matter how many pairs are stored in the dictionary. This is much faster than searching through a list, especially as your data grows.

- **Dictionaries use special rules** internally (hashing) to make lookups almost instant.
- **Each key** must be unique within the dictionary.
- **Values can be of any type** and can even be another dictionary or list.

```
capitals = {
    'Australia': 'Canberra',
    'New Zealand': 'Wellington',
    'Canada': 'Ottawa'
}

# Fast lookup by key
print(capitals['Australia']) # Output: Canberra
```

Key	Value
'Australia'	'Canberra'
'New Zealand'	'Wellington'
'Canada'	'Ottawa'

Note: Keys in a dictionary must be immutable, such as strings, numbers, or tuples. Avoid using lists or other dictionaries as keys.

You can add, modify, or remove key-value pairs using Python's syntax: `dict[key] = value`.

Dictionaries are one of Python's most useful data structures. They let you store information using keys and values. This format makes it easy to look up, add, change, or delete pieces of information quickly. In Python, dictionaries use curly braces `{ }` and each entry pairs a key with a value using a colon.

To **add or change** a key-value pair in a dictionary, use this syntax: `dict[key] = value`. If the key doesn't exist, it is added with the value you provide. If the key already exists, its value is updated.

You can also remove key-value pairs. The most common way is with the `del` statement. If you use `del dict[key]`, Python deletes the key and its associated value. Trying to delete a key that does not exist will cause an error.

- **Dictionaries** use keys to access values directly.
- **Adding or modifying** is done using `dict[key] = value`.
- **Removing pairs** is usually done with `del dict[key]`.
- **Keys must be unique** within each dictionary.
- **Values** can be any data type.

```
# Creating a dictionary
person = {"name": "Alice", "age": 30}

# Adding a key-value pair
person["city"] = "Melbourne"

# Modifying an existing pair
person["age"] = 31

# Removing a pair
del person["city"]
```

Action	Example
Add new pair	<code>person['email'] = 'alice@example.com'</code>
Change value	<code>person['age'] = 32</code>
Remove pair	<code>del person['email']</code>

Note: If you try to access or delete a key that does not exist in the dictionary, Python will raise a `KeyError`. You can avoid this by checking if the key is present using the `'in'` operator.

Dictionaries are widely used for mapping relationships and managing structured data in Python programs.

Dictionaries are versatile data structures in Python. They help you organise and manage data by mapping unique keys to corresponding values. This makes them ideal for representing relationships and handling structured information in your programs.

A dictionary lets you **look up values by their key quickly**. Think of it like a real-life dictionary where you search for a word (the key) to find its definition (the value). This model is useful whenever you need a fast way to access, update, or remove specific items in a collection.

Dictionaries use curly braces to group their key-value pairs. Each key and its value are separated by a colon, and pairs are separated by commas. Keys must be unique, and most often, they are strings or numbers. Values can be any data type, including numbers, strings, lists, or even other dictionaries.

- **Efficiently store and retrieve data** based on custom keys
- **Model real-world relationships** such as a person's name and phone number
- **Organise structured data** like settings, user profiles, or configurations

```
phone_book = {  
    'Alice': '0412 345 678',  
    'Bob': '0423 987 654',  
    'Charlie': '0400 555 777'  
}  
  
# Accessing a value by its key  
print(phone_book['Bob'])  # Output: 0423 987 654  
  
# Adding a new entry  
phone_book['Diana'] = '0411 222 333'  
  
# Updating an entry  
phone_book['Alice'] = '0412 111 111'  
  
# Checking if a key exists  
if 'Charlie' in phone_book:  
    print('Charlie is listed.')
```

Operation	Example
Create dictionary	<code>fruits = {'apple': 5, 'orange': 3}</code>
Get value	<code>fruits['apple'] # 5</code>
Add/update value	<code>fruits['banana'] = 7</code>
Remove value	<code>del fruits['orange']</code>
Check key existence	<code>'apple' in fruits # True</code>

Note: Important tip: Keys in a dictionary must be unique and immutable. You can use strings, numbers, or tuples as keys—but not lists or other dictionaries.

Dictionaries are also well-suited for processing data with named fields. For example, a user profile can be stored as a dictionary, with keys like 'first_name', 'email', and 'age'. This makes your code clearer and retrieval of specific details more straightforward.

Besides simple retrieval, you can use methods like `items()`, `keys()`, and `values()` to loop through a dictionary or access all keys and values separately. This is helpful when you need to process or display each entry in a structured dataset.

Note: Dictionaries do not maintain order in older versions of Python (before 3.7). In Python 3.7 and above, insertion order is preserved, meaning items remain in the order you add them.

ITERATING THROUGH COLLECTIONS

Understand the basics of iterating through different types of collections, such as lists, tuples, sets, and dictionaries.

Iteration is the process of accessing each item in a collection one at a time. In Python, you often use loops to go through elements in data structures such as lists, tuples, sets, and dictionaries. Knowing how to iterate through these collections is essential for processing and analysing data efficiently.

The most common loop for iteration is the **for loop**. With a for loop, you can step through every element in a collection, perform actions, or check for certain conditions.

Each collection type in Python—lists, tuples, sets, and dictionaries—behaves slightly differently when you iterate over it. Understanding these differences helps you choose the right approach for your data.

- **Lists and tuples** preserve order, so items are accessed in the order they appear.
- **Sets** do not preserve order, so the access order may differ each time.
- **Dictionaries** are made up of key-value pairs. By default, iterating over a dictionary accesses each key.

```
# Iterating over a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Iterating over a tuple
coordinates = (10, 20, 30)
for number in coordinates:
    print(number)

# Iterating over a set
colours = {'red', 'green', 'blue'}
for colour in colours:
    print(colour)

# Iterating over a dictionary's keys, values, and items
student = {'name': 'Ava', 'age': 19, 'grade': 'A'}
for key in student:
    print(key) # keys only
for value in student.values():
    print(value) # values only
for key, value in student.items():
    print(key, value) # key and value pairs
```

Collection Type	Iteration Result
List	Each item in order (e.g., 1, 2, 3)
Tuple	Each item in order (e.g., 'a', 'b', 'c')
Set	Each unique item, order not guaranteed
Dictionary	Each key by default; can access values or key-value pairs

Note: When working with sets or dictionaries, remember that the order of items may not be the same each time you run your code. If order is important, use lists or tuples.

Use loops (for and while) to access elements in a collection sequentially.

When working with data structures like lists, tuples, sets, and dictionaries in Python, you often need to process each item one at a time. This is called iterating through a collection. Python provides two main types of loops for this task: for loops and while loops.

A **for loop** is preferred when you want to go through each item in a collection, one by one, without needing to manage the position yourself. A **while loop** is more general. It keeps repeating a block of code as long as a certain condition stays true, which lets you control looping in different ways.

For loops are simple and clear for most collection tasks. You write a for loop by stating what variable will represent each item, the word 'in', and then the collection you want to iterate over. The body of the loop indented beneath will run once for every item.

- Use a for loop when you want to **process each element** without worrying about its position.
- Use a while loop when you **need more complex** control or want to update your loop condition yourself.
- **All Python collections**—lists, tuples, sets, and dictionaries—support for loops.
- While loops can be used with collections if you need to track an index or use other stopping conditions.
- Dictionaries need special care: looping over them by default gives you keys, not values.

```
# For loop with a list
numbers = [2, 4, 6, 8]
for number in numbers:
    print(number)

# While loop with a list
index = 0
while index < len(numbers):
    print(numbers[index])
    index += 1
```

Type	Typical Use Case
for loop	Stepping through items in a collection
while loop	Repeating until a condition changes, such as end of data or user input
for loop on a dictionary	Iterating over keys, or keys and values with items()

Note: If you want both the index (position) and the value in a for loop, use the `enumerate()` function.

Apply iteration techniques to perform actions on each item within a collection.

Iterating through a collection means processing each item in a group, one at a time. In Python, this is most often done using for loops. Collections could be lists, tuples, sets, or dictionaries. Iteration lets you perform actions with every element, such as printing, modifying, or evaluating values. Understanding iteration is essential for automating repetitive tasks and making your code more efficient.

When you **iterate** over a collection, you use a loop to access each item in order. Python offers clear ways to do this using the for loop. You can use iteration to read through a shopping list, search for a value, or calculate totals.

Let's look at a simple example. Imagine you have a list of numbers, and you want to print each one. A for loop will let you go through each item in the list without needing to know its position. The loop executes your action for every item.

- For loops are the most common way to iterate in Python.
- You do not need to know in advance how many items are in the collection.
- You can use iteration with lists, tuples, sets, and dictionaries.
- Code inside the loop block is run once for each item.
- You can use `continue` or `break` statements to control the loop.

```
numbers = [10, 20, 30, 40, 50]
for number in numbers:
    print("Number:", number)
```

Collection Type	Iteration Example
List	for x in [1,2,3]: print(x)
Tuple	for name in ('Ann', 'Bob'): print(name)
Set	for item in {'apple', 'banana'}: print(item)
Dictionary (keys)	for key in {'a': 1}.keys(): print(key)
Dictionary (values)	for value in {'a': 1}.values(): print(value)

Dictionaries are special, because they store pairs: a key and a value. By default, iterating through a dictionary gives you only the keys. If you want both the keys and values, use the `items()` method in your loop.

```
person = {"name": "Alex", "age": 30}
for key, value in person.items():
    print(key, ":", value)
```

Sometimes you may need to know the position of each item as you loop. The `enumerate()` function gives you both the index and the value. This is useful for tasks like numbering items or comparing positions.

```
names = ["Sara", "John", "Lily"]
for index, name in enumerate(names):
    print("Position:", index, "Name:", name)
```

Note: Important tip: Always be careful not to modify the collection itself (like adding or removing items) while you are iterating through it. This can cause unexpected results or errors.

Handle iteration over key-value pairs in dictionaries using appropriate methods (e.g., items()).

In Python, dictionaries are powerful data structures that store data as key-value pairs. When you need to access both the key and value at the same time, it is common to use special methods built into dictionaries. Understanding how to iterate through these pairs efficiently will help you process and manage your data more effectively.

The most common method for looping through both keys and values in a dictionary is using the **items()** method. This method returns each key-value pair as a tuple, allowing you to access them easily in a loop.

By default, if you just iterate through a dictionary using a for loop, you only get the keys. With the **items()** method, you can access both the key and its matching value in each step of the loop.

- **Dictionaries use curly braces { }** and store pairs in the format key: value.
- **The items() method** returns a view object containing (key, value) pairs.
- You can use `for key, value in dict.items():` to loop through both parts at once.
- This approach ensures your code works even when dictionaries are large or values are complex.
- It helps avoid mistakes, such as looking up values separately by key.

```
student_scores = {'Alice': 85, 'Bob': 72, 'Chloe': 91}
for name, score in student_scores.items():
    print(f"Student: {name}, Score: {score}")
```

Method	What it does
<code>for key in dict</code>	Loops through keys only
<code>for value in dict.values()</code>	Loops through values only
<code>for key, value in dict.items()</code>	Loops through both keys and values

Note: You can safely change values inside the loop if needed, but avoid adding or removing keys while iterating to prevent errors. Using the **items()** method is the most direct and readable way to work with both parts of each dictionary entry.

Utilise built-in Python functions (like enumerate() and zip()) for advanced iteration needs.

Python offers a range of built-in functions that make working with collections easier and more efficient. Two of the most useful ones are **enumerate()** and **zip()**. These functions help you solve common problems when looping through lists, tuples, and other collection types. Knowing how to use them can make your code cleaner, shorter, and less prone to mistakes.

When you use a regular for loop, you can access each item in a collection. However, sometimes you need more information—such as the position of the item or you want to combine more than one collection at the same time. This is where **enumerate()** and **zip()** help.

The `enumerate()` function adds a counter to an iterable. This means you can keep track of the index (position) of each item automatically, which is a common need in programming. The `zip()` function allows you to loop over two or more collections at the same time, pairing their items together. This is very handy when working with related data stored separately.

- **Use `enumerate()`** when you need both the item and its index in a loop.
- **Use `zip()` to loop** through multiple collections at once.
- Both functions return special objects, so you often use them directly in for loops.

```
# Example of enumerate()
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Example of zip()
names = ['Alice', 'Bob', 'Charlie']
ages = [24, 30, 18]
for name, age in zip(names, ages):
    print(f"{name} is {age} years old")
```

Function	Typical Use
<code>enumerate()</code>	Loop with item position
<code>zip()</code>	Loop through multiple collections together

Note: Important tip: If the collections you pass to `zip()` are not the same length, `zip()` will stop at the shortest collection. To use the index and item with `enumerate()`, remember the index always starts from zero, unless you set a different starting point.

7. File Handling and Regular Expressions

READING AND WRITING TEXT FILES

Learn how to open, read, and write text files in Python using built-in functions.

Reading and writing text files are essential skills for working with data in Python. Many programs need to save information or retrieve it from files. Python makes it easy to handle text files with simple and readable code.

In Python, the **open()** function is used to access files. With this function, you can open a file to read its contents, write new information, or even add more data to the end.

There are three main steps for working with files: open the file, do something with the file, then close the file. Python's built-in functions, like `open()`, make this process straightforward. Let's break down these steps.

- Open the file using **open()**
- **Read or write** content
- Close the file with **close()**

To open a file, you use the `open()` function. It needs at least one argument: the file name or file path. You can also tell `open()` how you want to work with the file by using a mode. The most common modes are 'r' for reading, 'w' for writing (which overwrites an existing file), and 'a' for appending new data to the end.

```
# Reading a text file
file = open('example.txt', 'r')
contents = file.read()
print(contents)
file.close()

# Writing to a text file
file = open('output.txt', 'w')
file.write('Hello, world!')
file.close()
```

When reading a file, `.read()` loads the entire contents as a single string. If the file is large, you can read one line at a time with `.readline()` or get all lines as a list using `.readlines()`. Writing follows a similar pattern. Use `.write()` to place text in the file. Remember, writing in 'w' (write) mode clears the file first.

Mode	Description
r	Read (default). Fails if file does not exist.
w	Write. Creates new or overwrites existing file.
a	Append. Adds data to the end of the file.
r+	Read and write. File must exist.

Python provides a safer way to handle files by using the with statement. This automatically closes the file when you are done, even if there is an error. This helps prevent mistakes that can happen if a file is left open.

```
with open('input.txt', 'r') as file:
    lines = file.readlines()
    for line in lines:
        print(line.strip())
```

Handling errors is important. If a file path is wrong, or a file does not exist, Python will raise an error. You should use try-except blocks if your program must handle such problems gracefully.

```
try:
    with open('missing.txt', 'r') as file:
        data = file.read()
except FileNotFoundError:
    print('File not found. Please check the name and path.')
```

Note: Always specify the correct mode ('r', 'w', or 'a') when opening files. For writing or appending, be careful: using 'w' will erase the file's current contents. Use the with statement to automatically manage file closing.

Understand the differences between file modes such as read ('r'), write ('w'), and append ('a').

When working with files in Python, the file mode determines how you open and interact with a file. The mode tells Python whether you want to read from a file, write new data to it, or add information onto the end. Choosing the right mode prevents errors, accidental data loss, and ensures your program works as expected.

The three most common file modes are **'r' (read)**, **'w' (write)**, and **'a' (append)**. Each has a specific use and behaves differently when opening files.

The **'r'** mode stands for read. It is the default mode. You can only read data from the file. If the file does not exist, Python will raise an error. No changes are made to the file.

The **'w'** mode stands for write. Use this mode to create a new file or overwrite an existing file. If the file does not exist, it will be created. **WARNING:** If the file does exist, all previous contents are deleted, so be careful.

The **'a'** mode stands for append. This mode allows you to add new data to the end of an existing file. If the file does not exist, it is created. With append, the original contents are kept safe, and only new lines or text are added after them.

- **'r' (read)** – Opens the file for reading only. The file must exist.
- **'w' (write)** – Opens for writing only. Creates a new file or overwrites an existing one.
- **'a' (append)** – Opens for writing at the end. Adds new content after existing data.

```
with open('example.txt', 'r') as file:
    data = file.read()  # Read contents

with open('example.txt', 'w') as file:
    file.write('Now the file only has this line')  # Overwrites
all contents

with open('example.txt', 'a') as file:
    file.write('\nAdded to the end of the file')  # Appends a new
line
```

Mode	Behaviour
'r'	Read from existing file. Fails if file does not exist.
'w'	Write to file. Overwrites or creates new file.
'a'	Append to existing file. Creates file if missing.

Note: Important tip: Always double-check your file mode before writing code. Using 'w' accidentally can permanently remove data from your file. Use 'r' mode when you only need to read, and 'a' mode if you want to add to a file without losing existing contents.

Practice handling file paths for both relative and absolute locations.

Understanding how to use file paths is crucial for reading and writing files in Python. Your program needs to know where to find files you want to read, or where to place files you are about to create. There are two main types of file paths: relative and absolute paths. As a beginner, it is important to practise using both types so you can confidently manage files in projects, on your computer, or even on remote systems.

A **relative path** tells Python how to reach a file or folder starting from your current working directory. An **absolute path** shows the exact location of the file or folder on your computer, starting from the root. If you learn the difference, you can reliably open, create and manage files, no matter where your program is running.

Suppose you have a file called 'data.txt' in the same directory as your Python script. If you run your script and simply use 'data.txt' as the path, Python will look for the file in your current directory. If the file sits in another folder, for example, 'docs/data.txt', then 'docs/data.txt' is the relative path. If you want to specify the file from anywhere on your system, you can use the absolute path, such as '/home/sammy/projects/docs/data.txt' on Linux or 'C:\Users\Sammy\projects\docs\data.txt' on Windows.

- **Relative paths** are shorter and flexible, but depend on your script's location.
- **Absolute paths** are precise and do not depend on where your script runs.
- **Mixing up relative and absolute paths** can lead to errors like 'file not found'.
- Python provides the `os` and `pathlib` modules to help with file path management.

```
from pathlib import Path

# Absolute path
abs_path = Path('/home/sammy/documents/data.txt')

# Relative path
rel_path = Path('docs/data.txt')

print('Absolute Path:', abs_path.resolve())
print('Relative Path:', rel_path.resolve())
```

Path Type	Example (Linux/Windows)
Relative	notes/summary.txt
Absolute	/Users/Jamie/projects/report.txt
Absolute	C:\Users\Jamie\Documents\report.txt

Note: Important tip: Always check which directory your script is running in. You can use `Path.cwd()` to get the current directory. If you are unsure, prefer using absolute paths for clarity, or carefully test your relative paths.

Recognise and manage errors when working with files, such as missing files or permission issues.

When working with files in Python, different things can go wrong. Sometimes a file might not exist where you expect it. Other times, you may not have permission to open or write to a file. If your code does not handle these situations, your program can crash or behave unpredictably. Recognising and managing these errors is essential for building reliable programs.

Python provides a way to handle problems with file operations. This is usually done through **error handling using try and except blocks**. When you use these tools, you can catch errors if they happen and decide what your program should do next.

The most common errors you will encounter with files include trying to open a file that does not exist, attempting to write to a file when you do not have permission, or reading a file that is currently in use by another program. Let's look at each of these error types using practical examples.

- **FileNotFoundError** – Raised if a file you are trying to open does not exist.
- **PermissionError** – Occurs when your program lacks the correct permission to read or write a file.
- **IOError or OSError** – General errors for input or output operations, such as disk problems or interrupted connections.

```
try:
    with open('example.txt', 'r') as file:
        data = file.read()
except FileNotFoundError:
    print('The file does not exist.')
except PermissionError:
    print('You do not have permission to access this file.')
except Exception as e:
    print(f'An unexpected error occurred: {e}')
```

Error Type	Typical Cause
FileNotFoundError	Trying to open a file that is missing
PermissionError	Lacking rights to read or write to a file
OSError	General issues (e.g., disk full, file locked)

When you use a try and except block, your program can decide what to do if something goes wrong. For example, you might show a message to the user, ask for a different file name, or try to create the file if it is missing.

Many professional programs also log file errors for future review. Logging is helpful when diagnosing problems that users experience. You can use **the built-in logging module in Python** to record errors.

- **Always check if a file exists** before trying to open it, especially for reading.
- **Handle errors** gracefully with user-friendly messages.
- **Use the correct file path** to avoid confusion between relative and absolute paths.
- Consider using **context managers** (the with statement) to manage files. They help close files automatically even when errors occur.

```
import os
path = 'data.txt'
if os.path.exists(path):
    try:
        with open(path, 'r') as f:
            print(f.read())
    except PermissionError:
        print('No permission to read this file.')
else:
    print('File not found.')
```

Note: Important tip: If your program is likely to run on different systems or with various permissions, always expect errors and handle them. This will make your programs reliable and user-friendly.

Close files properly to ensure data is saved and resources are released.

When you work with files in Python, it's important to properly close them once you're done. Closing a file means telling Python you have finished using it. This step is crucial for saving any changes and freeing up computer resources.

Leaving files open can cause problems. **If the file is not closed, your data might not be written to disk properly, and other programs might not be able to access it.** Always close your files to prevent these issues.

You can close a file by calling the `close()` method on your file object. This is easy to do, but it can be easy to forget. Open files also take up system resources, such as memory and file handles, which are limited. Always make sure you close files when you have finished reading or writing.

- **Closing a file** flushes any remaining data to disk.
- It **releases system resources** used by the file.
- It **helps prevent bugs and errors** related to unclosed files.
- It **allows other programs or scripts to** access the file safely.
- Forgetting to close files can lead to data loss or file corruption.

```
file = open('example.txt', 'w')
file.write('Hello, world!')
file.close() # Makes sure data is saved and resources are
released
```

Action	Result
Write to file, then close	Data saved to file
Write to file, forget to close	Data might not be saved correctly
Open too many files without closing	System runs out of resources

Note: A safer way to handle files in Python is to use a `with` statement. This will automatically close the file for you, even if there is an error. Always prefer the `with` statement when possible.

WORKING WITH CSV-STYLE DATA

Understand how to read and write CSV-style data using Python's built-in and external libraries.

CSV stands for Comma-Separated Values. It is a simple file format used to store tabular data, such as numbers and text in rows and columns. Many spreadsheet and database programs use CSV files because they can be opened with plain text editors or applications like Microsoft Excel.

Python provides excellent support for working with CSV files. The built-in **csv** module is included in the Python standard library and provides everything you need to read from and write to CSV files. For more complex tasks, you can use external libraries like **pandas** to work with large datasets or to perform data analysis.

When you read or write CSV files, you are dealing with structured data. Each line of the file represents a row, and each value is separated by a comma or another delimiter like a semicolon. Handling CSV files is a common task in programming, especially when exchanging data between programs.

- **Python's csv module** is part of the standard library.
- **pandas** provides advanced CSV handling and data manipulation.
- **CSV files** use plain text, which makes them easy to read and edit.
- You can specify different delimiters if your CSV uses tabs or semicolons instead of commas.
- Handle file encoding (such as UTF-8) to avoid reading errors with special characters.

```
import csv

# Reading a CSV file using the built-in csv module
with open('example.csv', newline='', encoding='utf-8') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)

# Writing to a CSV file using the csv module
rows = [['Name', 'Age'], ['Alice', 30], ['Bob', 25]]
with open('output.csv', 'w', newline='', encoding='utf-8') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(rows)

# Using pandas for reading CSV
import pandas as pd
df = pd.read_csv('example.csv')
print(df.head())

# Writing DataFrame to CSV
df.to_csv('output.csv', index=False)
```

Task	Example Library/Method
Read basic CSV file	csv.reader()
Write to CSV file	csv.writer()
Load CSV for analysis	pandas.read_csv()
Save DataFrame to CSV	pandas.to_csv()

Note: Remember to always close files, or use 'with' statements to handle files safely. If working with large or complex CSV files, pandas can save you a lot of time and makes handling the data much easier.

Learn to parse CSV files and handle common issues such as delimiters and quoting.

Comma-separated values (CSV) files are a common way to store tabular data, where each line represents a row and values are separated by a special character, usually a comma. Learning to parse CSV files is essential for working with data in Python, especially when sharing information between different programs or platforms.

The **csv module** in Python makes it easy to read and write CSV files. It takes care of many issues such as different delimiters, quoting, and special characters.

While a basic CSV file uses commas as separators, sometimes data contains commas or quotes as part of the values. In those cases, the file may use different delimiters (like tabs or semicolons), and values with special characters are often enclosed in quotes. Knowing how to handle these details prevents errors when reading or saving data.

- **CSV files** use a delimiter to separate values, commonly a comma, but sometimes other characters such as tabs or semicolons.
- Fields containing the delimiter character, line breaks, or quotes **must be enclosed in quotation marks, usually double quotes**.
- **Extra spaces, missing values, or inconsistent quoting** can cause issues during parsing.
- The Python csv module manages these problems by allowing you to **specify delimiters and quoting options**.
- **Always review the CSV file** to understand its structure before writing code to parse it.

```
import csv

# Reading a CSV file with commas as delimiters
with open('example.csv', newline='') as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        print(row)
```

```
# Reading a CSV file with a semicolon as a delimiter
with open('example_semicolon.csv', newline='') as csvfile:
    reader = csv.reader(csvfile, delimiter=';')
    for row in reader:
        print(row)
```

Problem	How to Handle
Different delimiter (e.g., semicolon, tab)	Use the delimiter option in csv.reader
Values contain commas	Values must be enclosed in quotes
Quotation marks inside values	Use double quotes by doubling them (e.g., "She said, ""Hello""")
Missing or extra spaces	Trim or process data when reading

For more complex CSV files, such as those where the first row holds column names, you can use `csv.DictReader`. This lets you access each value by its column name, which helps if the column order changes or is not consistent.

```
import csv

with open('people.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        print(row['First Name'], row['Email Address'])
```

Note: Important tip: Always check the file's encoding (such as UTF-8 or ISO-8859-1). If you see strange characters or errors, try opening the file with a different encoding using the `encoding` parameter in `open()`. This helps avoid data loss or parsing failures.

Practice filtering, sorting, and extracting data from CSV files using Python.

CSV (Comma-Separated Values) files are used to store structured data in a simple text format. Each row holds a record, and columns are separated by commas. Python can easily read, filter, sort, and extract data from these files. Understanding this process is essential for handling data in many real-world tasks.

Python includes a built-in library called **csv** for working with these files. The library lets you access CSV data row by row, search for specific values, sort entries, or extract key information easily with just a few lines of code.

Common tasks include selecting only the data you need (filtering), arranging the data in a specific order (sorting), and pulling out values from certain columns (extracting). Let's walk through some examples.

- **Filtering** lets you keep only the rows that match your criteria, such as students who passed an exam.
- **Sorting** arranges the rows by a particular field, like names alphabetically or marks from highest to lowest.
- **Extracting** targets one or more columns, such as all email addresses or product codes.

```
import csv

# Example: Filter students with marks above 70 and sort by name
with open('students.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    filtered = [row for row in reader if int(row['Marks']) > 70]
    sorted_rows = sorted(filtered, key=lambda x: x['Name'])
    emails = [row['Email'] for row in sorted_rows]
    for row in sorted_rows:
        print(row['Name'], row['Marks'])
    print('Emails:', emails)
```

Task	Description
Filtering	Keep rows based on a condition (e.g., Marks > 70)
Sorting	Arrange rows by a field (e.g., Name A-Z)
Extracting	Select specific columns (e.g., Email)

Note: Always check your CSV file's structure before reading it in Python. Column names and formatting may vary, so verify field names like 'Marks' or 'Email' match your file.

Implement error-handling techniques when working with CSV data.

When working with CSV (Comma Separated Values) data in Python, it is important to manage possible errors. CSV files can have inconsistent formatting, missing values, or incorrect file paths. Good error-handling ensures your program runs smoothly and helps you quickly find issues. This approach also prevents your program from stopping unexpectedly if something goes wrong.

Using error-handling when working with CSV files can help you **catch and manage problems such as missing files, invalid data, or formatting errors**. It also makes your code easier to maintain and debug.

The most common form of error-handling in Python uses the try and except keywords. You put the code that might cause issues inside a try block. If an error occurs, the except block will run instead of letting your whole program break. This lets you control how your program reacts to unexpected situations.

- Protect your program from crashing if the CSV file is missing or moved.
- Handle incorrectly formatted CSV data gracefully.
- Manage permission errors (e.g., trying to write to a file you cannot access).
- Provide clear error messages to help with debugging.
- Skip or log problematic rows instead of failing completely.

```
import csv

try:
    with open('data.csv', 'r', newline='') as file:
        reader = csv.reader(file)
        for row in reader:
            try:
                # Example: expect each row to have 3 columns
                if len(row) != 3:
                    raise ValueError('Incorrect number of
columns')

                print(row)
            except ValueError as ve:
                print(f"Row skipped: {ve}")
except FileNotFoundError:
    print('The CSV file was not found.')
except PermissionError:
    print('Permission denied when accessing the file.')
except Exception as error:
    print(f'An unexpected error occurred: {error}')
```

Error Type	Typical Cause
FileNotFoundError	File does not exist at specified path
PermissionError	No access rights to read/write the file
ValueError	Invalid values or wrong number of columns in data

Note: Important tip: Use specific exceptions like `FileNotFoundError` and `ValueError` to handle predictable issues more effectively. Only use a general Exception as a last resort so you do not hide other problems.

Explore how to automate data processing tasks involving CSV files.

CSV (Comma-Separated Values) files are a popular way to store and move data between programs. Many applications, like spreadsheets and databases, can read and write CSV files. In Python, automating data processing with CSVs saves time and helps you avoid manual errors. You can use Python to read, filter, modify, and generate CSV files efficiently.

Before working with CSV files, you should understand their structure. A CSV file is a plain text file where each line is a record, and each field within the record is separated by a comma. Python's **csv module** provides tools to read and write these files easily. Automating CSV tasks is common in data analysis, reporting, and process automation.

When you automate CSV processing, you can: extract only the data you need, combine data from multiple files, change data formats, and create new files as results. This is especially useful for repetitive tasks, like preparing reports or cleaning data.

- Read data from a CSV and display it in a user-friendly way.
- Filter rows based on specific criteria (like dates or numbers).
- Combine multiple CSV files into one for larger analyses.
- Remove duplicates or unwanted data automatically.
- Change values, add new columns, or calculate summaries.

```
import csv

# Open a CSV file for reading
with open('people.csv', newline='') as infile:
    reader = csv.DictReader(infile)
    for row in reader:
        # Show only people older than 30
        if int(row['age']) > 30:
            print(f"{row['name']} is {row['age']} years old.")
```

Task	Python CSV Technique
Reading data	csv.reader, csv.DictReader
Writing data	csv.writer, csv.DictWriter
Handling headers	Use fieldnames parameter
Filtering rows	if-conditions in for loop
Updating files	Read, process, then write new file

Note: Always check your CSV data for issues like missing fields, extra spaces, or unusual characters. The csv module has settings to handle these, but test with sample data before automating large jobs.

FILE PATHS AND ERROR HANDLING

Understand how to specify and use file paths to access files in Python.

A file path tells your Python program where to find or save a file on your computer. Understanding how to work with file paths is an essential part of programming in Python, especially when you need to read data from or write data to files.

File paths can be **absolute**, which means they point to a fixed location starting from the root of your file system. They can also be **relative**, pointing to a file location based on where your program is currently running.

When you want to access a file, you give Python the file path as a string. This tells Python which folder to look in, and which filename to use. Typing the path correctly is very important or you may get errors saying the file cannot be found.

- **An absolute path gives the whole address** (for example: C:\Users\emma\docs\notes.txt or /home/emma/docs/notes.txt).
- **A relative path** shows how to get to the file from the current working directory (such as data/records.csv).
- **Use double backslashes (\\) or raw strings (r'...')** when writing Windows paths, because a single backslash is special in Python.

```
# Absolute path on Windows
f = open('C:\\Users\\emma\\documents\\file.txt')

# Absolute path on Mac/Linux
f = open('/home/emma/documents/file.txt')
```

```
# Relative path (subfolder)
f = open('data/info.txt')

# Using raw string for Windows path
f = open(r'C:\Users\emma\documents\file.txt')
```

Path Type	Example
Absolute (Windows)	C:\Users\andrew\report.txt
Absolute (macOS/Linux)	/Users/andrew/report.txt
Relative	project/data/input.csv

Note: Important tip: If you try to open a file with an incorrect path, Python will show an error message. Always check the spelling, folder, and capitalisation of your file paths. Use the `os.path` or `pathlib` modules to help build paths in a way that works on all operating systems.

Learn methods for handling errors that may occur during file operations.

When working with files in Python, errors can happen for many reasons. The file you want to open might not exist. You may not have permission to read or write it. The file could be in use by another program, or there might not be enough space to save changes. Python provides effective ways to handle these errors so your program does not crash and you can respond helpfully.

To manage errors, Python uses a system called **exception handling**. Exceptions are special messages for errors or unexpected events in your code. You can use `try` and `except` statements to catch and respond to exceptions, helping your program work smoothly—even if something goes wrong when dealing with files.

The basic idea is to try an operation (like opening a file) and catch any errors that happen. This lets you show a useful message, try a backup plan, or simply stop the program in a safe way. Proper error handling makes your code safer and more user-friendly.

- **FileNotFoundError:** Tries to access a file that does not exist.
- **PermissionError:** Tries to read or write a file without enough rights.
- **IsADirectoryError:** Tries to open a directory like a file.
- **IOError or OSError:** Covers general input/output problems, such as disk errors.

```
try:
    with open('mydata.txt', 'r') as file:
        contents = file.read()
        print(contents)
```

```
except FileNotFoundError:
    print('The file was not found. Please check the file name and
    path.')
except PermissionError:
    print('You do not have permission to access this file.')
except Exception as e:
    print('An unexpected error occurred:', e)
```

Error Type	Typical Reason
FileNotFoundError	The file does not exist at the specified path.
PermissionError	Insufficient rights to access, read, or write the file.
OSError	A general input/output issue, such as a full disk or hardware failure.

Note: Important tip: Always catch the most specific errors first. This helps you give clearer feedback and avoid hiding problems.

Apply best practices for managing file locations and organising your data.

Managing file locations and organising your data correctly are important parts of any Python project. Good organisation helps you find files later, makes your programs more reliable, and allows others to understand your work. It also reduces the risk of errors and data loss.

When you begin a Python project, **plan where your files will live**. This includes both your script files and your data files, such as text files, CSVs, or images. Avoid scattering files across your computer. Instead, keep related items together in a single folder structure for better organisation.

Give your files meaningful names that describe their contents. For example, use 'customer_list.csv' instead of 'data1.csv'. This makes it easier to understand a project at a glance and avoid confusion, especially when you return to your code after some time.

- **Create a dedicated folder** for each Python project.
- **Group similar files** in separate subfolders, such as 'data', 'output', or 'scripts'.
- **Avoid using spaces** in file or folder names—prefer underscores (_) or hyphens (-).
- **Use clear, descriptive names** for your files.
- **Keep your data and code files separate** when possible.
- **Document the location and purpose** of files in a README file.

```
# Example: Creating and using organised file paths
import os

# Define base project folder
data_folder = os.path.join('project_name', 'data')
```

```
output_folder = os.path.join('project_name', 'output')

# Use these paths in your code
input_file = os.path.join(data_folder, 'customer_list.csv')
output_file = os.path.join(output_folder, 'cleaned_customers.csv')

with open(input_file, 'r') as infile:
    data = infile.read()
```

Practice	Benefit
Meaningful file names	Easier to understand content at a glance
Consistent folder structure	Reduces confusion, makes navigation simple
Keep data and code separate	Prevents accidental changes or deletes
Document file roles	Helps others understand and reuse your project

Note: Important tip: Never use hard-coded absolute paths (like C:\Users\YourName\Documents) in your programs. Use relative paths and Python's 'os' or 'pathlib' modules to make your code portable and easier to maintain.

Use Python tools to prevent and recover from common file-related errors.

When you work with files in Python, there are many things that can go wrong. Files may not exist, you might not have permission to access them, or they could be in use by another program. Understanding how to handle these errors is an essential skill for anyone writing reliable Python programmes.

Python provides **built-in error handling tools** that help you manage problems when working with files. By using these tools, you can prevent your programmes from crashing and give users clearer information about what went wrong.

The most common way to handle file errors is to use the try-except statement. This allows you to attempt actions that might fail and respond sensibly if an error occurs. For example, you can try to open a file and, if the file is missing, display an error message instead of letting your programme stop unexpectedly.

- **Use try-except** to catch errors when opening, reading, or writing files.
- **Check if a file exists** before trying to open it, using tools like `os.path.exists`.
- **Handle specific exceptions** (like `FileNotFoundError` and `PermissionError`) to give more helpful messages.
- **Always close files properly**, even if an error happens.

- Use the with statement for opening files. This automatically manages closing the file.

```
import os

file_path = 'example.txt'

try:
    if os.path.exists(file_path):
        with open(file_path, 'r') as file:
            content = file.read()
            print(content)
    else:
        print('The file does not exist.')
except FileNotFoundError:
    print('File not found error.')
except PermissionError:
    print('You do not have permission to access this file.')
except Exception as e:
    print(f'An unexpected error occurred: {e}')
```

Error Type	When it Happens
FileNotFoundError	The file does not exist at the location specified
PermissionError	You do not have permission to read or write the file
IsADirectoryError	You try to open a directory as if it were a file

Note: Important tip: Always use the with statement when working with files in Python. It helps prevent errors by automatically closing files, even if something goes wrong during reading or writing.

Practice writing Python code that reads from and writes to files safely.

Reading from and writing to files is a key skill in Python programming. Being able to handle files safely means your code can create, read, update, and close files without causing errors or data loss. In this section, you'll learn how to use Python's built-in tools for file handling. You'll see how to open files for reading and writing, deal with common pitfalls, and use best practice techniques to reduce buggy or unsafe code.

Whenever you work with files, you need to **open** the file, **read or write** the data, and then **close** the file when you're done. Forgetting to close a file can cause data to be lost or files to become locked.

Python provides the `open()` function to work with files. You can use different modes for opening a file. The most common are 'r' for read mode and 'w' for write mode. Use 'a' to append data to the end of a file. If you try to read a file that doesn't exist, you'll get an error. That's where error handling comes in. By using try and except blocks, you can make your program more reliable, even if mistakes or unexpected situations occur.

- Always close files after reading or writing.
- Use 'with open' to automatically close files safely.
- Handle exceptions (errors) when files are missing or in use.
- Distinguish between modes: 'r' (read), 'w' (write), 'a' (append), 'x' (exclusive creation).
- Be careful: opening in 'w' mode will overwrite the file.

```
try:
    # Safest way to open files is using 'with', which auto-closes
    them
    with open('data.txt', 'r') as f:
        contents = f.read()
    print("File read successfully:")
    print(contents)

    # Writing to a file
    with open('output.txt', 'w') as file:
        file.write('This is some new text.\n')
        file.write('Each write adds another line.\n')
    print("File written successfully.")
except FileNotFoundError:
    print("Error: The file does not exist.")
except IOError as e:
    print(f"An input/output error occurred: {e}")
```

Mode	Description
'r'	Read (default). File must exist.
'w'	Write. Creates or overwrites the file.
'a'	Append to the end of the file.
'x'	Create new. Fails if file exists.

Note: Important tip: Always use the 'with open(...) as ...:' statement for file operations. It automatically closes your files, even if something goes wrong. This protects your data and keeps your program safe.

INTRODUCTION TO REGULAR EXPRESSIONS

Learn the basics of regular expressions (regex) for pattern matching within text.

Regular expressions, often called regex, let you describe patterns in text. They work like a search language. You can use them to find, match, or replace specific parts of strings. Regular expressions are built into Python, making it easy to sift through large files and pick out what you need. Regex is useful for checking formats, cleaning messy data, or extracting useful information.

A regular expression is a **sequence of characters** that describes a search pattern. For example, a regex can match an email address, a phone number, or any set of letters. Think of regex like a toolkit – with a few special symbols, you can build simple or complex searches to suit almost any job.

Regex uses special characters to define rules for matching. Some rules are simple: a dot (.) matches any one character, and an asterisk (*) means 'zero or more' of the previous character. You often use square brackets [] to match a set of characters, and the caret (^) or dollar sign (\$) to match the start or end of a line.

- **A dot (.)** matches any character except a new line.
- **A star (*)** repeats the character before it zero or more times.
- **Square brackets [a-z]** match any single character in the range.
- **A caret (^)** marks the start of a line.
- **A dollar sign (\$)** marks the end of a line.
- **A question mark (?)** makes the previous character optional.

```
import re

# Check if text contains 'cat'
result = re.search(r"cat", "The black cat sat")
if result:
    print("'cat' found!")

# Match an email address
email = "person@example.com"
pattern = r"[\w.-]+@[ \w.-]+\.\w+"
if re.match(pattern, email):
    print("Valid email!")
```

Symbol	Meaning
.	Any single character except new line
*	Zero or more of the previous element
[abc]	Matches 'a', 'b', or 'c'
^	Start of string or line
\$	End of string or line

Note: Important tip: Use raw strings (prefix with r) in Python, like r"pattern". This avoids confusion with escape sequences.

Understand the syntax and structure of regular expressions, including special characters and quantifiers.

Regular expressions (often shortened to 'regex') are a special way to search, match, and extract patterns from text. They allow you to find specific words, numbers, or sequences within larger bodies of text. Understanding their syntax is key to using them effectively in Python.

The structure of a regular expression relies on using unique combinations of **special characters** and **quantifiers** to control the search pattern. You can build simple searches, like finding an exact word, or complex patterns, such as email addresses or phone numbers.

A regular expression pattern is a sequence of characters. Some characters match themselves (like a or 9), but others have special meaning. These special characters and quantifiers help you describe a range of possible matches, not just fixed text.

- **Full stops (.)** match any single character except a new line.
- **Square brackets []** allow you to specify a set or range of characters. For example, [abc] matches 'a', 'b', or 'c'.
- **Caret (^)** marks the start of a string, and dollar sign (\$) marks the end.
- **Backslash (\)** escapes special characters, making them literal (e.g., \\$ matches an actual dollar sign).
- **Quantifiers like *** (zero or more), **+** (one or more), and **?** (zero or one) control how many times a character or group should appear.
- **{n}** specifies exact repeats, such as {3} for exactly three times.
- **{n,m}** means between n and m repeats, such as {2,4}.

```
import re

# Example: match repeated digits
pattern = r'\d{2,4}' # Matches 2 to 4 digits in a row
text = 'My PIN is 4593.'
result = re.findall(pattern, text)
print(result) # Output: ['4593']
```

Character	Meaning
.	Any character except new line
[abc]	Either 'a', 'b', or 'c'
^abc	Starts with 'abc'
abc\$	Ends with 'abc'
\d	Any digit (0-9)
*	Zero or more times
+	One or more times
?	Zero or one time

Note: Important tip: Use raw string notation in Python (prefix with r) for regex patterns. This avoids confusion with backslashes (e.g., r'\d+' instead of '\d+').

Practice using regular expressions to find, filter, and extract specific data from files or user input.

Regular expressions (or regex) are a powerful way to search for and work with patterns in text. By using regular expressions, you can quickly find specific words, check if data follows a particular format, or pull out only the information you need from a much larger file. This practice is especially useful when working with large log files, user input, or data exports.

Regular expressions act like a set of instructions for matching certain sequences of characters. For example, you can use a regular expression to **find** phone numbers in a list, **filter** lines that contain an email address, or **extract** dates from a text file.

Let's look at how you'd use Python's built-in re module to put this into practice. Imagine you have a text file containing user-submitted information, and you want to pull out only the email addresses. The re module helps with tasks like this by allowing you to define patterns and then search for matches.

- **Finding:** Search for all phone numbers in a file.
- **Filtering:** Keep only lines that contain a valid email address.
- **Extracting:** Pull out postcodes from user-submitted addresses.

```
import re

# Example: Extract all email addresses from a text file
with open('users.txt', 'r') as file:
    content = file.read()

pattern = r'[\w\.-]+@[\w\.-]+'
emails = re.findall(pattern, content)

for email in emails:
    print(email)
```

Task	Example Regular Expression
Find all numbers	\d+
Filter email addresses	[\w.-]+@[\w.-]+
Extract dates (dd/mm/yyyy)	\d{2}/\d{2}/\d{4}

Note: Important tip: Regular expressions can look confusing at first, but you don't have to memorise all their patterns. Use online tools, and experiment with simple searches before moving on to more complicated extractions. Always test your expressions on real data to make sure they work as expected.

Apply regex in Python with the re module for practical text processing tasks.

Regular expressions, often called regex, are powerful tools for searching, matching, and processing text based on patterns. In Python, the built-in re module provides the main functions needed to work with regular expressions efficiently. These patterns let you perform complex searches, extract information, or edit text, making them invaluable for cleaning data, validating user input, and automating repetitive text handling tasks.

To use regular expressions in Python, you must **import the re module**. The re module contains functions such as match(), search(), findall(), and sub() for performing matches, extracting results, or making substitutions in text.

Let's say you want to find all phone numbers in a large text file, or check if an email address is valid. With regex, you can define a pattern, and the re module will find all strings that fit. For example, searching for all words starting with a capital letter is easy with a specific regex pattern.

- Compile a regex pattern using `re.compile()` for repeated use.
- Find all matching sub-strings with `re.findall()`.
- Check for a match at the start of text with `re.match()`.
- Search for a pattern anywhere in text using `re.search()`.
- Replace matched text with new content using `re.sub()`.
- Extract groups of information from a match using parentheses in your pattern.

```
import re

# Example: Extract all email addresses from a sentence
text = 'Contact alice@example.com or bob.smith@work.edu for details.'
email_pattern = r'[\w\.-]+@[\w\.-]+'
emails = re.findall(email_pattern, text)
print(emails)  # Output: ['alice@example.com', 'bob.smith@work.edu']

# Example: Replace all numbers with X
text2 = "Invoice: 2023-05-20, Amount: 1500"
new_text = re.sub(r'\d+', 'X', text2)
print(new_text)  # Output: Invoice: X-X-X, Amount: X
```

Function	Purpose
<code>re.match()</code>	Check if pattern matches at the beginning of the string
<code>re.search()</code>	Find first location of pattern within the string
<code>re.findall()</code>	Return all non-overlapping matches as a list
<code>re.sub()</code>	Replace matched patterns with a new string

Note: Important tip: Always test your regex pattern on sample data before using it in your main program. A small typo in your pattern can affect your results.

Recognise common use cases and limitations of regular expressions in programming.

Regular expressions, often called regex, are a powerful tool in many programming languages. They enable you to search, match, and manipulate text based on specific patterns. Understanding when and how to use regular expressions will help you tackle a variety of real-world text processing tasks more easily.

Regex is commonly used for text searching and manipulation. You might use regular expressions to: **validate email addresses, split log files into their parts**, extract data from user input, or ensure passwords meet certain rules. It is often used in data cleaning, especially when you need to work with unstructured or messy data.

Regular expressions can save you a lot of effort because they let you describe complex search criteria in a compact way. However, they are not always the best solution for every text handling problem. Understanding what regex can and cannot do will help you decide when to use it.

- Validation: Checking if an input matches a pattern, like phone numbers or emails.
- Searching: Finding all matches of a word or phrase within a large document.
- Substitution: Replacing parts of a string, such as swapping all dates to a uniform format.
- Splitting: Dividing text based on patterns, like breaking sentences into words.
- Extracting: Pulling out matching information, such as prices from invoices or URLs from text.

```
import re

email = "user@example.com"
pattern = r"^[\\w.-]+@[\\w.-]+\\.\\w{2,3}$"
if re.match(pattern, email):
    print("Valid email")
else:
    print("Invalid email")
```

Use Case	Example Regular Expression
Email validation	<code>^[\\w.-]+@[\\w.-]+\\.\\w{2,3}\$</code>
Find numbers	<code>\\d+</code>
Extract dates (DD/MM/YYYY)	<code>\\b\\d{2}/\\d{2}/\\d{4}\\b</code>

Despite their usefulness, regular expressions have clear limitations. They are difficult to read and maintain, especially as patterns become complex. Regex can also be inefficient for very large data or highly repetitive text. They are not suited to parsing structured data like XML or JSON, or to matching nested patterns.

- Complex patterns are hard to debug and maintain.
- Readability suffers when expressions get long.
- Errors in regex can lead to unexpected matches or security holes.
- Regular expressions cannot parse nested or recursive structures well.
- May not be suitable for high-performance scenarios dealing with huge files.

Note: Important tip: Use regular expressions for simple pattern matching or extraction. For complex or deeply structured data, consider using more specialised parsing libraries or tools.

PATTERN MATCHING AND DATA FILTERING

Learn to read from and write to text files in Python.

Reading from and writing to text files is a key skill in Python programming. It allows you to store, retrieve, and process data efficiently. Files are often used for saving settings, logs, or large amounts of information that do not fit comfortably into program memory.

When working with files, you open a file using the **open()** function. After using the file, you should close it to free up resources. Python makes file handling easy and reliable. This skill is often needed for tasks like loading templates, recording results, or reading data for analysis.

Text files are stored as plain characters, making them suitable for settings, logs, or data exchange between systems. Some common text file formats include .txt, .csv, and .log. The main operations with files are reading (collecting data from a file) and writing (saving data to a file).

- **Opening a file:** Needed before reading or writing. Use `open()`.
- **Reading:** Use methods like `read()`, `readline()`, or `readlines()`.
- **Writing:** Use `write()` or `writelines()`.
- **Closing a file:** Always close it with `close()` or by using a 'with' statement.
- **Handling errors:** Use try-except blocks to avoid crashes if files are missing or unavailable.

```
# Writing data to a text file
with open("example.txt", "w") as file:
    file.write("Hello, Python!\n")
    file.write("This is a new line.\n")

# Reading data from a text file
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```


Mode	Purpose
r	Read existing file
w	Write (creates new file or overwrites existing)
a	Append new data to end of file

Note: Whenever possible, use a 'with' statement when opening files. This ensures the file is closed properly, even if an error occurs, and keeps your code more readable and safe.

Handle CSV-style data and manage file paths and related errors.

Working with data files is a common task in Python programming. One popular format is CSV (Comma-Separated Values), which stores data in a table-like structure. You will also need to read, write, and sometimes create or move these files. Handling file paths and errors is essential for building reliable programs.

CSV files are simple text files where each line represents a row of data. Columns are separated by commas, for example: name,age,city. Handling CSV files in Python is straightforward with the built-in **csv module**. You can also work with other delimiters, like tabs or semicolons, by adjusting the settings.

When dealing with files, paths are important. An absolute path shows the full location on your computer, while a relative path shows the location from your program's current folder. Errors can happen if a file path is wrong or the file does not exist, so handling these situations gracefully is important.

- **CSV stands for Comma-Separated Values** — a common format for spreadsheets and data exports.
- Use the csv module in Python to read and write CSV files easily.
- File paths can be absolute (full address) or relative (from current folder).
- Always check if files exist or are accessible before working with them.
- Handle errors using try-except blocks to avoid crashes.

```
import csv
from pathlib import Path

file_path = Path('data/example.csv')

try:
    with file_path.open('r', newline='') as csvfile:
        reader = csv.reader(csvfile)
        for row in reader:
```

```

        print(row)
except FileNotFoundError:
    print('The file does not exist.')
except PermissionError:
    print('Permission denied while accessing the file.')
except Exception as e:
    print(f'An error occurred: {e}')
```

Path Type	Example
Absolute Path	/home/user/data/example.csv
Relative Path	data/example.csv
Windows Path	C:\Users\user\data\example.csv

Note: If your CSV data uses a different delimiter, like a semicolon, pass `delimiter=';` into `csv.reader` or `csv.writer`. Always validate file paths using the `Path` module from the `pathlib` library for better portability and safety.

Understand and use regular expressions for pattern matching.

Regular expressions give you a powerful way to search for, match, or extract text patterns in data. In Python, regular expressions let you describe what you want to find, rather than writing long, detailed code to handle different cases. They are often used for tasks such as finding phone numbers in a document, checking email formats, or filtering lines in a file.

A **regular expression (regex)** is a sequence of characters that defines a search pattern. You use special symbols to describe the kind of text to match, such as any digit, any letter, or a specific word. Learning regular expressions can save you time when you need to process or check large amounts of text.

Python's standard library includes the `re` module for working with regular expressions. You import this module whenever you want to use regex features. The `re` module provides functions like `search`, `match`, `findall`, and `sub`, helping you find and process patterns in strings.

- Import the `re` module to use regular expressions.
- Use raw strings (add an `r` before the quotes) to write regex patterns. This avoids confusing errors with backslashes.
- Common regex symbols: `.` (any character), `\d` (digit), `\w` (word character), `\s` (whitespace), `*` (zero or more), `+` (one or more), `?` (zero or one).
- `findall()` returns all matches in a list, `search()` finds the first match, and `sub()` replaces matched text.
- Regular expressions are case-sensitive by default.

```
import re

text = 'Anna has 3 cats and 2 dogs.'

# Find all numbers in the text
numbers = re.findall(r'\d+', text)
print(numbers)    # Output: ['3', '2']

# Check if a line has an email address
line = 'Contact us at help@example.com.'
pattern = r'[\w.-]+@[ \w.-]+\.\w+'
if re.search(pattern, line):
    print('Valid email found!')
```

Pattern	Description
\d	Any digit (0-9)
\w	Any word character (letters, digits, underscore)
.	Any character except newline
^pattern	Match at the start of a line
pattern\$	Match at the end of a line
[abc]	a, b, or c
a b	a or b

Note: Important tip: Always test your regular expressions with sample data. Small differences in your pattern—such as missing a backslash or the wrong symbol—can change the result. Use online testers or Python’s re module in interactive mode to experiment safely.

Practice filtering and extracting data using pattern matching techniques.

Pattern matching is a technique you use to search for specific sequences in data. In Python, pattern matching is commonly done using regular expressions. This skill is vital for parsing logs, extracting useful information, and filtering out irrelevant content. Understanding how to filter and extract data based on patterns helps automate data analysis, clean up messy files, and focus only on what matters.

When working with data, you often need to **find and extract only the information matching certain patterns**. For example, if you have a list of email addresses mixed with other text, you may want to

pull out just the email addresses. Regular expressions allow you to describe these patterns clearly and locate them in your data.

Filtering and extracting are two related but different tasks. Filtering means picking lines or pieces of data that match your pattern and ignoring the rest. Extracting means locating and lifting out the matching portion, which could be a word, number, or sequence inside a line. Regular expressions give you a toolkit to perform both operations efficiently.

- Filtering: Reading a log file and keeping only lines with error messages.
- Extracting: Pulling phone numbers or postcodes from a document.
- Combining filtering and extracting: Saving only the dates from lines that mention appointments.

```
import re

lines = [
    'Order ID: 91234 placed on 23/02/2024',
    'Invalid entry',
    'Order ID: 84578 placed on 15/01/2023',
    'Contact: john.smith@email.com'
]

# Filter lines containing 'Order ID:'
filtered = [line for line in lines if 'Order ID:' in line]
print('Filtered:', filtered)

# Extract order numbers and dates using regex
for line in filtered:
    match = re.search(r'Order ID: (\d+) placed on (\d{2}/\d{2}/\d{4})', line)
    if match:
        order_id, date = match.groups()
        print(f'Order ID: {order_id}, Date: {date}')
```

Technique	Purpose
Filter	Keep only data matching a pattern
Extract	Pull out pattern matches from data
Combine	First filter, then extract details

Note: Important tip: Always test your pattern on example data before using it on your real dataset. This helps you avoid missing important information or accidentally pulling out the wrong details.

Develop problem-solving skills focused on working with data in practical scenarios.

Working with data is a central task in many programming projects. You will often face real-world problems where you need to extract, filter, or transform information. Developing strong problem-solving skills helps you approach these scenarios with confidence and find clear solutions.

The key to effective problem solving is to start by carefully understanding the requirements. **Break down the task into smaller steps, such as loading the data, identifying patterns, filtering records, and collecting results.** By practising these steps, you'll build a structured approach to tackling data tasks.

Let's consider an example. Suppose you have a text file containing a list of client email addresses, but only want to find addresses from a specific domain (for example, ending in '@example.com'). To solve this, you would: open the file, read each line, check if it matches the domain, and collect the addresses you find. This approach is common in many data filtering tasks.

- Read and understand the requirements of the data task.
- Break the problem into simple, logical steps.
- Plan the data flow: input, processing, and expected output.
- Choose the right tools: basic string methods, data structures, or regular expressions.
- Test your solution with examples to ensure it works correctly.

```
filtered_emails = []
with open('clients.txt', 'r') as file:
    for line in file:
        email = line.strip()
        if email.endswith('@example.com'):
            filtered_emails.append(email)
print(filtered_emails)
```

Step	Description
1. Input	Open and read the file line by line
2. Process	Check if each line matches your filter
3. Output	Store matching lines for later use

Note: Important tip: Always test your approach on small samples before working with larger data files. This helps you catch mistakes early and build confidence.

8. User Input and Error Handling

ACCEPTING USER INPUT

Use the `input()` function to accept data from users during program execution.

A key part of writing interactive Python programs is accepting information from users. You do this using the built-in `input()` function. This allows your program to pause, prompt the user, and capture whatever they type. User input can make your Python scripts flexible and responsive to real-world situations.

The `input()` function displays a prompt, waits for the user to type something, then stores what they enter as a string. You can use the information from the user to control your program's behaviour. This adds interactivity and can make your programs much more useful.

The `input()` function is often used to gather data, such as a user's name or an option from a menu. Whenever you want your script to receive instructions or data from someone running it, `input()` is the right tool. After calling `input()`, you can store the result in a variable for later use. This powerful feature allows programs to adapt to users rather than working with fixed values.

- The `input()` function always returns a string, even if the user types numbers.
- You can **include a prompt in the `input()` function** to let users know what to enter.
- If you need a number, you should **convert the string to an integer or float**.
- Input is only captured **when the user presses Enter**.
- `input()` **pauses your program** until the user responds.

```
name = input("What is your name? ")
print("Hello, " + name + "!")

age_string = input("How old are you? ")
age = int(age_string)
print("Next year, you will be", age + 1)
```

User Enters	Result in Python
Alice	"Alice"
42	"42" (as a string)
3.14	"3.14"

Note: Important tip: Always remember that `input()` returns text, not numbers. If you want to use the input as a number (for calculations), use `int()` or `float()` to convert it. If the user enters something that can't be converted, your program will raise an error—so be careful and consider using error handling.

Always validate user input to ensure it meets the expected requirements before processing.

In any program that interacts with users, you will often need to collect input. This might be a number, a word, or a line of text. Before you use this input, it is important to check that it matches what your program expects. This process is called input validation.

User input is unpredictable. People may enter the wrong data by accident. Sometimes, input is missing, misspelt, or the wrong type. Validation helps you **protect your program from errors** and keep it running smoothly.

Validating input means checking whether it meets certain conditions before using it. For example, if your program expects a number, you should confirm the input is, in fact, a number. If you need an email address, you can check for the right format. Doing this helps avoid crashes, bugs, or unexpected behaviour in your program.

- Ask the user for information
- Check if the input follows the right format or type
- Show an error or ask again if the input is not valid
- Only use the input once it is valid

```
while True:
    user_age = input("Please enter your age: ")
    if user_age.isdigit():
        age = int(user_age)
        if 0 < age < 120:
            break
        else:
            print("Enter a realistic age between 1 and 119.")
    else:
        print("That doesn't look like a number. Please try again.")

print(f"Your age is {age}.")
```

Input	Should Accept?
25	Yes
zero	No
-5	No
130	No
42	Yes

Note: Important tip: Never assume users will always enter the right information. Validation keeps your program stable and secure. Always check input before using it in calculations or saving it.

Utilise the **try and except blocks** to handle potential errors that may occur during input and processing.

When your Python program asks a user for input, things can go wrong. You may want the user to enter a number, but they might type something else. If you try to use that input in your code, it can cause errors. Python offers tools to manage these situations using try and except blocks.

A **try block** is a section of code where you place actions that might cause an error. If Python finds a problem inside the try block, it immediately jumps to handling code in the **except block**. The except block explains what to do if an error happens. This makes your program less likely to crash and more helpful to users.

For example, suppose you ask someone to enter their age using `input()`. If they type letters instead of a number, Python will throw an error. Wrapping the code in try and except lets your program handle this problem smoothly.

- Use try and except blocks when user input might be incorrect.
- Handle errors gracefully so your program does not stop unexpectedly.
- Offer feedback to users so they know what went wrong and how to fix it.

```
try:
    age = int(input('Enter your age: '))
    print('You are', age, 'years old.')
except ValueError:
    print('Please enter a valid number for your age.')
```


What you type	What happens
22	You are 22 years old.
twenty	Please enter a valid number for your age.
	Please enter a valid number for your age.

Note: Important tip: Use specific error types in except blocks, like `ValueError`, to catch only the errors you expect. This makes your code clear and safe.

Provide helpful feedback or prompts to users when invalid input is detected.

When you design programs that interact with users, you often need to ask for input. However, users can sometimes enter data in the wrong format or give values your program cannot use. Handling these situations gracefully is essential. You should provide clear, helpful feedback so users know what went wrong and how to correct their input.

Effective feedback improves the user experience. **It builds trust by showing users that your program anticipates and manages mistakes.** Giving clear prompts also reduces frustration and helps users succeed on their next attempt.

Suppose your program asks for a number, but the user types in words instead. If the program only displays a confusing error or stops working, users will not know what went wrong. By providing specific feedback, such as 'Please enter a valid number', you guide the user to give the right input.

- Tell users what kind of input is expected (e.g., 'Enter your age as a whole number').
- Point out what was wrong with the received input (e.g., 'That was not a number').
- Offer users another chance to try again, along with a clear prompt.
- Keep feedback messages short and polite.
- Avoid technical jargon to make messages easy to understand.

```
while True:
    age_input = input('Please enter your age: ')
    if age_input.isdigit():
        age = int(age_input)
        break
    else:
        print('Invalid input. Please enter a whole number for your age.')
```

User Input	Feedback Displayed
twenty	Invalid input. Please enter a whole number for your age.
-5	Invalid input. Please enter a whole number for your age.
30	No error. Program continues.

Note: Important tip: Always tailor error messages to the situation. Avoid generic errors like 'Input error' and instead give a helpful, specific message – this reduces confusion and helps users respond correctly.

Implement error handling strategies to make your programs robust and prevent them from crashing due to unexpected input.

When you write Python programs that interact with users, you must expect the unexpected. Users might type numbers where text is needed, leave fields blank, or provide unexpected data. If your program does not account for these possibilities, it may crash or behave unpredictably. This is where error handling strategies become essential. They help you catch problems before they cause your program to fail, making your software more robust and professional.

Python uses the **try** and **except** keywords to manage errors. By placing risky code inside a try block, you can catch and manage errors using except. This lets your program continue running or display helpful messages instead of crashing. Proper error handling also prevents security risks and helps users understand what went wrong.

Suppose you are asking a user to enter their age as a whole number. If the user types something that cannot be converted into a number, Python will raise an error (ValueError). Without error handling, this stops your program. By planning for these problems, you can provide clear feedback and ask the user to try again instead of letting the program stop.

- Use try/except to catch errors such as ValueError or TypeError.
- Validate user input before processing it.
- Show helpful error messages rather than technical details.
- Loop until valid input is received if needed.
- Log errors for troubleshooting and improvement.

```
while True:
    age_input = input("Enter your age: ")
    try:
        age = int(age_input)
        if age <= 0:
            print("Please enter a positive age.")
```

```

        continue
    break
except ValueError:
    print("That is not a valid whole number. Try again.")
print(f"Your age is {age}.")

```

Strategy	Example
try/except block	Catch invalid input with except ValueError
Input validation	Check for empty or negative numbers
User feedback	Print user-friendly error messages

Note: Important tip: Never trust user input completely. Always check and handle errors, even if users are experienced. This keeps your programs stable and your data safe.

INPUT VALIDATION

Always validate user input to ensure it is of the expected type and format before processing.

Whenever your Python program accepts data from a user, you must check that the input is correct. This is called input validation. Input validation helps make sure the data your program uses matches the type and format you expect. If you skip this step, your code might crash, give wrong results, or even create security problems.

Let's break down why validation is important. **Python reads user input as text (a string) by default.** If you expect a number or a specific format, you need to check before you use that input in your code. For example, if someone enters letters when your program expects a number, you want to catch this mistake early.

Not all users will follow your prompts exactly. They might enter unexpected data by accident or on purpose. By validating input, you reduce bugs and help users correct their errors before your program goes any further.

- Validation prevents runtime errors (like trying to add letters and numbers together).
- Helps protect your program from malfunctions and crashes.
- Gives users helpful feedback when they make mistakes.
- Can stop harmful or dangerous data (such as code injection or invalid files).

```
user_age = input("Enter your age: ")
if user_age.isdigit():
    age = int(user_age)
    print(f"Your age is {age}")
else:
    print("Invalid input. Please enter a whole number.")
```

Input Example	Validation Result
25	Accepted as valid number
twenty	Rejected – invalid format
25.5	Rejected – not a whole number
	Rejected – empty input

Sometimes you must check more than just the type. For example, if you need a postcode, you should check the length and whether it only contains numbers. Here is another example with formats:

```
postcode = input("Enter your Australian postcode: ")
if postcode.isdigit() and len(postcode) == 4:
    print(f"Valid postcode: {postcode}")
else:
    print("Postcode should be exactly 4 digits.")
```

Note: Always guide users with clear error messages when validation fails. This helps them fix mistakes and keeps your program user-friendly.

Use built-in Python functions like `input()` to collect information from users.

In many Python programs, you will want to interact with the person running your code. One of the most common ways is to ask the user to type something while the program is running. Python has built-in functions that make this easy and reliable.

The most essential function for collecting user input is `input()`. This function pauses your program and waits for the user to type a response, then stores that response as a string.

When you use `input()`, you can provide a message (also known as a prompt) that will be displayed to the user. The person running your program can read this prompt, type their answer, and press Enter. The program will then continue, with their answer stored in a variable you choose.

- `input()` always returns a string. Even if the user types numbers, their response is stored as text.
- You need to use another function (like `int()` or `float()`) to convert the input to a number if required.
- You can handle invalid input with error checking. This is very important to prevent your program from crashing if someone types something unexpected.
- It is good practice to provide clear and meaningful prompts to help users understand what is expected.

```
name = input('What is your name? ')
print('Hello,', name)

age_input = input('How old are you? ')
try:
    age = int(age_input)
    print('Next year, you will be', age + 1)
except ValueError:
    print('That was not a valid number. Please enter a number for your age.')
```

Scenario	How <code>input()</code> Works
User types their full name	The full name (including spaces) is stored as a string
User enters a number	The number is returned as a string; must be converted with <code>int()</code> or <code>float()</code>
No input (just presses Enter)	An empty string ('') is returned

Note: Always assume the user could type anything. Validate or convert their input as needed to avoid errors in your program.

Implement error handling with `try` and `except` blocks to manage unexpected user inputs and exceptions.

When users interact with your Python program, there is always a chance they may enter data in an unexpected format or cause errors you did not anticipate. Python provides a structured way to handle such situations using `try` and `except` blocks. These tools help your program remain robust by managing errors gracefully instead of crashing.

A **try block** is used to write code that may cause an error. If an error, also called an exception, occurs in the try block, Python skips the rest of that block and looks for an **except block** below it to handle the problem.

Using try and except blocks lets you control what your program does when something goes wrong. For example, if you ask the user to enter a number but they type text instead, you can show a friendly message instead of letting the program stop with an error.

- Prevents unexpected program crashes
- Allows custom messages to guide users when they make mistakes
- Enables you to handle different types of errors in specific ways
- Improves user experience by making the program more forgiving
- Helps with debugging by catching and logging errors

```
try:
    age = int(input('Please enter your age: '))
    print(f'You are {age} years old.')
except ValueError:
    print('That was not a valid number. Please enter your age as a number.')
```

Error Type	When it Occurs
ValueError	User enters invalid data type (e.g., 'abc' instead of a number)
ZeroDivisionError	Trying to divide by zero (e.g., 10 / 0)
FileNotFoundError	Trying to open a file that does not exist
TypeError	Using the wrong type of value in an operation

Note: Important tip: You can have multiple except blocks to handle different types of errors. Always handle the most specific errors first. Only use a general except block (without specifying the error type) as a last resort, as it will catch all exceptions—including those you may want to notice during development.

It's important to validate user input and use error handling together. First, check if the data makes sense (for example, ensuring a number is positive). Combine this checking with try and except blocks to catch both mistakes in format and unexpected issues.

For more complex programs, you can write your own error messages and even continue asking the user for correct input until they provide it. This approach builds programs that feel more professional and are less likely to confuse or frustrate users.

Provide clear and helpful messages when invalid input is received to guide the user.

Input validation is an essential part of writing reliable Python programs. When users enter data, there is always a chance they may make mistakes or provide information in the wrong format. Handling these situations gracefully is important to create a good experience for users and ensure your programs behave as expected.

When you detect **invalid input**, your program should display clear and helpful messages. This means telling the user what went wrong and, ideally, what they need to do differently. Avoid vague messages like 'Error' or 'Invalid input' without further explanation.

Imagine you ask a user to enter their age as a whole number. If they type 'twenty-five' or leave the input blank, your program should not just fail or print an unhelpful message. Instead, you should guide them to enter information in the correct format, making your program easier and friendlier to use.

- State clearly what type of input is expected.
- Explain why the input was considered invalid.
- Suggest what the user should do next.
- Keep messages polite and encouraging.
- Do not include technical jargon.

```
while True:
    age_input = input("Please enter your age in years: ")
    if age_input.isdigit():
        age = int(age_input)
        print(f"Thank you. You entered: {age} years old.")
        break
    else:
        print("Invalid input. Please enter your age as a whole
number (for example, 25). Try again.")
```

User Input	Helpful Message
abc	Invalid input. Please enter your age as a whole number (for example, 25). Try again.
	Invalid input. Please enter your age as a whole number (for example, 25). Try again.
30.5	Invalid input. Please enter your age as a whole number (for example, 25). Try again.

Note: Important tip: The more precise and friendly your messages are, the less frustrated your users will feel. This leads to fewer mistakes, better feedback, and more successful use of your program.

Test your input validation and error handling with different edge cases to build resilient applications.

Testing your input validation and error handling is vital for building applications that work well in all situations. Users can enter unexpected values, and your program should respond gracefully. Edge cases refer to unusual or extreme types of input that might break your code if not managed properly.

When you design input validation, always consider the widest possible range of user behaviour. **Edge cases are inputs at the boundary or outside the typical range.** These might include negative numbers, empty strings, very large values, or even the wrong data type, such as letters where numbers are expected.

For beginner programmers, handling easy and obvious input is not enough. Your code should be able to manage exceptions and errors without stopping unexpectedly. Edge case testing finds risks early, so applications do not crash and cause poor user experiences.

- Try empty input, such as blank entries or just pressing Enter.
- Enter unexpected data types, like text instead of a number.
- Test boundary values, such as zero, negative numbers, or extremely large numbers.
- Include special characters—symbols or even entire sentences where single words are expected.
- Test maximum lengths if there are input size restrictions.

```
def get_positive_integer(prompt):
    while True:
        user_input = input(prompt)
        try:
            number = int(user_input)
            if number > 0:
                return number
        except ValueError:
            print("Please enter a positive integer.")
            print("That is not a valid whole number. Try again.")

# Test with: '', 'hello', '0', '-5', '999999999', '3'
age = get_positive_integer("Enter your age: ")
print(f"You entered: {age}")
```


Test Case	Expected Result
Empty input ('')	Prompt again, message about valid whole number
Letters ('hello')	Prompt again, message about valid whole number
Zero ('0')	Prompt again, message about positive integer
Negative number ('-5')	Prompt again, message about positive integer
Large number ('999999999')	Accept if within system limits
Valid input ('3')	Accept and return value

Note: Important tip: Testing with many edge cases early helps you catch problems before users do, reducing chances of unexpected application crashes and improving quality.

EXCEPTION HANDLING (TRY/EXCEPT)

Always capture user input using input() and provide clear instructions to the user.

When building interactive Python programs, you often need to gather information from users. This is usually done using the `input()` function. Clear instructions help users know exactly what is expected, reducing mistakes and frustration.

The `input()` function pauses program execution and waits for the user to type something. Whatever they type is returned as a string. You must always provide clear instructions in the prompt, so users know what to enter.

Suppose you want the user's age. Instead of a blank prompt, guide them clearly. This ensures the data entered is more likely to be valid and your program is easier to use.

- `input()` pauses the program until the user enters information.
- Always include a prompt explaining what is required.
- Assume the user has no prior knowledge of your program.
- Be specific about the format or type of input (e.g., number, text).
- Repeat or clarify instructions if input is invalid.

```
age = input('Please enter your age in years: ')
name = input('What is your first name? ')
print('Thank you,', name, '. You are', age, 'years old.')
```

Prompt Example	Why it's Clear
Enter your age in years:	Specifies number and unit.
Type your full email address:	Explains exactly what is needed.
How many litres of fuel did you use?	Indicates the type (number) and unit.

Note: Important tip: Never leave the input prompt blank. Users are much more likely to make mistakes or feel confused if there are no clear instructions.

Validate user input to ensure it matches the expected format or type before proceeding.

Whenever someone uses your Python program, they might be asked to enter information. This could be their name, a number, or a choice from a menu. People do not always enter data in the way your code expects. For example, they might type letters when a number is needed, or leave a field blank. If your program does not check and respond to this, it can cause errors, crashes, or incorrect behaviour.

To create robust programs, you should always **validate user input**. This means checking that what the user types matches what your program expects before moving on. If you need a number, check that the input really is a number. If you are expecting a certain format, such as an email address, you need to ensure it fits that format.

Python provides several ways to validate input. The simplest method is to use the built-in `input()` function to receive input, then test whether the input is the right type or follows the required rules. If it does not, you can ask the user to try again, or display an informative message.

- Use built-in functions like `int()` or `float()` to check for numeric values.
- Check string patterns using methods such as `.isdigit()` or regular expressions for specific formats.
- Provide clear feedback so users know what is expected and what went wrong.
- Repeat the request in a loop until valid input is received.

```
while True:
    age_input = input('Enter your age: ')
    if age_input.isdigit():
        age = int(age_input)
        break
    else:
        print('Please enter a valid number for your age.')
```

Expected Type	Validation Method
Integer	Use <code>int()</code> , <code>.isdigit()</code> or exception handling
Floating-point number	Use <code>float()</code> , <code>try/except</code>
Email address	Use regular expressions (<code>re</code> module)
Yes/No answer	Test for 'yes', 'no', 'y', 'n'

Note: Important tip: Always assume the user might make mistakes. Validating input protects your program from crashes and confusing errors. It also leads to a better experience for your users.

Use try/except blocks to handle exceptions and prevent program crashes.

In Python programming, exceptions are errors that occur during program execution. These errors can cause your program to crash if you do not manage them. By using try/except blocks, you can catch and respond to these errors, keeping your program running and providing helpful feedback to users.

When you use a **try/except** block, you tell Python to 'try' running some code. If an error occurs during this attempt, Python looks for an 'except' block and executes that code instead. This lets you prevent unhandled errors from crashing your application.

Every program can encounter unexpected situations—a user entering text instead of a number, or the program trying to open a file that doesn't exist. Handling these situations gracefully improves user experience and makes your programs more reliable.

- try blocks contain code that might cause an error.
- except blocks catch and handle specific or general exceptions.
- Using try/except ensures your program does not crash unexpectedly.
- You can display friendly messages to users when errors happen.

```
try:
    number = int(input('Enter a number: '))
    print('You entered:', number)
except ValueError:
    print('That was not a valid number! Please try again.')
```

Error Example	How try/except Helps
User enters letters where a number is expected	Catches ValueError and displays a user-friendly message
File not found when opening a file	Catches FileNotFoundError instead of crashing
Division by zero	Catches ZeroDivisionError and lets you handle it gracefully

Note: Important tip: Always use specific exception types in except blocks when possible. Catching general exceptions can hide unexpected errors and make debugging harder.

Display helpful error messages when an exception occurs to guide the user.

When a program encounters an error during execution, it is known as an exception. Instead of letting the program crash or show confusing messages, you should display clear and helpful error messages for the user. This helps users understand what went wrong and how they might fix it.

Python uses the **try/except** structure to catch and manage exceptions. If something goes wrong inside the try block, Python will run the code in the except block. Here, you can display a friendly and informative error message. This makes the program easier to use and prevents confusion.

For example, if your program asks a user to enter a number but they type letters instead, a `ValueError` occurs. Instead of showing a raw error or crashing, you can catch this and display a message that gently explains what went wrong and what the user should do next.

- Clearly state what went wrong in the user's language.
- Avoid technical jargon; keep the message simple.
- Suggest how to fix the problem or what input is expected.
- Never display confusing internal error details to users.
- Use exception handling to prevent crashes and keep the program running.

```
try:
    age = int(input('Please enter your age: '))
except ValueError:
    print('Invalid input: please enter a number, such as 25.')
```

Bad Error Message	Helpful Error Message
<code>ValueError: invalid literal for int() with base 10: 'abc'</code>	Please enter a valid number for your age.
<code>IndexError: list index out of range</code>	That option does not exist. Please choose a valid menu number.
<code>FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'</code>	Could not find the file 'data.txt'. Please check the filename and try again.

Note: Important tip: Always check where users might give unexpected input, and use try/except to catch these situations. Give error messages that teach and guide, not messages that blame or confuse.

Build resilient applications by anticipating edge cases and handling unexpected input or errors.

Building resilient applications means making sure your programs can handle unexpected situations without crashing. In real-world scenarios, users might make mistakes, files might be missing, or data might not look the way you expect. By anticipating edge cases and handling errors properly, you help your programs recover gracefully, provide helpful feedback, and maintain a good user experience.

Resilient Python applications are able to deal with **unexpected input, mistakes, or unusual scenarios**. For example, what if someone enters text where a number is needed? Or if your program tries to open a file that is missing? Instead of failing immediately, your code can spot these outliers and handle them with care.

To anticipate edge cases, first think about what might go wrong. Then, use exception handling (try/except), input validation, and sensible defaults to prepare for those possibilities. This requires you to ask, 'What if?' at every step of your program. Planning for mistakes will help you write more reliable code.

- Check for valid data types (e.g., numbers instead of letters)
- Test for empty input or missing files
- Handle division by zero or invalid calculations
- Look for values outside expected ranges
- Catch operating system errors, like permission denied

```
while True:
    try:
        number = int(input('Enter an integer: '))
        result = 10 / number
        print(f'Result: {result}')
        break # Exit loop if successful
    except ValueError:
        print('Please enter a valid integer.')
    except ZeroDivisionError:
        print('Cannot divide by zero. Try again.')
    except Exception as e:
        print(f'Unexpected error: {e}')
```

Edge Case	How to Handle
User enters text instead of a number	Catch ValueError with try/except
Calculation tries to divide by zero	Catch ZeroDivisionError
Attempt to open missing file	Catch FileNotFoundError
Data outside allowed range	Check before use, provide feedback

Note: Important tip: Always test your programs with a variety of valid and invalid inputs. This helps you spot weaknesses and improve your code before users encounter issues.

WRITING ROBUST PROGRAMS

Always validate user input to ensure your program receives data in the expected format.

When you ask a user for input, you cannot always trust that they will enter exactly what your program expects. Whether people make mistakes or type unexpected values on purpose, it is your job as a programmer to make sure every input is checked. This is known as validating user input.

Failing to validate user input **can cause your program to crash**. Even worse, it can lead to incorrect results, security risks, or data loss. Validation is not just something for big businesses or advanced applications—it should be part of every program you write from the beginning.

Validation means your code checks information before working with it. For example, if you ask a user for their age, you want to make sure they actually enter a number, and that number makes sense for an age. If you accept anything the user types, your program might break or behave strangely.

- Ask the user for input
- Check that the input matches the format you need (such as a number or an email address)
- Give helpful feedback if the input is not valid (for example, ask them to try again)
- Only use the input if it passes your checks

```
while True:
    age_input = input('Please enter your age: ')
    if age_input.isdigit():
        age = int(age_input)
        if 0 < age < 120:
            print('Thank you!')
            break
        else:
            print('Please enter a sensible age between 1 and
119.')
    else:
        print('That is not a number. Please enter your age as a
number.')
```

Input Example	Why It Fails Validation
twenty	Not a number
-5	Age cannot be negative
140	Unrealistic age

Note: Important tip: Always assume the user might make a mistake. Be polite and clear in your error messages. Good validation makes your programs safer and easier to use.

Use try and except blocks to catch and manage errors, preventing your program from crashing unexpectedly.

When writing programs, errors can happen for many reasons—such as invalid user input, missing files, or network problems. By default, if your Python program encounters an error, it will stop and display an error message (called an exception). This immediate stopping is called 'crashing.' To write robust programs, you want to handle these situations smoothly so your program can keep running or fail gracefully.

You can defend against unexpected crashes by using **try and except blocks**. These let you 'catch' errors and decide how your program should respond, instead of the program crashing.

A try block is a section of code where you think something might go wrong. If an error happens inside the try block, Python looks for a matching except block to handle the error. If it finds one, the code in the except block is run instead of stopping the program.

- A try block lets you test code that might cause errors.
- An except block lets you respond to those errors.
- You can use multiple except blocks to handle different types of errors.
- Catching errors prevents your program from crashing unexpectedly.
- You can provide user-friendly messages and default actions when errors occur.

```
try:
    number = int(input('Enter a whole number: '))
    print('You entered:', number)
except ValueError:
    print('That was not a valid whole number. Please try again.')
```


Code Area	Purpose
try	Protects the code that might cause an error
except	Handles a specific error if it occurs
input()	Gets user input (might not be a number)
int()	Tries to convert input to a whole number

In the example above, the user is asked to enter a number. If they type something that is not a number (like 'cat'), Python raises a `ValueError`. Instead of your program crashing, the `except` block handles the problem and shows a helpful message.

You can also catch any kind of exception by writing **except Exception as e:** which handles all standard errors. However, it is better to catch specific errors (like `ValueError`) when possible.

```
try:
    file = open('data.txt', 'r')
    contents = file.read()
    file.close()
except FileNotFoundError:
    print('File not found. Please check the filename and try again.')
```

Note: Important tip: Always use try and except blocks around code that interacts with unreliable data or external systems (such as files or user input). This makes your Python programs much more reliable and user-friendly.

Inform users of invalid input or errors with clear and user-friendly messages.

Robust Python programs do more than just run correctly—they provide helpful feedback to users when something goes wrong. Clear and friendly messages guide users, help them fix problems, and improve their overall experience with your software.

When your program receives invalid input or encounters an error, you should always **explain the problem in plain language**. Avoid technical terms where possible. If the error needs to be fixed by the user, make sure your message tells them what they need to do next.

Consider a simple scenario: your program asks for a number, but the user types a word instead. If your code just crashes or prints a confusing error like `'ValueError'`, the user won't know what went wrong. Instead, a well-written message like `'Please enter a number, not text.'` makes it clear what's needed.

- State what went wrong in easy-to-understand language.
- Tell the user exactly what kind of input you expect.
- Avoid blaming or shaming the user.
- If possible, show an example of acceptable input.
- Don't display technical errors unless the user is a developer.

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Invalid input. Please enter your age as a whole number (e.g., 25).")
```

Bad Message	Improved Message
Error: ValueError	Please enter a valid number.
Input failed	Your password must be at least 8 characters long.
Invalid entry	Date not recognised. Please use the format DD/MM/YYYY.

Note: Important tip: Friendly error messages make your program feel more professional and help users avoid frustration. Test your program by giving it wrong inputs and see how helpful your error messages are.

Test your code with different types of input, including invalid or unexpected values, to confirm it handles errors gracefully.

Testing your code with various inputs is an essential part of creating robust and reliable programs. It helps you ensure that your program behaves as expected, even when the user enters something you did not anticipate. Making your code resilient against mistakes and unusual input types results in better user experience, fewer crashes, and easier maintenance.

When writing Python programs, it is very common for users to enter data in unexpected ways. **Your aim is to make the program react calmly and sensibly** when anything goes wrong, such as typing letters when numbers are needed, or leaving a required field blank.

For example, suppose your program asks a user to enter their age. A user might type a whole number, write something like 'twenty', leave the input empty, or even type special characters. Your program should be able to handle all these situations without crashing.

- Test with normal, valid input (e.g., numbers when asked for age).
- Test with invalid input (e.g., text, symbols, or nothing at all).

- Test with values that are technically valid but unusual (e.g., negative numbers or extremely large numbers).
- Test with unexpected data types (e.g., floating point numbers, or even files where text is expected).

```
while True:
    age_input = input('Please enter your age: ')
    try:
        age = int(age_input)
        if age < 0:
            print('Age cannot be negative. Try again.')
            continue
        print(f'Your age is {age}.')
        break
    except ValueError:
        print('That is not a valid number. Please enter a whole number.')
```

Test Input	Program Response
25	Your age is 25.
-3	Age cannot be negative. Try again.
twenty	That is not a valid number. Please enter a whole number.
	That is not a valid number. Please enter a whole number.

Note: Important tip: Never assume the user will enter data exactly as you expect. By testing all possibilities and handling errors gracefully, you build confidence in both your code and your own programming skills.

Strive to make your applications resilient by anticipating and handling possible user mistakes or system errors.

Resilience means your program can withstand common problems and continue working, even when things go wrong. By thinking ahead and planning for mistakes or unexpected situations, your software can work reliably and avoid crashing. This approach is essential when creating programs for real users, who may make typos, enter unexpected data, or encounter technical issues like missing files.

As a programmer, you should ask yourself: **What could go wrong here?** Consider how users might interact with your program and where errors could happen. For example, what if a user types letters

instead of numbers? What if a file can't be found? By expecting the unexpected, you can prepare smart solutions.

Python provides tools to help your applications handle mistakes gracefully. You can use if-statements to check conditions before performing actions, and try/except blocks to catch and deal with errors when they happen. A resilient program communicates problems clearly and recovers without confusing the user or crashing.

- Validate user input to make sure it is the correct type or format.
- Check if files or network resources exist before trying to use them.
- Use try/except blocks to capture and respond to errors.
- Provide helpful messages to guide users when mistakes happen.
- Test your program for unusual or incorrect input.

```
while True:
    user_input = input("Enter a number: ")
    try:
        number = int(user_input)
        print(f"You entered: {number}")
        break
    except ValueError:
        print("That was not a valid number. Please try again.")
```

Scenario	Resilient Solution
User enters text instead of a number	Ask again until a number is entered
File is missing	Show an error message, prompt for another file, or create a new file
Internet connection fails	Warn the user, retry, or work offline

Note: Important tip: Always test your program with real data and think like a user who might make mistakes. The more errors you can foresee and handle, the more robust your application will be.

9. Using Python Libraries

IMPORTING MODULES AND PACKAGES

Python allows you to extend its functionality by importing modules and packages.

One of Python's most powerful features is the ability to extend what your programs can do by importing modules and packages. Modules are files containing extra Python code. Packages are collections of modules grouped together. When you import them, you get access to new functions, classes, and tools that will save you time and effort.

The base Python language includes some useful built-in functions, but to handle more advanced tasks you often **import external modules or packages** such as for maths, working with files, making web requests, or processing dates and times.

Think of modules and packages as toolkits. Instead of writing every bit of code yourself, you can use what others have already created. Python comes with a large number of modules in its standard library. You can also install extra packages written by other people.

- A module is a file containing Python code, such as functions or classes.
- A package is a folder that contains multiple module files and an extra file called `__init__.py`.
- Importing gives you access to external code in your own program.
- Python comes with a broad standard library.
- You can install third-party packages using pip—the Python package installer.

```
import math
print(math.sqrt(16))

from datetime import date
print(date.today())

import requests # You may need to install this package first
response = requests.get('https://www.python.org')
print(response.status_code)
```

Type	Example
Module	math
Standard Library Package	datetime
Third-party Package	requests

Note: Important tip: Only import what you need. Unnecessary imports can slow down your program and make your code harder to read.

Use the 'import' statement to access standard and third-party modules.

Python has a powerful feature called modules. Modules are collections of code—functions, variables, and classes—that you can use in your own programs. This lets you avoid re-inventing the wheel and helps keep your code simpler and more organised.

To use a module in Python, you need the **import** statement. This statement tells Python to bring in extra code or functionality from a standard or third-party module so you can use it in your script.

Standard modules come with Python itself. For example, the 'math' module provides mathematical functions. Third-party modules are created by others and shared, often by installing them using tools like pip. These let you do things like making web requests, reading Excel files, or working with data.

- Use the import statement to bring modules into your script.
- Standard modules are already included with Python (for example, math, datetime, os).
- Third-party modules need to be installed before you can import them.
- After import, you can use the module's functions and classes in your code.

```
import math
print(math.sqrt(25))  # Uses the sqrt function from the math
module

import requests  # Third-party module for web requests
response = requests.get('https://www.python.org')
print(response.status_code)
```

Import Style	What It Does
import math	Imports the whole module named math
import os	Imports the whole module named os
import requests	Imports a third-party module (needs to be installed first)

Note: If you try to import a third-party module that hasn't been installed, Python will show an error. Use 'pip install modulename' to add third-party modules before using import.

Modules are individual Python files that provide reusable code, while packages are collections of modules grouped in a directory.

In Python, using modules and packages helps you organise your code and keep it tidy. You can also make your code reusable, so you do not repeat yourself. Knowing the difference between modules and packages is a key skill for beginner programmers.

A **module** is simply a single Python file that contains code—functions, variables, and classes. You can create your own modules or use ones made by others. By keeping your code in modules, you make it reusable across multiple programs.

For example, you might write a module called `math_tools.py` that has useful mathematical functions. Once you have this file, you can use these functions in any Python program, just by importing the module.

- Modules help you break up large programs into smaller, logical pieces.
- You can import modules to use their code without copying and pasting.
- Python's standard library is made up of many useful modules.

```
# my_module.py
def greet(name):
    print(f"Hello, {name}!")
```

A **package** is a way to group related modules together in a folder (directory). This folder will contain multiple `.py` files (modules) and must include a special file called `__init__.py`. The `__init__.py` file tells Python that this folder is a package.

Packages are useful for keeping code for larger projects organised. For example, you might have a package called `utilities` which contains several modules: `file_tools.py`, `math_tools.py`, and `string_tools.py`. Each module handles a different type of task, but they are all related.

```
utilities/
  __init__.py
  file_tools.py
  math_tools.py
  string_tools.py
```

Term	Description
Module	A single Python (<code>.py</code>) file with code you can reuse by importing it.
Package	A directory with multiple modules and an <code>__init__.py</code> file for organisation and reuse.

When you want to use code from a module or a package, you use Python's `import` statement. This statement tells Python where to look for the code you need.

```
# Importing a module
import math_tools

# Importing from a package
from utilities import math_tools
```

You will find that many Python libraries and third-party tools are delivered as packages. This lets you use a whole collection of tools by installing a single package, making it simpler to extend your programs.

Note: Important tip: Make sure each package directory has an `__init__.py` file, even if it is empty. This file tells Python to treat the directory as a package.

You can import specific functions, classes, or entire modules as needed for your project.

Python lets you organise and reuse code by using modules, packages, functions, and classes. This makes your work easier and helps avoid duplicating code. You can decide what to import depending on what your program needs. Importing only what you need keeps your code tidy and efficient.

When using Python, you can **import an entire module**, bring in just a **specific function**, or select a **class**. This flexibility means you only load what you require.

A module is a file that contains Python code, such as functions or classes. A package is a collection of modules grouped together in a directory. This helps you organise larger projects. Python comes with a set of built-in modules, and you can create your own or use those made by others.

- Import an entire module if you need many functions or classes from it.
- Import specific functions or classes to keep your code neat and avoid naming conflicts.
- Use 'as' to give an imported item a different name (an alias) for convenience

```
# Importing an entire module
import math

# Importing a specific function
from math import sqrt

# Importing multiple functions
from math import sin, cos

# Importing and giving an alias
import datetime as dt

# Importing a specific class from a module
from datetime import date
```


Import Style	Description
<code>import module</code>	Imports the complete module; access items using 'module.name'
<code>from module import item</code>	Imports only the item (function/class); can use it directly
<code>import module as alias</code>	Imports module with a new name; useful for long module names

Note: Important tip: Importing only what you need makes your program clearer and can improve performance, especially as your projects become larger.

Using modules and packages helps organise your code and enables access to a wide range of pre-built Python tools.

When writing Python programs, your projects can grow quickly and become complicated. Organising your code into tidy sections makes it much easier to manage over time. In Python, you do this by using modules and packages. These features help you split your program into logical, manageable pieces.

A **module** is simply a file that contains Python code—often functions, variables, or classes that perform specific tasks. By breaking code up into modules, you make it reusable and easier to maintain. A **package** is a directory or folder that contains multiple modules, helping to further organise related modules together.

Python's rich ecosystem includes thousands of pre-built modules and packages, both in its standard library and from the wider open-source community. This means you can add complex functionality, like working with files, the internet, or even data analysis—without writing everything from scratch.

- Modules keep code organised and reusable.
- Packages group related modules for easier management.
- You can use third-party packages to add new features quickly.
- Python's standard library includes many helpful modules.
- Using packages saves development time and reduces bugs.

```
import math
print(math.sqrt(25))

from datetime import date
today = date.today()
print('Today is', today)
```

Concept	Description
Module	A single Python file containing code you can import and use.
Package	A folder containing multiple modules and an <code>__init__.py</code> file.
Standard Library	A collection of useful modules included with Python.
Third-Party Package	Modules developed by others, usually installed via pip.

Note: Important tip: Always check if there is an existing module or package before writing new code—chances are someone has already solved your problem!

STANDARD LIBRARY OVERVIEW

Learn to import and use modules and packages to extend Python programs.

One of Python's greatest strengths is its rich ecosystem of modules and packages. These allow you to add new features to your programs without having to write everything from scratch. By importing modules and packages, you can use code written by others to handle tasks like mathematics, dates, web access, and more.

In Python, a **module** is a single file with Python code. A **package** is a directory containing one or more modules, and possibly other packages. Both help keep your code organised and reusable.

To use a module in your program, you use the import statement. For example, importing the 'math' module gives you access to mathematical functions like square roots or trigonometry. Importing the 'datetime' module lets you work easily with dates and times. Packages work the same way—just import what you need.

- Modules are individual files containing Python code.
- Packages are folders that contain collections of modules and special files.
- The import statement is used to bring modules and packages into your program.

```
import math
print(math.sqrt(16))  # Output: 4.0

from datetime import date
today = date.today()
print("Today's date is", today)

import os
print(os.name)  # See the name of your operating system
```

Import method	Example
Import the whole module	import random
Import a specific item	from random import randint
Import with an alias	import numpy as np

Note: If you try to import a module that is not installed, Python will display an error. For extra modules not included in the standard library, use **pip** to install them, for example: **pip install requests**.

Gain an understanding of what the Python standard library offers.

The Python standard library is a collection of modules and packages included with every Python installation. It contains ready-made code for a wide range of tasks, letting you write useful programs more efficiently. Instead of building everything from scratch, you can use these resources to speed up development, boost reliability, and encourage best practice.

The standard library is sometimes described as “**batteries included**” because it provides solutions for many common programming problems straight out of the box.

Modules in the standard library cover a variety of categories. You’ll find tools for mathematics, text processing, file input/output, internet protocols, data storage, compression, working with dates and times, and much more. This rich set of tools means that you can write powerful programs without needing to search for many external packages.

- No installation is required—all modules are available as soon as Python is set up.
- The code is well-tested, secure, and updated regularly with each Python version.
- You can easily access documentation for each module, making it simpler to learn and apply new functionality.

```
import math
print(math.sqrt(16))

import datetime
today = datetime.date.today()
print("Today's date:", today)

import os
files = os.listdir('.')
print("Files in current directory:", files)
```

Module	Typical Uses
math	Mathematics, calculations, scientific work
os	Interacting with the operating system, file management
datetime	Dealing with dates and times
json	Reading and writing JSON data
random	Generating random numbers

Note: Important tip: You do not need to memorise every module. Instead, get used to searching the Python documentation to find what you need. Familiarity with what's possible is often more useful than remembering specific details.

Use standard library modules for web requests, handling dates, and processing JSON data.

The Python standard library is a powerful collection of modules included with every Python installation. You do not need to install anything extra to access these modules. They provide tools for many common programming tasks. Among these, modules for web requests, date handling, and working with JSON data are some of the most useful for beginners.

When you need to obtain information from the internet, such as downloading a file or reading the content of a web page, you use the **urllib.request** module. Dates and times in programs are managed with the **datetime** module, which helps you create, compare, and format dates and times. For reading, writing, and processing data in JSON format — the most popular data exchange format on the web — the **json** module is your main tool.

Let's look at each of these modules in action. Suppose you want to download some data from a web service, record when you accessed it, and then extract information formatted in JSON. All of this can be done using just the standard library.

- Use **urllib.request** to fetch data from URLs (such as websites or APIs)
- Use **datetime** to create date and time objects, or format dates for output
- Use **json** to convert between Python data structures and JSON strings

```
import urllib.request
import datetime
import json

# Step 1: Fetch data from a web URL
url = 'https://jsonplaceholder.typicode.com/todos/1'
response = urllib.request.urlopen(url)
```

```
data = response.read().decode('utf-8')

# Step 2: Process JSON data
item = json.loads(data)

# Step 3: Get the current date and time
now = datetime.datetime.now()

print('Fetched at:', now.strftime('%Y-%m-%d %H:%M'))
print('ID:', item['id'], '| Title:', item['title'])
```

Module	Typical Uses
urllib.request	Send requests to websites or APIs, download text or files
datetime	Create and format dates, measure time intervals, schedule actions
json	Read JSON files, convert between JSON strings and Python objects

Note: Always check documentation for each module to see the range of features available. Some tasks, like complex web authentication or handling time zones, may require additional steps or modules.

Practice applying libraries to solve real-world programming tasks.

Python libraries add a great deal of power to your programs. You can use them to solve common tasks without writing everything from scratch. Some libraries come with Python, while others need to be installed. Practising library use helps you learn what's available and how to apply them when building solutions.

Let's consider some practical **real-world tasks** where Python libraries make life easier: working with web data, managing dates, and handling files. In each case, using the right library saves time and helps avoid common mistakes.

For example, suppose you want to fetch data from a website. Instead of writing code to handle internet protocols yourself, you can use the requests library. If you need to work with dates—perhaps scheduling reminders or checking deadlines—the datetime library comes in handy. Libraries like json help you work with data formats used for communications between programs.

- Use requests to get content from web pages or APIs.
- Handle different time zones or date calculations with the datetime library.
- Read information from JSON files using the json module.
- Perform maths or statistics using the math or statistics libraries.
- Copy, move, or delete files with shutil or os modules.

```
import requests
import json

# Fetch weather data from an online API
response =
requests.get('https://api.weatherapi.com/v1/current.json?key=YOUR_
API_KEY&q=Sydney')
data = response.json()
print('Temperature in Sydney:', data['current']['temp_c'], '°C')
```

Task	Useful Python Library
Download a web page	requests
Parse dates and times	datetime
Read and write CSV files	csv
Work with file paths	os, pathlib
Handle JSON data	json

Note: Important tip: If you're not sure which library to use for a task, try searching the Python documentation or community forums. Practising with libraries will help you solve problems faster and write more reliable code.

Become familiar with finding and integrating external Python modules.

Python's functionality can be greatly extended by using external modules. These modules are packages of code developed by others that you can add to your own projects. This helps you avoid reinventing the wheel when trying to solve common problems.

Modules can help you with a huge range of tasks, such as working with spreadsheets, sending emails, requesting web data, and more. Python has a standard library (covered earlier), but **external modules** are not included by default and need to be installed from outside sources.

Most external Python modules are shared via a central repository called the Python Package Index (PyPI). PyPI is the official place to find third-party packages written for Python. You use a tool called `pip` to search for, install, and manage these modules on your computer.

- External modules are developed and shared by the Python community.
- PyPI hosts thousands of modules for different needs.
- `pip` is the standard tool for managing external Python modules.
- Modules can be installed once and used by many of your Python scripts.
- Regularly update your modules to keep your projects secure and up to date.

```
pip install requests
```

The example above uses pip to install the popular 'requests' module, which makes it easy to make web requests in your Python code. Once installed, you can import and use it in your projects without needing to copy code manually.

Step	Description
1. Search	Use pip or the PyPI website to look for a module you need.
2. Install	Run 'pip install module-name' in your command prompt or terminal.
3. Import	In your Python code, use 'import module_name' to start using it.

You can combine multiple modules in a single project. Carefully check their documentation and make sure they're suitable for your needs. Some modules are very large or have dependencies, so always read their installation instructions.

Note: Important tip: When working on new projects, consider using a virtual environment. This keeps each project's modules separate and avoids version conflicts.

WORKING WITH REQUESTS (WEB CALLS)

Learn how to use the 'requests' library to make web calls from Python programs.

The 'requests' library is a popular way to make web calls from your Python programs. It lets you communicate with websites or APIs over the internet. By using this library, your Python scripts can download web pages, send data to a website, or get information from external services. This is useful when you want to access data that is not stored on your own computer.

Web calls **allow communication** between your Python program and resources on the internet. The 'requests' library handles much of the hard work for you. It manages connections, sends requests, and collects responses so you do not have to deal with complicated details.

Before you can use the 'requests' library, you need to install it. This is usually done with the pip tool. Once installed, you can import it into your program and start making requests. The most common type is a GET request, which asks a server for information.

- Install the library with: `pip install requests`
- Import the library using: `import requests`
- Use `requests.get()` to get data from a website
- Use `requests.post()` to send data to a website
- Check the response status and read the contents

```
import requests

response = requests.get('https://www.example.com')

if response.status_code == 200:
    print('Success!')
    print(response.text)
else:
    print('Error:', response.status_code)
```

Method	Description
<code>requests.get(url)</code>	Fetches information from a website (GET request)
<code>requests.post(url, data=...)</code>	Sends data to a website (POST request)
<code>response.status_code</code>	Shows the HTTP status from the server
<code>response.text</code>	Contains the website response as plain text
<code>response.json()</code>	Converts JSON replies into Python objects

When you make a request, you often want to check if it worked. The `status_code` value tells you if the call was successful (200 is OK). If the website responds with data in JSON format, you can use `response.json()` to convert it into a Python dictionary. This makes it simple to work with data from web APIs.

Note: Important tip: Never share passwords, personal data, or secret keys directly in your code. Use environment variables or configuration files for sensitive information.

Understand how to send GET and POST requests and handle responses.

Web applications exchange information over the internet using protocols. The most common one is HTTP, which is used by browsers and many software tools. In Python, you can use the `requests` library to easily send information to websites and receive responses. There are two basic ways to ask for information from the web: GET and POST.

A **GET request** is used when you want to retrieve information from a server. For example, visiting a web page in your browser is a GET request behind the scenes.

A **POST request** is used when you want to send information to a server, like submitting a form or uploading data. In programming, POST is usually used when you need to pass data along with your request.

- GET requests are for retrieving or reading data.
- POST requests are for sending or uploading data.
- Both can send and receive data, but POST is preferred for sensitive or large amounts of information.

```
import requests

# Send a GET request
g_response = requests.get('https://api.github.com')
print(g_response.status_code)
print(g_response.text)

# Send a POST request
data = {'name': 'Alice', 'role': 'developer'}
p_response = requests.post('https://httpbin.org/post', data=data)
print(p_response.status_code)
print(p_response.json())
```

Request Type	Typical Use Case
GET	Read data, like viewing a web page
POST	Send data, such as filling out a form
PUT	Update existing data (less common for beginners)

Note: Always check the documentation of the web service or API you are using. Some services need special headers or ways to send data. Responses often return data in JSON format, which you can process in Python using `.json()`. If you are not allowed to access a resource, you may get a status code like 401 (unauthorised) or 403 (forbidden). Use `status_code` to check for success (200 means OK).

Practice parsing response data, such as JSON, from web APIs.

When working with web APIs, the data you receive is often in a special format called JSON. JSON stands for JavaScript Object Notation. It is simple, easy to understand, and widely used for transferring data. As a Python programmer, knowing how to read (parse) this data is essential for many real-world tasks.

After making a web request using a library like **requests**, the response often contains useful information. You need to extract and use these details in your program.

The requests library in Python provides a very handy way to access JSON data. When you receive a response object from an API, you can use the response's `json()` method to automatically parse the JSON content into Python data structures.

- JSON from APIs can include lists, dictionaries, and nested structures.
- The `json()` method converts JSON data into Python dictionaries or lists.
- You can access values in the parsed data using standard Python syntax.
- Always check that the API response was successful before parsing.
- If the data isn't in JSON format, using `json()` will cause an error.

```
import requests

# Send a GET request to a sample API
response =
requests.get('https://jsonplaceholder.typicode.com/todos/1')

# Check if the request was successful
if response.status_code == 200:
    # Parse the JSON data
    data = response.json()
```

```
# Access and print values
print('User ID:', data['userId'])
print('Title:', data['title'])
print('Completed:', data['completed'])
else:
    print('Request failed with status:', response.status_code)
```

Response Data	Python Access Example
Simple value	data['userId']
Nested value	data['address']['city']
List	data['items'][0]

Note: Important tip: Not all API responses are guaranteed to be JSON. If you try to parse non-JSON data with `json()`, Python will raise an error. Check the Content-Type header or read the API documentation to confirm the response format.

Explore handling errors and exceptions that may occur during web requests.

When writing Python programs that call web services or APIs, things may not always work as expected. Connections can fail, websites can be offline, or the data you get back might not be what you expect. Handling errors and exceptions in your code helps your program manage these situations smoothly rather than crashing.

In Python, most web requests are made using libraries like 'requests', which make it simple to fetch data from a website or API. However, calling remote services means dealing with things out of your control. By being ready for errors, your programs can show useful messages or retry if needed.

The requests library throws exceptions when something goes wrong—like network problems, timeouts, or invalid responses. You use Python's built-in try-except blocks to catch and handle these exceptions. This prevents your program from stopping suddenly.

- Common errors include connection errors, timeouts, or getting a response with the wrong status code.
- You can catch specific exceptions from requests, like `requests.ConnectionError` or `requests.Timeout`.
- It's good practise to handle errors gracefully—show the user a readable message or try the request again.

```
import requests

try:
    response = requests.get('https://api.example.com/data',
                           timeout=5)
    response.raise_for_status()  # Check for HTTP errors
    data = response.json()      # Try to decode JSON
except requests.ConnectionError:
    print('Could not connect to the server.')
except requests.Timeout:
    print('The request timed out.')
except requests.HTTPError as err:
    print(f'HTTP error occurred: {err}')
except ValueError:
    print('Could not decode the response as JSON.')
except Exception as e:
    print(f'An unexpected error occurred: {e}')
else:
    print('Data received:', data)
```

Exception	When it Occurs
requests.ConnectionError	Network problem – can't reach the site
requests.Timeout	Server took too long to respond
requests.HTTPError	Unexpected HTTP status (e.g. 404, 500)
ValueError	Data could not be decoded (e.g. bad JSON)

Note: Important tip: Always check the status code of a web response before trying to use its content. This can prevent your code from working with incomplete or unexpected data.

Apply web requests in real-world scenarios, such as retrieving or posting data online.

In modern programming, many applications need to connect to online services or share information across the web. Python makes this possible with easy-to-use libraries like requests. These libraries let you retrieve information from websites, submit data to servers, and interact with online APIs. Understanding how to apply web requests opens up new possibilities, such as collecting the latest news, automating tasks, or uploading data to cloud services.

When you **retrieve data**, you are sending a request to a website or server, asking for information. This is often called a GET request. On the other hand, when you want to **send or submit data** (like filling in a web form), you use a POST request. Both request types are common in real-world projects.

Let's look at some examples of these scenarios. Imagine you want to automatically fetch current weather information from an online service—this is a GET request in action. If you need to send details to a website, such as submitting survey responses, you would use a POST request. Python's requests library allows you to perform both simply, so you can automate interactions and data sharing with the wider internet.

- Collect the latest exchange rates from a currency API
- Submit a contact form automatically to a website
- Download images or files from an online gallery
- Send new messages or data to cloud-based chatbots
- Update a customer database on a remote server

```
import requests

# Example 1: Retrieve data (GET request)
response =
requests.get('https://jsonplaceholder.typicode.com/posts/1')
print('Title:', response.json()['title'])

# Example 2: Submit data (POST request)
data = {'name': 'Chris', 'message': 'Hello!'}
submit_url = 'https://jsonplaceholder.typicode.com/posts'
result = requests.post(submit_url, json=data)
print('Server Response:', result.json())
```

Action	Typical Use Case
GET request	Fetching weather data from a weather API
POST request	Sending survey results to an online form
PUT request	Updating your user profile on a website

Note: Important tip: Always check the documentation for any online API you use. Some services may require you to include authorisation tokens or handle errors if the service is unavailable. Handle sensitive information such as passwords carefully and use secure connections (https) wherever possible.

WORKING WITH DATETIME

Learn how to import and use Python modules and packages to extend your programs.

Python can be expanded using modules and packages. Modules are files containing reusable code such as functions and classes. Packages are collections of related modules, grouped in folders with a special file inside. By importing modules and packages, you add new tools to your programs without writing everything from scratch.

To start using extra features in Python, you need to **import** modules or packages into your script. This tells Python to load the code so you can access its functions and objects.

The most common way to import is using the import statement. There are different ways to import, depending on what you need. Some modules come with Python by default—these are called the standard library. Others are external and must be installed separately.

- Standard library modules like random or datetime are included with Python.
- External packages like requests must first be installed (often using pip).
- You can import whole modules, or just parts you need.

```
# Importing a whole module
import math

# Importing a specific function from a module
from random import randint

# Importing a package (e.g. requests) after installation
import requests
```

Import Style	Example
Basic module import	import os
Import part of a module	from math import sqrt
Import with a nickname	import numpy as np

Note: If you get an ImportError, the package might not be installed. Use 'pip install package-name' to add new packages.

Understand the resources available in the Python standard library for everyday programming needs.

The Python standard library is a collection of modules and packages included with every Python installation. These resources provide ready-made tools for common programming tasks, allowing you

to accomplish more without having to start from scratch. The standard library covers a wide range of areas such as file handling, working with dates and times, performing mathematical operations, and managing data formats like JSON.

Many of the tasks you encounter in programming have already been **solved by the Python standard library**. For beginners, this means you do not need to reinvent the wheel each time you want to read a file, work with web data, or process text.

Importing modules from the standard library is straightforward. Using the import statement, you can access functions, classes, or constants from a module. This saves you both time and effort, and helps ensure your code is reliable.

- `os` – Interact with the operating system, such as listing files in folders or getting environment variables.
- `datetime` – Manage dates and times, calculate intervals, and format time data.
- `json` – Read and write data in JSON format, commonly used in web applications.
- `math` – Perform mathematical functions such as square roots, trigonometry, or constants like `pi`.
- `random` – Generate random numbers or select random items for games or testing.
- `re` – Use regular expressions for advanced pattern-matching and text processing.
- `urllib` – Make web requests or handle URLs.
- `csv` – Read and write Comma-Separated Values files, useful for spreadsheets and data exchange.

```
import datetime

today = datetime.date.today()
print('Today is:', today)

import json
data = '{"name": "Alice", "age": 30}'
parsed = json.loads(data)
print('Parsed name:', parsed['name'])
```

Module	Purpose / Example Use
<code>os</code>	Automate tasks such as renaming files or creating directories
<code>datetime</code>	Calculate someone's age from their birthdate
<code>json</code>	Parse API response data from a website into usable Python objects

Note: Important tip: Before searching the internet for a solution or installing external packages, check if the Python standard library already offers what you need. Using the standard library can often result in simpler, more portable, and more reliable code.

Practice using external libraries to simplify tasks such as making web requests.

Python has a rich ecosystem of external libraries. These libraries are collections of pre-written code that you can use to add features or solve problems easily, rather than writing everything from scratch. Using libraries makes your programs shorter, easier to maintain, and helps you avoid common mistakes.

One of the most common tasks is connecting to websites or accessing data from the internet. Python's **requests** library is popular for making web requests. With just a few lines of code, you can download web pages or interact with online APIs.

Before using an external library, you usually need to install it. Most libraries are available through the Python Package Index (PyPI), which you can access using `pip`—the Python package installer. Once installed, you import the library in your script and use its functions.

- External libraries save time and reduce coding effort.
- Python's `pip` tool helps you install these libraries.
- The `requests` library is used for HTTP operations—like reading data from websites.
- Always read the documentation of the library you use.
- Libraries are updated by the community or maintainers.

```
# Install the requests library (do this in your terminal)
pip install requests

# Example: Use requests to fetch a webpage
import requests
response = requests.get('https://www.example.com')
print(response.status_code)      # Outputs the HTTP status code
print(response.text[:200])      # Prints the first 200 characters
of the page
```


Task	Library Example
HTTP web requests	requests
Handle dates and times	datetime or dateutil
Read and write Excel files	openpyxl
Parse JSON data	json
Plot graphs	matplotlib

Note: Important tip: When using an external library for the first time, be sure to check your spelling in both the install command and the import statement. Python package names are case-sensitive.

Work with Python's tools to manage dates and times in your applications.

Many applications need to handle dates and times—for example, scheduling events, logging activity, or calculating durations. In Python, you have built-in tools that make managing dates and times straightforward. The main way to do this is by using the `datetime` module, which comes with Python and supports a vast range of date and time operations.

The **`datetime` module** lets you create, manipulate, and format date and time values in your programs. It gives you different classes and functions to work with dates, times, or a combination of both. This helps in solving problems such as calculating someone's age, timing how long tasks take, or converting date formats.

Using the `datetime` module, you can get the current date or time, combine date and time into a single object, or perform calculations such as adding and subtracting days. You can also convert dates from strings—which is helpful for reading user input or data files—or format output in the way your users expect.

- Store a user's date of birth and calculate their precise age.
- Log when events happen in a program or system.
- Schedule future actions or reminders.
- Parse dates from text files or web APIs.
- Display dates and times in user-friendly formats.

```
import datetime

# Get the current date and time
now = datetime.datetime.now()
print('Current date and time:', now)

# Extract just the date or time
print('Date:', now.date())
print('Time:', now.time())

# Create a specific date
birthday = datetime.date(1990, 6, 15)
print('Birthday:', birthday)

# Calculate how many days until the next birthday
next_birthday = datetime.date(now.year, 6, 15)
if next_birthday < now.date():
    next_birthday = datetime.date(now.year + 1, 6, 15)
delta = next_birthday - now.date()
print('Days until next birthday:', delta.days)

# Format dates as strings
date_str = now.strftime('%d/%m/%Y')
print('Formatted date:', date_str)
```

Class/Function	Purpose
datetime.date	Represents a date (year, month, day) without time info
datetime.time	Represents a time (hour, minute, second, microsecond) without a date
datetime.datetime	Combines date and time into a single object
datetime.timedelta	Represents a difference or duration between dates or times
datetime.strptime()	Converts a string into a date/time object
datetime.strftime()	Turns a date/time object into a formatted string

Note: Dates and times can be complicated, especially with time zones and daylight-saving changes. For most beginner projects, the datetime module is enough. For more advanced needs, look into third-party libraries like pytz or dateutil.

Handle and process JSON data using appropriate libraries for real-world data handling.

JSON (JavaScript Object Notation) is a very popular way to represent and exchange data. Many web APIs, configuration files, and real-world applications use JSON. In Python, handling JSON is straightforward thanks to the built-in json library, which lets you read (parse) and write (serialise) JSON data with ease.

When working with **real-world data**, you often need to extract information from JSON files, send structured data, or convert between Python objects and JSON. Understanding these tasks will help you integrate your Python code with modern applications and services.

Python's json library offers two main functions: load/dump for files, and loads/dumps for strings. These functions let you convert (serialise) Python objects like dictionaries or lists into JSON, or do the reverse (deserialise) by loading JSON back into Python objects.

- json.load() – Reads JSON data from a file and converts it to a Python object
- json.loads() – Converts a JSON string to a Python object
- json.dump() – Writes a Python object to a file as JSON
- json.dumps() – Converts a Python object to a JSON formatted string

```
import json

# Example 1: Load JSON data from a string
json_str = '{"name": "Alice", "age": 30, "active": true}'
data = json.loads(json_str)
print(data["name"]) # Output: Alice

# Example 2: Write Python object to a JSON file
person = {"name": "Bob", "age": 25, "active": False}
with open("person.json", "w") as f:
    json.dump(person, f)

# Example 3: Read JSON data from a file
with open("person.json", "r") as f:
    loaded = json.load(f)
print(loaded["age"]) # Output: 25
```

Function	Purpose
json.load()	Reads JSON from a file object
json.loads()	Reads JSON from a string
json.dump()	Writes Python object to a file as JSON
json.dumps()	Converts Python object to JSON string

Handling real-world JSON often requires you to inspect data structure. JSON data maps neatly to common Python types. JSON objects become Python dictionaries, arrays become Python lists, and primitives (like strings, numbers, true/false) convert directly. This makes it easy to pull out specific values, loop through items, or modify the data as needed.

Remember to **always check the data structure** before using keys or indexes in your code. JSON is flexible, so fields might be optional or data may be nested several levels deep. Be ready to access nested dictionaries or lists using multiple steps.

- Use `data["key"]` to get a top-level field
- Use `data["outer"]["inner"]` for nested fields
- Loop through lists inside JSON data using `for item in data["list"]`
- Handle missing fields safely with `dict.get()`

Note: Important tip: For complex or very large JSON, consider using external libraries like 'pandas' for analysis, or 'requests' to pull JSON directly from web APIs. Always close your files and use 'with' statements to manage file handles properly.

WORKING WITH JSON

Learn how to import and use Python modules and packages to extend your program's functionality.

One of Python's most powerful features is its ability to use modules and packages. They allow you to expand your program's capabilities without having to write everything from scratch. Modules and packages are collections of code written by others. You can bring them into your script to solve common problems, such as working with dates, files, the internet, or data formats like JSON. Learning to use them will make your coding more efficient and enjoyable.

Modules are single files containing Python code, like functions or variables. **Packages are collections of modules organised in folders with a special file named `__init__.py`.** The 'import' statement lets you access modules and packages. This is the main way to add built-in or external features to your projects.

The Python standard library comes with many modules ready to use. Common examples include 'math' for mathematics, 'datetime' for dates and times, and 'json' for working with JSON data. You can also install third-party packages using a tool called pip. Once a module or package is available, you can import all or part of it to use its functions.

- Import a whole module using: `import module_name`
- Import specific items using: `from module_name import item`
- Assign a short alias with: `import module_name as alias`
- Use `pip install package-name` to add new external packages

```
import math

print(math.sqrt(25))  # Outputs: 5.0

from datetime import date
print(date.today())  # Outputs today's date

import json as js
sample = '{"name": "Sam", "age": 28}'
data = js.loads(sample)
print(data['name'])
```

Statement	Description
import os	Imports the entire 'os' module (operating system utilities)
from sys import argv	Imports only 'argv' from the 'sys' module
import numpy as np	Imports 'numpy' and creates the shorter name 'np'

Note: Important tip: Always check if a module is part of the Python standard library before installing anything new. Using built-in modules can save time and avoid extra dependencies.

Get familiar with the Python standard library and explore its range of pre-built modules.

The Python standard library is a collection of ready-made modules and packages that come bundled with every Python installation. These modules cover a wide range of common programming tasks, including file input and output, mathematical operations, working with dates and times, handling different data formats, and managing system resources. By using the standard library, you can save time and effort, because much of the functionality you need is already built-in.

When you **import** a module from the standard library, you can use its features without writing the code yourself. This lets you write efficient, reliable programs faster. For example, the standard library includes modules like **json** for working with JSON data, **datetime** for dates and times, and **os** for interacting with your operating system.

You do not have to install standard library modules separately. They are always available, no matter which computer you are using. This makes it easier to create cross-platform code. To use a module, you simply import it at the beginning of your script. You can also look up the official Python documentation to find descriptions and usage examples for each module.

- File handling (reading and writing files) with the built-in `open()` function and modules like `os` and `shutil`
- Network communication using modules such as `urllib`, `http`, and `socket`
- Data serialisation and parsing with `json`, `csv`, and `xml` modules
- Mathematics and statistics functions in `math`, `statistics`, and `random`
- Date and time manipulation with `datetime`, `time`, and `calendar`

```
import json
import datetime

# Convert a Python dictionary to a JSON string
person = {"name": "Alice", "age": 30}
json_string = json.dumps(person)
print(json_string)

# Get the current date and time
now = datetime.datetime.now()
print("Current date and time:", now)
```

Module	Purpose
<code>os</code>	Work with files and directories, interact with the operating system
<code>json</code>	Parse and create JSON data
<code>datetime</code>	Manipulate dates and times
<code>random</code>	Generate random numbers
<code>math</code>	Perform mathematical operations

Note: Important tip: Always check the Python standard library documentation if you need a certain function or tool – chances are, there is already a module for it, saving you coding time and reducing errors.

Work hands-on with external libraries to make web requests and process online data.

Python allows you to extend its capabilities by using external libraries. One very common use is making web requests, which lets your programs get data from online sources. You can also use Python to process data you find online, such as information in JSON format. These skills are very useful, especially for automation, data analysis, or connecting to other services on the internet.

The most popular library for making web requests in Python is **requests**. This library makes it easy to download data from websites, application interfaces (APIs), or even your own servers. By combining it with the built-in **json** module, you can read and work with data from many online sources.

To use an external library, you need to install it first. This is usually done with the pip package manager, which comes with most Python installations. Once the library is installed, you import it in your Python code before you want to use its functions.

- Install the external library using pip on your command line.
- Import the library at the top of your Python file.
- Use the library's functions to send requests to a web address (URL).
- Read the data returned from the website or API.
- If the data is in JSON format, use the json module to easily explore or process it.

```
# Step 1: Install requests (done in command line)
# pip install requests

import requests
import json

# Step 2: Make a web request
response =
requests.get('https://jsonplaceholder.typicode.com/todos/1')

# Step 3: Check if the request was successful
if response.status_code == 200:
    # Step 4: Convert the response JSON data to a Python
    dictionary
    data = response.json()
    print('Task Title:', data['title'])
else:
    print('Failed to retrieve data. Status code:',
response.status_code)
```

Task	Python Code Example
Install a library	pip install requests
Import the library	import requests
Make a GET request	requests.get(url)
Read JSON response	response.json()

This approach is the basis of many real-world projects, such as downloading weather data, connecting with social media APIs, or reading government datasets. It is the starting point for building complex data pipelines or interactive applications.

Note: Important tip: Always handle errors when working with web requests. Networks can fail, and sites might be unavailable. Always check the response code and consider using try/except blocks to manage errors gracefully.

Understand how to handle dates and times using Python's built-in modules.

Working with dates and times is a common need in programming. Whether you are logging when an event occurs, processing timestamps in data, or displaying the current date to a user, understanding how to handle these in Python is essential. Python provides built-in modules, notably `datetime` and `time`, that help you manage dates and times efficiently.

The **`datetime` module** is the most commonly used tool for handling dates and times in Python. It allows you to create objects to represent dates, times, and date-time combinations. You can perform arithmetic, format output, and convert between different time formats.

There are several reasons you might need to work with dates and times. You may want to find out how long a process took to run, add or subtract days to a given date, or store the date when a file was created. Python's built-in functions make this straightforward once you learn the basics.

- The `datetime` module can handle dates (year, month, day) and times (hour, minute, second, microsecond).
- The `time` module focuses on working closely with the system clock and is useful for performance measurement.
- You can convert dates to strings and back, making it easy to display or store values.
- Python allows for easy calculations, such as adding days or finding differences between dates.
- The modules support handling time zones and daylight saving, though this is more advanced.

```
import datetime

# Get the current date and time
now = datetime.datetime.now()
print('Current date and time:', now)

# Create a specific date
birthday = datetime.date(1990, 12, 15)
print('Birthday:', birthday)

# Add 7 days to the current date
next_week = now + datetime.timedelta(days=7)
print('One week from now:', next_week)

# Format the date as a string
formatted = now.strftime('%d/%m/%Y, %H:%M')
print('Formatted:', formatted)
```


Function	Purpose
<code>datetime.now()</code>	Get the current date and time
<code>date(year, month, day)</code>	Create a date object
<code>timedelta(days=...)</code>	Represents a difference between two dates/times
<code>strftime()</code>	Format a date/time as a string
<code>strptime()</code>	Parse a string into a date/time object

Note: Important tip: When storing or transmitting dates, use the ISO 8601 format (YYYY-MM-DD). It is clear and avoids confusion from different date orders.

Read, write, and manipulate JSON data for data exchange and storage in Python applications.

JSON (JavaScript Object Notation) is a popular format for storing and transferring data. It is simple to read and write, and works well with Python. Many web services and APIs use JSON as the standard way to exchange information. In Python applications, you will often need to work with JSON files and data.

Python provides the built-in **json** module to make it easy to read, write, and manipulate JSON data. This module can convert Python objects (like dictionaries and lists) to JSON, and parse JSON strings back into Python objects.

To use JSON in Python, you first need to understand the relationship between Python data types and JSON data types. For example, Python dictionaries become JSON objects, and lists become JSON arrays. This makes it simple to store and exchange structured data in a way that both humans and machines can understand.

- Reading JSON means converting a JSON string or file into Python objects.
- Writing JSON means turning Python objects into JSON strings or files.
- Manipulating JSON involves changing the data after loading it into Python.

```
import json

# Example: Read JSON from a string
json_string = '{"name": "Alice", "score": 94}'
data = json.loads(json_string)
print(data['name']) # Output: Alice

# Example: Write JSON to a string
py_data = {"animal": "koala", "number": 2}
```

```

json_out = json.dumps(py_data)
print(json_out)  # Output: {"animal": "koala", "number": 2}

# Example: Read and write JSON files
with open('data.json', 'w') as file:
    json.dump(py_data, file)

with open('data.json') as file:
    loaded = json.load(file)
    print(loaded)
  
```

Python Data Type	JSON Data Type
dict	object
list	array
str	string
int, float	number
True, False	true, false
None	null

When exchanging data between applications or web services, JSON helps keep information structured and consistent. You can also use JSON for application settings, saving results, or passing data to other programs.

Note: Important tip: Always open files using the correct mode (e.g., 'r' for reading, 'w' for writing) and handle file exceptions. Use the 'indent' argument in json.dump or json.dumps to make the output easier to read.

10. Debugging and Testing

COMMON ERRORS (SYNTAX, RUNTIME, LOGICAL)

Learn to recognise and differentiate between common Python error types, including syntax, runtime, and logical errors.

You will often encounter errors and unexpected behaviour when writing Python programs. These errors can stop your program from running, cause it to behave unpredictably, or produce incorrect results. Learning to identify the type of error helps you diagnose and fix issues quickly.

Python errors fall into three main categories: **syntax errors**, **runtime errors**, and **logical errors**. Understanding these types will improve your debugging skills.

Syntax errors occur when you break rules of the Python language. These errors prevent your program from running. Python points out the line and gives a message, such as if you forget a colon or misspell a keyword.

- **Syntax errors:** Mistakes in the structure of your code. The interpreter cannot start your program.
- **Runtime errors:** Happen while your program is running. The code may start but fails when it reaches the error.
- **Logical errors:** The program runs but produces the wrong output, due to a mistake in your logic or design.

```
# Syntax error example
print('Hello, world!'
# Missing closing parenthesis above

# Runtime error example
number = int(input('Enter a number: '))
result = 10 / number # Error if number is 0

# Logical error example
# This should add two numbers, but multiplies instead
def add(a, b):
    return a * b # Wrong operator
```

Error Type	Example and Description
Syntax	Missing colon: <code>for i in range(5) print(i)</code>
Runtime	<code>ZeroDivisionError: result = 5 / 0</code>
Logical	Using <code>*</code> instead of <code>+</code> : <code>result = a * b</code>

Let's explore each type further. Syntax errors are usually easy to spot because Python tells you exactly where they are. You need to correct the syntax before your program will run.

Runtime errors can be trickier. Your code can run for a while and then stop due to invalid operations, such as dividing by zero or opening a file that does not exist. Python will show a traceback message, indicating the line and the type of error.

Logical errors are the hardest to catch. Your program runs, but the output is incorrect. There are no error messages. These mistakes are usually due to a miscalculation, the wrong variable, or a misunderstanding of what the code should do.

- **Check for syntax errors** first if your program will not run.
- **Watch for runtime errors** when your code stops midway.
- **Test your program** with different inputs to spot logical errors.

Note: Important tip: Practice reading Python's error messages. They often contain clues to help you find and fix your mistakes faster.

Apply effective debugging techniques to identify and diagnose issues in your code.

Debugging is the process of finding and resolving errors or unexpected behaviour in your Python programs. Effective debugging helps you create reliable software, understand your code better, and improve your problem-solving skills. Python offers several techniques and tools to help you track down and fix issues, even if you are just starting out. Learning to debug systematically will save you time and frustration.

Start by carefully reading any error messages. **Error messages in Python are designed to provide clues** about what went wrong and where. Pay close attention to the type of error, the line number, and the description. Even if the message seems confusing, it often contains enough information to get you started on the right path. Never ignore error messages—they are valuable hints in the debugging process.

One of the most common and effective techniques for beginners is using print statements. By inserting print statements at different points in your code, you can check the values of variables, see which parts of your code are running, and confirm whether logic flows as you expect.

This method can help isolate where a problem is occurring. For example, if your program is producing the wrong output, printing the variables used in your calculation will allow you to track down unexpected values.

- **Use print statements to display variable values** and trace program flow.
- **Work step-by-step:** Focus on small sections, not the entire program at once.
- **Check for common issues:** spelling errors, incorrect indentation, and missing punctuation.
- Read error messages fully: They often identify the exact issue and its location.
- Comment out parts of code to isolate problems without deleting code.
- Test changes immediately after making them.
- Use debugging tools such as Python's built-in pdb module or IDE debuggers for more control.

```
def divide(a, b):
    print("a:", a)    # Debug print
    print("b:", b)    # Debug print
    result = a / b
    print("result:", result)  # Debug print
    return result

try:
    divide(10, 0)
except ZeroDivisionError as e:
    print("Error:", e)
```

Technique	How it helps
Print statements	Track variable values and identify where logic fails.
Isolating code blocks	Helps pinpoint where errors start appearing.
Using a debugger	Step through code line-by-line and inspect variables.
Reading error messages	Guides you to the source of the issue quickly.
Writing test cases	Catches bugs early by checking code works as intended.

Note: Important tip: If you are stuck, try explaining your code and the issue to someone else, or even to yourself out loud. This technique, called 'rubber duck debugging', often helps you see problems you missed.

Use print statements and available debugging tools to trace and resolve problems.

When you write a Python program, mistakes will happen. These mistakes might be small typing errors, wrong logic, or unexpected data. Learning how to find and fix these problems is a vital skill for every programmer. Two of the most useful ways to trace and fix issues in your code are using print statements and debugging tools.

Python **print statements** are simple commands that display text or values. You can place them throughout your code to show what is happening as your program runs. This helps you see if your variables have the values you expect, or if parts of your code are being reached at all. For more advanced tracing, you can use built-in tools like **IDLE's Debugger** or external tools such as **pdb (Python Debugger)**.

Print statements are most useful when you want quick feedback. You can quickly add print statements to show variable values at different points. This is called 'print debugging'. However, for complex programs or when you need to pause and inspect your program step by step, a debugger is more suitable.

- Use print statements to display variable values and track program flow.
- Print messages before and after important operations to check where errors occur.
- Debuggers let you set breakpoints, watch variables, and step through your code.
- Common debugging tools include Python's pdb, IDE debuggers, and online Python sandboxes.

```
# Example: Using print statements to debug a simple function

def calculate_total(items):
    total = 0
    for item in items:
        print('Adding item:', item) # Shows current item
        total += item
        print('Current total:', total) # Shows total after adding
    return total

shopping_list = [12, 4, 10]
result = calculate_total(shopping_list)
print('Final result:', result)
```

Tool/Method	Main Features
Print Statements	Quick and easy; shows simple values and progress.
pdb (Python Debugger)	Step-by-step execution; inspect variables; set breakpoints.
IDE Debugger	Graphical interface; advanced features like watchpoints and call stacks.

Note: Important tip: After making changes to your code for debugging, remember to remove or comment out print statements before releasing or sharing your program. Too many print statements can clutter your output and make your code harder to read.

Write simple test cases to check the correctness and stability of your Python programs.

Testing is an important part of writing reliable programs. When you create test cases, you check if your code works as expected. Test cases help you find errors early and make sure changes do not break existing features. Writing simple test cases is an essential skill for every Python programmer, whether you work on small scripts or larger projects.

A **test case** is a small piece of code or an idea that checks if part of your program produces the right output for a given input. By creating test cases, you can compare the expected result with the actual result from your program.

When you write test cases, start by thinking about what your function or code should do. Decide on some sample inputs and what the outputs should be. Then, run your code with these inputs to see if it returns the expected results. If there are differences, you can investigate and fix the problem.

- Choose simple, clear inputs for each function or feature.
- Decide what the correct output should be for each input.
- Compare your program's output with the expected output.
- Write down the test cases so you can reuse them later.
- Test both normal and edge cases, such as zero or empty values.

```
def add(a, b):
    return a + b

# Simple test cases
print(add(2, 3) == 5) # Test with two positive numbers
print(add(-1, 1) == 0) # Test with one negative, one positive
print(add(0, 0) == 0) # Test with zeros
```

Input	Expected Output
add(2, 3)	5
add(-1, 1)	0
add(0, 0)	0

Note: Important tip: Test cases are most helpful when you check both normal and unusual situations. Do not forget to test for errors, empty data, and boundary values.

You can also automate testing by using Python's in-built unittest module. This allows you to organise test cases into test functions, so they are easy to run and repeat. Basic testing can start with simple print statements, but moving to formal test functions improves your workflow as your programs grow.

```
import unittest

class TestAddFunction(unittest.TestCase):
    def test_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)
    def test_negative_positive(self):
        self.assertEqual(add(-1, 1), 0)
    def test_zeros(self):
        self.assertEqual(add(0, 0), 0)

if __name__ == '__main__':
    unittest.main()
```

Note: When you write clear test cases, it is easier to find out what went wrong if your program behaves in an unexpected way. Over time, a good set of tests gives you confidence that your code is correct and stable.

Develop a problem-solving mindset for troubleshooting and improving Python code.

A problem-solving mindset is essential for troubleshooting and improving your Python code. This approach helps you find solutions efficiently and prevents you from becoming stuck or frustrated.

Troubleshooting Python code means breaking down issues into manageable pieces and identifying what is causing a problem. **Start by clarifying the issue.** Ask yourself what should happen and what actually happens. Understanding the gap helps you focus your efforts.

Next, isolate the problem. Make your code simpler by commenting out sections or running only part of your program. Try to reproduce the error with the smallest possible amount of code. This process is sometimes called minimising the failing example.

- Describe clearly what the code should do
- Test your assumptions—do not just read code, run it with real data
- Look for simple solutions before complex ones
- Break bigger problems into smaller steps
- Use print statements or debugging tools to check what is happening
- Be patient; not all bugs are obvious or easy to fix

```
# Example: Unexpected behaviour
def calculate_average(numbers):
    total = 0
    for number in numbers:
        total += number
    return total / len(numbers)

# Suppose this crashes with an error if numbers is empty.
# Problem-solving mindset:
# 1. What is the input? []
# 2. What happens? ZeroDivisionError
# 3. What should happen? Perhaps return None or a message.
```

Problem-Solving Step	Action in Python
Clarify the problem	What should the program do? What does it do?
Reproduce error	Run the code with the same input that caused the issue
Minimise the example	Test a small snippet that shows the issue
Change one thing at a time	Alter one part of the code and rerun
Check assumptions	Print values and types to understand what's happening

Note: Remember, every programmer—even professionals—regularly encounters bugs. Developing a patient and systematic approach to troubleshooting will make you more confident and effective.

DEBUGGING TECHNIQUES

Learn how to recognise and differentiate between common Python error types such as syntax errors, runtime errors, and logical errors.

When writing Python programs, you'll encounter different types of errors. Recognising and understanding them is crucial for becoming a confident coder. The three main error types in Python are syntax errors, runtime errors, and logical errors. Each one happens at different stages of a program's life and requires different approaches to resolve.

A **syntax error** occurs when the rules of the Python language are broken. For instance, forgetting a colon or mismatching brackets. These are caught by Python before your code runs. A **runtime error** happens when your code runs into an issue while it's being executed, such as dividing by zero or trying to open a file that does not exist. These errors stop your program immediately when they occur. **Logical errors** are trickier—they do not stop the program, but they cause it to behave incorrectly because your logic or calculations are not producing the right result.

Identifying the error type helps you fix problems more efficiently. If your code won't run at all, it's usually a syntax error. If the program runs but then crashes, a runtime error is likely. If your result isn't what you expect but the program finishes, you're probably facing a logical error.

- Syntax errors prevent your code from running.
- Runtime errors crash your program during execution.
- Logical errors produce wrong results but don't stop the program.

```
# Syntax Error Example
print('Hello world'  # Missing closing bracket

# Runtime Error Example
a = 5
b = 0
print(a / b)  # Division by zero

# Logical Error Example
length = 10
width = 2
area = length + width  # Logic mistake: should multiply, not add
print('Area:', area)
```

Error Type	When It Occurs
Syntax Error	Before program starts (compilation phase)
Runtime Error	During program execution
Logical Error	Any time, but produces incorrect output

Note: Remember, syntax and runtime errors usually show error messages, but logical errors may not be obvious. Testing your code helps catch logic mistakes early.

Apply proven debugging techniques, including systematic code inspection and step-by-step execution.

Debugging is a core skill for every programmer. When something goes wrong in your Python program, it is essential to know how to find and fix the issue quickly. This means using reliable techniques to inspect and test your code. Learning to debug properly will save you time and help you write better programs.

Two of the most effective methods are **systematic code inspection** and **step-by-step execution**. These strategies help you break down complex problems, closely examine your program, and find exactly where things have gone wrong.

Systematic code inspection means going through your code one part at a time. It is useful for catching mistakes that are easy to overlook—missing colons, indentation errors, typos in variable names, or logic mistakes. Reading code carefully, line by line, is often the fastest way to spot simple problems.

- Read each line of code out loud or explain it to someone else (rubber duck debugging).
- Check that brackets, colons, and indentation are correct.
- Verify variable names and function calls match where they are used.
- Compare your code to known examples or reference solutions.

Step-by-step execution involves running your program in small chunks. You watch what happens at each stage, checking the values of variables as the program moves forward. This allows you to see the flow of logic and notice when something goes wrong.

```
def divide(a, b):  
    return a / b  
  
n = 12  
m = 0  
result = divide(n, m)  
print(result)
```

In the sample above, an error will occur when you run the program. By adding print statements or using a debugger, you can find out where and why it fails.

- Insert print statements before and after calculations.
- Print variable values to check they are what you expect.
- Pause the program to investigate the state at a specific line.

Technique	Description
Systematic code inspection	Read code line by line for mistakes or inconsistencies.
Step-by-step execution	Run code stepwise, checking values and program flow.
Print statement debugging	Insert print statements for insight into values and logic.

Note: Important tip: Always start debugging with the simplest method. Often, a close look at the code or a well-placed print statement is all you need to solve a tricky problem.

Use print statements and Python's built-in debugging tools to identify and fix issues in your programs.

Debugging is the process of finding and fixing mistakes in your code. Even simple programs can have errors or behave in unexpected ways. Learning how to spot and resolve these problems efficiently is an essential skill for any Python programmer. There are several approaches to debugging, but two of the most common methods are using print statements and Python's built-in debugging tools.

The **print statement** is a simple yet effective tool for tracking what your program is doing. By adding print statements at strategic places in your code, you can display key information such as variable values, function progress, and the flow of execution. This technique helps you pinpoint where things start to go wrong.

For example, if your program does not return the output you expect, you can insert print statements before and after suspicious lines to display the value of variables as the program runs. This helps you confirm whether a value is what you expect it to be at a particular point.

- Insert print statements inside loops or functions to check intermediate results.
- Print the values of variables before performing important calculations or logic checks.
- Remove or comment out print statements once you have found and fixed the issue to keep your code clean.

```
number = 10
print('Before doubling:', number)
number = number * 2
print('After doubling:', number)
```

While print statements are useful, they have some limitations, especially in larger or more complex programs. That's where Python's built-in debugging tools come in. Python includes a built-in module called `pdb` (the Python Debugger). This tool allows you to pause your program, inspect variables, step through code line by line, and better understand exactly what's happening in your code.

Debugging Method	When to Use
Print statements	Quick checks on variable values or program flow
<code>pdb</code> module	Complex bugs, step-by-step inspection, or interactive investigation
IDLE/debugger in IDE	Visual debugging with breakpoints and watch windows

```
import pdb

def divide(a, b):
    pdb.set_trace() # Pause and start interactive debugging
    session
    return a / b

result = divide(10, 2)
```

When you run this program, execution will pause at `pdb.set_trace()`. You can then enter commands in the terminal to inspect variables, step through the code, or continue execution. Try commands like `'n'` (next line), `'c'` (continue), or `'p variable_name'` (print a variable's value).

Note: Important tip: Don't leave print statements or `pdb` calls in your code before sharing or deploying your program. Only use them during development and debugging.

Develop and run simple test cases to verify that your code works as intended.

Testing is a crucial step in programming. It helps ensure that your code produces the correct results and behaves as you expect. After writing any function or section of code, you should check that it works as planned by running tests. In Python, this is often done using simple test cases—small examples where you know what the output should be if the code is correct.

When you **develop and run test cases**, you provide both the input and the expected output for your code. Then you compare the actual result to see if it matches. If it does, your code is likely working. If not, you can investigate and fix problems early.

Testing does not have to be complicated. At the start, you can test your code by calling functions with known inputs and printing the outputs. The goal is to catch mistakes before they cause bigger problems later.

- Test cases help you confirm your code works as expected.
- They can reveal errors, incorrect assumptions, or unexpected behaviour.
- Writing simple tests early makes it easier to fix issues and understand your code.

```
# Function to add two numbers
def add_numbers(a, b):
    return a + b

# Simple test cases
print(add_numbers(2, 3)) # Expected output: 5
print(add_numbers(-1, 1)) # Expected output: 0
print(add_numbers(0, 0)) # Expected output: 0
```

Input	Expected Output
add_numbers(2, 3)	5
add_numbers(-1, 1)	0
add_numbers(0, 0)	0

Note: Important tip: Always test both typical inputs and unusual cases, such as negative numbers or zero. This helps you catch more potential errors and makes your code more reliable.

Develop good debugging habits to efficiently troubleshoot and resolve coding problems.

Good debugging habits are an essential part of becoming a confident Python programmer. Debugging is the process of finding and fixing errors in your code. By developing effective habits, you can save time, avoid frustration, and consistently deliver reliable programs.

When you **approach debugging methodically**, you reduce guesswork and increase your chances of identifying the real cause of a problem. This structured approach is more productive than randomly changing parts of your code.

Start by carefully reading error messages. Python error messages often tell you exactly where something went wrong. Do not ignore them. Look for the file name and line number to locate the issue. If the message is unclear, search online or consult the Python documentation for clarification.

Break big problems into smaller parts. Debug one piece at a time. If your code produces unexpected results, isolate the section causing the problem. Test your assumptions by using print statements or a debugger to watch how values change as your program runs.

- Read error messages carefully – look for keywords and line numbers.
- Comment out or temporarily remove sections of code to narrow down issues.
- Use print statements to check the values of variables at different steps.
- Keep your code tidy and named sensibly, which makes mistakes easier to spot.
- Test your code with simple inputs before using more complex data.
- Fix one issue at a time and re-test to confirm it is resolved.

```
def calculate_total(items):
    total = 0
    for item in items:
        print(f'Adding price: {item["price"]}') # Debug print
        total += item["price"]
    return total

shopping_list = [{"name": "apple", "price": 2}, {"name": "banana",
"price": 1}]

print(calculate_total(shopping_list))
```

Habit	Benefit
Read errors closely	Locate bugs quickly
Test in small steps	Easier to find where things go wrong
Use version control	Undo mistakes and track changes
Break down problems	Makes large issues manageable

Note: Important tip: Do not be afraid to ask for help. Many developers get stuck. Talking through the problem or searching for similar issues online can often lead to a solution much faster.

USING PRINT VS DEBUGGER

Use print statements to display variable values and track program flow during debugging.

When trying to find and fix errors in your Python program, you often need to know what is happening as the code runs. Using print statements is one of the simplest ways to help you see what your program is doing and understand how data is changing at each step.

A **print statement** sends output to your screen. By placing these statements at key points in your code, you can check the values of variables and make sure the program is following the path you expect. This technique is helpful when learning and also useful in larger projects, especially in situations where you can't or don't want to use a more advanced debugging tool.

For example, suppose you have a program that calculates the average of a list of numbers, and the result seems wrong. To understand what is happening, you can print the values at different stages. You might want to print the list, the running total, or even which items are being processed.

- Use print statements after variables change to check their new values.
- Insert print statements inside loops to follow the flow and see repeated actions.
- Print helpful messages showing when a certain part of the code is reached.
- Remove or comment out print statements when debugging is complete.

```
numbers = [10, 20, 30, 40]
total = 0
for number in numbers:
    total += number
    print('Current number:', number)
    print('Running total:', total)
average = total / len(numbers)
print('Final average:', average)
```

Location	Example Print Output
Inside loop	Current number: 10 Running total: 10
After calculation	Final average: 25.0
Conditional branch	Item skipped: None

Note: Important tip: When using print statements for debugging, remember to keep the messages clear and brief. You may want to add labels or extra context, especially if you have many values to check. Always tidy up your code before sharing it or putting it into production—remove any print statements that are no longer useful.

A debugger allows you to set breakpoints and inspect the program state at specific points.

A debugger is a specialised tool that helps you find and fix errors in your program. It lets you pause your program at chosen lines, known as breakpoints, and closely examine what is happening. This can save you time and effort compared to using print statements.

When you **set a breakpoint**, the debugger will pause your program's execution at that line. While the program is paused, you can check the values of variables, see which code has been executed, and step through your code one line at a time.

Breakpoints are useful for checking the program state at specific points, such as before or after a calculation, inside a loop, or when a particular condition is met. You do not need to modify your source code with extra print statements. Instead, you interact with your debugger's interface to pause and inspect the program.

- Breakpoints help pause your program at chosen lines.
- You can watch variables and expressions as your code runs.
- Stepping through the code shows how data changes.
- Debuggers highlight where errors or bugs appear.
- No need to add or remove print statements repeatedly.

```
x = 10
for i in range(5):
    x += i # Set a breakpoint here to watch 'x' and 'i'
    print(x)
```

Method	What It Does
Breakpoint	Pauses the program at a set line so you can inspect
Step Over	Runs the next line, then pauses again
Inspect Variables	Shows current values of your data

Note: Important tip: In many code editors like VS Code or PyCharm, you can set breakpoints by clicking next to the line number. Use the debug panel to view variables and control the flow without changing your code.

Print statements are simple and quick for small issues, but can clutter code if overused.

When you run into problems in your Python program, a common way to figure out what's happening is to add print statements. Print statements can quickly display the value of variables or show which parts of your code are being reached. This is helpful when you have a small problem or are just starting out with debugging.

However, **using print statements everywhere can make your code messy**. If you add too many, it gets hard to see which prints are for debugging and which ones are for something else. This clutter can make it tricky to find bugs and to maintain your code later.

Print statements work best for quick checks, such as confirming the value of a variable, or seeing if a certain part of your program is being run. But as your programs grow bigger or become more complex, relying on lots of print statements can be a problem. You might forget to remove them, or accidentally print sensitive information.

- Print statements are easy to add and remove.
- They help you see values and program flow immediately.
- Too many print statements can make your code hard to read.

```
def add_numbers(a, b):  
    print('Adding', a, 'and', b)  
    result = a + b  
    print('Result is', result)  
    return result  
  
add_numbers(2, 3)
```

Advantage	Disadvantage
Quick to use	Can clutter code
Shows values immediately	May forget to remove
No extra tools needed	Not suitable for complex bugs

Note: Important tip: Use print statements for simple, short-term checks. For bigger programs, consider learning to use a debugger. This allows you to inspect values and program flow without changing your actual code.

A debugger provides more control and helps trace complex bugs without modifying your code.

Debugging is an essential skill for every programmer. In Python, you have two main approaches: using print statements or using a debugger. Print statements are quick and easy, but a debugger offers much more control when tracking down problems, especially in larger or more complex code.

A debugger is a specialised tool built for finding and fixing errors in your code. **Unlike print statements, a debugger allows you to pause your program, inspect what is happening at any point, and examine your variables step by step.** You do not have to add or remove lines of code just to see values, which saves time and reduces mistakes.

When you use print statements, you must edit your code by inserting temporary prints to observe values. This can clutter your script and is often not enough when dealing with complicated logic, loops, or external libraries. If you forget to remove a print, it might appear in production or cause confusion later.

- A debugger lets you pause (break) at a chosen line.
- You can inspect variables and data structures instantly.
- You can step through your code one line at a time.
- It allows you to watch how values change as your program runs.
- You do not need to change your source code for tracing.

```
def divide(a, b):
    result = a / b
    return result

# Suppose a user enters a wrong value
print(divide(10, 0)) # This will crash with ZeroDivisionError

# Instead of adding prints, use a debugger to track what happens
```

Feature	Print Statement	Debugger
Modify code?	Yes	No
Pause program?	No	Yes
Step through lines?	No	Yes
Inspect all variables?	Manual, limited	Full, instant
Track complex bugs?	Difficult	Efficient

Note: Important tip: For longer or more complicated programs, or when you face errors you cannot explain, switch to a debugger. Most modern code editors (such as VS Code, PyCharm, or Thonny) have built-in debuggers that are easy to use. With practice, the debugger will save you time and help you grow as a Python programmer.

Choose print for rapid checks; use the debugger for in-depth troubleshooting.

When you are writing Python code, it's important to have tools to help you find and fix errors. Two common tools for this are print statements and debuggers. Each has its own strengths and best uses. Knowing when to use print or a debugger will make your programming experience smoother and help you solve problems more quickly.

If you want a **quick check** on a variable's value or to see if a bit of code is running, using print is often the fastest choice. For more complex issues—such as tracking how a program's values change over time, or understanding why your code is not behaving as expected—using a debugger is a much better option. The debugger lets you pause your code, inspect variables, and step through your program line by line.

Trying to solve all problems with print can become messy. Print statements are quick to add and remove, making them ideal when you want to confirm if something is happening or see what a variable contains at a specific moment. However, if you need deeper insight, or if the issue is happening in a larger, more complicated program, using a debugger is far more efficient and organised.

- Use print to quickly show variable values or confirm code execution.
- Choose the debugger when you need to pause, step through, or analyse complex issues.
- Mixing both methods can speed up troubleshooting, but over-using print can clutter your code.

```
# Example: Using print for a rapid check
x = 7
y = 3
print('The total is:', x + y)

# Example: Using the built-in debugger
import pdb
x = 7
y = 3
pdb.set_trace()  # This will pause at this line
result = x + y
print('Result:', result)
```

Method	Best Use Case
Print	Quick, simple checks of variables or code flow
Debugger	Stepping through code, inspecting complex logic
Print + Debugger	Initial rapid checks followed by deeper inspection

Note: Important tip: After using print for spot checks, remember to remove or comment out your print statements when you finish debugging to keep your code clean. For persistent or tricky bugs, don't hesitate to use the debugger—it provides much more detail and control.

WRITING SIMPLE TEST CASES

Understand the importance of testing code to ensure correct program behaviour.

Testing is a crucial part of writing reliable and safe programs. When you write code, it is important not only to make it work once, but to make sure it works every time, in different situations. Testing helps guarantee that your program produces the right results, catches mistakes early, and builds confidence in your work.

Whenever you create a program, there can be **unexpected bugs** or small typing mistakes. Testing helps you find these problems before the program is used by others or connected to bigger systems.

You cannot always predict how users will interact with your program or what kind of input will be given. Testing your code with different values and under different conditions ensures your program reacts correctly. For instance, what happens if someone leaves a field empty or enters a negative number? Testing helps you find out.

- Testing checks for errors that are easy to miss by reading code.
- It makes your program more robust, so it handles surprises gracefully.
- Well-tested code is easier to change and improve later.
- Testing is valued in professional software development teams.
- It saves time and effort by preventing bugs from reaching users.

```
def add_numbers(a, b):
    return a + b

# Simple test cases
print(add_numbers(2, 3))      # Expected: 5
print(add_numbers(-1, 1))    # Expected: 0
print(add_numbers(0, 0))     # Expected: 0
```

Test Input	Expected Output
add_numbers(2, 3)	5
add_numbers(-1, 1)	0
add_numbers(0, 0)	0
add_numbers(10, -5)	5
add_numbers(100, 200)	300

Note: Important tip: Always test the most common inputs first, then try unusual or edge case values. This can reveal hidden problems that may not appear during ordinary use.

[Learn how to write and run simple test cases to check individual functions and code blocks.](#)

Testing your Python code is an essential part of programming. Test cases help you make sure that your code works as expected. By writing and running simple test cases, you can check the behaviour of individual functions and small sections of your code. This ensures that each part works correctly before you combine everything together.

When you develop programs, mistakes can slip in easily. **A test case is a small, repeatable check that proves your code produces the correct result.** For example, if you have a function that adds two numbers, a test case could check that the function returns 7 when you add 3 and 4.

In Python, you can write test cases in several ways. Beginners usually start by writing simple test code using print statements to check outputs. However, Python's built-in unittest module lets you automate and organise your tests for better results. Later, you may use other testing frameworks, but unittest is a good place to start.

- Test cases help you confirm that a function does what it's supposed to.
- They catch errors early, before your programs become large and complex.
- Automated tests save you time—no need to check by hand after every change.
- Well-written tests make it safer to improve or update your code later.

```
# Example: Testing a simple add function
def add(a, b):
    return a + b

# Basic manual test using print
print('Test 1:', add(2, 3) == 5) # Should print: Test 1: True
```

```
# Using unittest for a better approach
import unittest

class TestAddFunction(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)
    def test_add_zero(self):
        self.assertEqual(add(0, 0), 0)
    def test_add_negative(self):
        self.assertEqual(add(-1, 1), 0)

if __name__ == '__main__':
    unittest.main()
```

Approach	Description
Manual print test	You compare the output yourself in the terminal.
unittest module	You write reusable tests and Python checks the output for you.
Other frameworks	Tools like pytest offer extra features and simpler code, useful as you advance.

Note: Important tip: Start simple. Begin by checking outputs with print statements, but learn to use unittest soon. Automated tests catch more errors and help your confidence grow, especially as your projects expand.

Use print statements and assertions to verify program output during testing.

Testing is a key part of programming. When you write Python code, you need ways to check that your program behaves as expected. Two simple, effective ways to test your code are using print statements and assertions. These methods help you verify results, spot mistakes, and build confidence in your solutions.

When starting out, **print statements** are often the first tool you use. They show what your program is doing by displaying information in the terminal. You can add print statements in your code to check variable values or see which parts of the code are running. This is known as ‘print debugging’. While print statements are easy to use, they do not stop your program if something is wrong; they just provide clues about the program’s state.

Assertions, on the other hand, make the program check a condition. An assertion is a statement that must be true at a certain point in your code. If it isn’t, Python stops the program and gives you an error message. Assertions help you catch mistakes early. They make your tests automatic rather than manual. For example, you can assert that the output of a function matches the value you expect.

- Print statements help you observe what's happening inside your program.
- Assertions enforce correctness by stopping the code when a condition fails.
- Both tools are helpful for beginners and professionals alike.
- Use print statements to track progress and values.
- Use assertions to define what should always be true.

```
# Example using print statements

def add_numbers(a, b):
    result = a + b
    print('Adding:', a, '+', b, '=', result)  # Shows the
    calculation
    return result

sum_value = add_numbers(2, 3)
print('Sum is:', sum_value)  # Shows final result

# Example using assertions
expected = 5
assert sum_value == expected, 'Test failed: Expected 5, got
{}'.format(sum_value)
```

Method	What it Does
Print statement	Displays information for manual checking
Assertion	Automatically checks a condition and halts if false

Note: Important tip: After you move beyond simple programs, consider removing or commenting out print statements to keep your output clean. Keep assertions in place for ongoing checks if they help your program stay correct.

Identify and fix errors found during test case execution.

Once you have written test cases for your Python code, you will run these tests to check if your program works as expected. When a test fails, it means there is an error—sometimes in your code, sometimes in the test itself. It is important to identify exactly where the problem is and then fix it systematically.

Start by reviewing the **output from your test runner**. Most Python test tools will show you which test failed and provide details about the failure. Pay close attention to the error message and any traceback shown, as this information often tells you which line in your code caused the problem.

Debugging is a process of investigation. First, read the failing test and confirm what it is supposed to check. Verify if the test itself is correct. Next, look at the code being tested. Think about what the error message is telling you and whether you can spot anything obviously incorrect, such as a typo or incorrect logic.

- Check the error message for clues about the problem.
- Read the test case and your program code carefully.
- Identify if the test or code needs to be corrected.
- Fix the problem and run the test again.
- Repeat the process until all tests pass.

```
# Example function and test case
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0

test_add()

# Suppose you accidentally wrote:
def add(a, b):
    return a - b # Error here!
# The test will fail on the first assert.
```

Error Type	Usual Cause
AssertionError	Test condition was not met; likely logic bug in your code.
TypeError	You used an incorrect data type (e.g., adding number and string).
NameError	Variable or function name is not defined; possible typo.

Note: Important tip: When a test fails, do not guess the cause—always read the error message and test details carefully. Fix one error at a time and re-run your tests to make sure the issue is resolved.

Document test cases and results to support ongoing program development.

Testing your programs is an important habit. Just as important is documenting each test case and its results. This helps keep track of which parts of your code work and which need fixing. Good documentation also makes it easier to improve your program over time and helps others understand your work.

When you **document test cases**, you record all the details about how you tested your program: what you tried, what you expected to happen, and what actually happened.

Writing down test information means you do not have to rely on your memory. It also provides a reference for other programmers who might work with your code in the future. Clear documentation helps everyone keep track of changes, fixes, and new features.

- Test case name or ID: A simple label that identifies the test.
- Inputs: The data or conditions you used to run the test.
- Expected outcome: What you hope to see if the code works correctly.
- Actual result: What happened when you ran the test.
- Pass/fail status: Did the test produce the expected outcome?

```
# Example function to test
def add(a, b):
    return a + b

# Example test case documentation
# Test Case ID: TC001
# Description: Add two positive numbers
# Inputs: a=2, b=3
# Expected Result: 5
# Actual Result: 5
# Status: PASS
```

Test Case ID	Inputs	Expected Result	Actual Result	Status
TC001	a=2, b=3	5	5	PASS
TC002	a=-1, b=5	4	4	PASS
TC003	a='2', b=3	Error	TypeError	PASS

Note: Important tip: Keep your test case documentation up to date. Whenever you fix a bug or add a feature, update your tests and results. This makes it easier to spot problems early and keeps your code reliable over time.

11. Introduction to Object-Oriented Programming

CLASSES AND OBJECTS

Object-oriented programming (OOP) in Python centers on creating classes and objects to structure code efficiently.

Object-oriented programming, often called OOP, is a method of organising code that groups related data and behaviour together. In Python, this is achieved through classes and objects. OOP helps you build programs that are easier to understand, update, and reuse.

You can think of OOP as a way to model real-world ideas in your code. **A class acts as a blueprint or template** for creating objects, which are specific instances based on that template. This allows you to create multiple objects with shared structure, but each can have its own unique values.

For example, imagine you are creating a program about cars. Instead of writing separate code for every car, you could make a Car class. This class defines what a car is and what it can do. You can then create several car objects, each representing a specific car with its own details.

- A class defines attributes (data) and methods (functions) that an object will have.
- An object is a concrete example created from the class.
- OOP structures code into logical pieces, improving clarity and maintenance.

```
class Car:
    def __init__(self, make, model, colour):
        self.make = make
        self.model = model
        self.colour = colour

    def drive(self):
        print(f'The {self.colour} {self.make} {self.model} is driving!')

my_car = Car('Toyota', 'Corolla', 'blue')
my_car.drive()
```

Term	Description
Class	A template for creating objects with specific attributes and methods.
Object	An individual piece of data created from a class.
Attribute	A value stored inside an object (for example, a car's colour).
Method	A function defined inside a class that operates on the object.

Note: Remember: OOP in Python helps you reduce repetition and keeps your code organised as your projects grow. Start small with simple classes and objects, and you'll soon find it easier to manage bigger programs.

A class defines a blueprint for objects, specifying attributes (data) and methods (functions) related to the object.

In Python, a class is a powerful way to group related data and behaviour together. You can think of a class as a template or blueprint for building objects. Objects made from this template are known as instances. Each object can hold its own information and perform actions defined by the class.

A class describes two main things: **attributes**, which are pieces of data that belong to every object of the class, and **methods**, which are functions that describe what actions the objects can perform.

This means that every object created from a class will have its own set of data (the attributes) and will be able to perform certain tasks (the methods), based on what the class says.

- A class acts as a template for creating objects.
- Attributes store data related to the object.
- Methods are functions that describe object behaviour.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} says woof!")

my_dog = Dog("Rex", 5)
my_dog.bark()
```

Part	Description
Class	Defines the blueprint (Dog)
Attributes	Data inside each object (name, age)
Methods	Functions attached to the class (bark)

Note: When you create a new object from a class, it's called an instance. Each instance can have different attribute values, but they all start with the same structure set by the class.

Objects are instances of classes, each with their own unique values for the attributes defined in the class.

In Python, object-oriented programming is built around the use of objects and classes. A class provides a blueprint for creating objects. When you create an object based on a class, you are said to be creating an instance of that class. Each instance or object has its own set of values, known as attributes.

Think of a class as a recipe for making biscuits. The recipe contains instructions, but you can make many **biscuits**, each biscuit coming out of the oven will be slightly different depending on the ingredients used, size, or shape. In Python, you can create multiple objects from a single class, and each object can have different attribute values.

Attributes are variables that are stored inside an object. When you define a class, you specify which attributes every object of that class will have. However, the actual values of these attributes can differ between objects. This allows you to represent real-world entities, where each object is similar in structure but different in details.

- A class defines what attributes an object will have.
- An object is an individual instance of a class.
- Each object has its own set of values for those attributes.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Create two Dog objects
fido = Dog("Fido", 4)
bella = Dog("Bella", 2)

print(fido.name)    # Outputs: Fido
print(bella.age)    # Outputs: 2
```

Object	name	age
fido	Fido	4
bella	Bella	2

Note: Remember, although fido and bella are both Dog objects, they can have different values for their attributes. Changes to one object will not affect the other unless they are explicitly linked.

Constructors, using the `__init__` method, allow you to set initial values for object attributes when an object is created.

In Python, objects are created from classes. When you create a new object, you often want it to start with certain values already set. This is where constructors come in. In Python, the constructor method is named `__init__`. It runs automatically every time you make a new object from a class. This allows you to initialise object attributes as soon as the object is created.

Think of `__init__` as a special method that sets things up for your object. When you call the class to make a new object, Python runs this method behind the scenes. The values you pass when making the object help you set up its attributes.

For example, imagine a class called `Car`. When you create a new `Car` object, you may want to set its colour, model, and year. By writing an `__init__` method, you can ask for these values and store them as attributes. This makes sure each `Car` object can have its own unique information from the start.

- The `__init__` method is always named with double underscores before and after: `__init__`.
- It is called automatically when an object is created from a class.
- You use `self` to refer to the current object, allowing each object to store its own attributes.
- You can pass in values as arguments to set up the initial state of each object.
- Without an `__init__` method, objects still get created, but you would have to set attributes after creation.

```
class Car:
    def __init__(self, colour, model, year):
        self.colour = colour
        self.model = model
        self.year = year

my_car = Car('blue', 'Toyota Corolla', 2020)
print(my_car.colour)    # Outputs: blue
print(my_car.model)     # Outputs: Toyota Corolla
print(my_car.year)      # Outputs: 2020
```

Part	What it does
<code>def __init__(self, ...)</code>	Defines the constructor method for the class.
<code>self.colour = colour</code>	Stores the colour provided as an attribute of the object.
<code>my_car = Car('blue', 'Toyota Corolla', 2020)</code>	Creates a new object, setting the attributes immediately.

Note: Important tip: You can make attributes optional by giving them default values in the `__init__` method. This lets you create objects without passing every value every time.

Inheritance enables a class to use features of another class, supporting code reuse and the creation of more complex systems.

Inheritance is one of the key principles of object-oriented programming. It allows you to create a new class based on an existing class, so that the new class inherits characteristics and behaviour from the original. This makes it easier to organise code, avoid duplication, and build on previous work.

The class you base your new class on is called the **parent class**, or sometimes the **superclass**. The new class is the **child class**, or **subclass**. The child class inherits attributes and methods from its parent. You can then add new features to the child class, or change how it behaves.

With inheritance, you can avoid writing the same code in multiple places. When you want to change a shared behaviour, you only need to update it in one class. Inheritance also lets you organise related classes into logical groups, building complex systems step by step.

- Inheritance helps you avoid repeating code by sharing features between classes.
- You can add or override attributes and methods in the child class as needed.
- Complex behaviour can be built by layering more specialised child classes on top of general parent classes.

```
class Animal:
    def speak(self):
        print('Some generic animal sound')

class Dog(Animal):
    def speak(self):
        print('Woof!')

pet = Dog()
pet.speak()  # Output: Woof!
```

Term	Meaning
Parent Class	The original class that shares its features
Child Class	The new class that inherits features from the parent
Override	Redefine a method in the child class to change how it works

Note: Important tip: Inheritance is best used when there is a clear 'is a' relationship between classes. For example, a Dog is a type of Animal. Avoid using inheritance simply to share code between unrelated objects.

ATTRIBUTES AND METHODS

Attributes are variables that hold data within a class, representing the properties or state of objects created from the class.

When you start working with object-oriented programming in Python, you'll hear a lot about attributes. Attributes are a key part of what makes classes powerful. They are variables you create and store inside a class. These variables hold data that belong to each object you make from that class. In other words, attributes are what help describe the unique properties of each object.

Imagine you are building a class to represent a **Car**. A Car object might have attributes like **colour**, **make**, **model**, and **year**. Each of these describes something specific about a particular car.

By storing details this way, you can create many different car objects from the same class, but each with its own unique data. For example, one car object could be a blue Ford made in 2020, while another could be a red Toyota from 2015. This is possible because each object has its own set of attribute values.

- Attributes are variables defined in a class.
- They exist within each object created from that class.
- Each object has its own independent attribute values.
- Attributes represent the state or information of an object.
- They make it easy to organise and keep track of data in your program.

```
class Car:
    def __init__(self, colour, make, model, year):
        self.colour = colour
        self.make = make
        self.model = model
        self.year = year
```



```
car1 = Car('blue', 'Ford', 'Focus', 2020)
car2 = Car('red', 'Toyota', 'Corolla', 2015)

print(car1.colour)    # Output: blue
print(car2.make)      # Output: Toyota
```

Object	colour	make	model	year
car1	blue	Ford	Focus	2020
car2	red	Toyota	Corolla	2015

Note: Important tip: In Python, attributes are usually set up as part of a class's initialiser function, called `__init__`. This lets you set up each object with its own unique data when it is created.

Methods are functions defined inside a class, used to perform actions or manipulate the data of the class objects.

In Python, methods are a key part of classes. They are blocks of code written inside a class. Methods help you carry out actions or change the state of objects created from that class. When you write a method, you are describing a specific action that objects of your class can do.

The main purpose of methods is to **perform actions using the data that belongs to an object**. Methods do this by using the attributes (data) stored in the object. You use methods to do things like update details, calculate values, or display information.

To define a method in a class, use the 'def' keyword, just as you would for a regular function. The first parameter of every method must be 'self', which is a reference to the current object. This allows the method to access and change the object's data.

- A method is a function defined inside a class.
- Methods can read or change data belonging to the object.
- You call a method using the dot (.) operator, for example: `object.method()`.
- The 'self' parameter lets the method know which object it is working with.

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print(f"{self.name} says meow!")
```

```
# Create a Cat object
my_cat = Cat("Whiskers")
my_cat.meow() # Output: Whiskers says meow!
```

Term	Explanation
Method	A function defined inside a class that acts on class data
self	A reference to the current object; allows access to attributes and other methods
Object	An instance created from a class; has its own data and can use the class methods

Note: Remember, every method in a class needs 'self' as the first parameter, even if you do not use it inside the method. This is how Python links the method to the object.

Attributes can be accessed and modified using 'self' within class methods or by referencing the object instance.

In Python object-oriented programming, attributes are the named values that belong to an object or class. You often need to access or change these values inside your class methods or from outside the class, using the object itself. Understanding how to work with attributes using 'self' and object references is a key skill in Python.

Inside a class, methods refer to attributes using 'self'. This keyword always points to the current object (the instance) that is using the method. Outside the class, you refer to attributes directly using the object instance's name.

The 'self' parameter must come first in all methods you create within a class. This lets you access and modify the attributes that belong to each specific object. Importantly, 'self' is not a keyword, but it has become the standard convention in Python, so you should always use it.

- Use 'self.attribute' to get or set values inside class methods.
- Use 'object.attribute' to get or set values from outside the class.
- Every object has its own set of attribute values.

```
class Dog:
    def __init__(self, name):
        self.name = name # set attribute using 'self'

    def rename(self, new_name):
        self.name = new_name # modify attribute with 'self'

my_dog = Dog('Banjo')
```

```
print(my_dog.name)      # Access via object instance

my_dog.rename('Bingo')
print(my_dog.name)      # Attribute has been updated
```

Where	How to Access/Modify
Inside class method	self.attribute
Outside class	object.attribute
Constructor (__init__)	self.attribute = value

Note: Remember, using 'self' is essential for Python classes. It keeps your object's data separate from others, so each object works independently.

Defining methods allows objects to have specific behaviours, making the class more functional and organised.

In object-oriented programming with Python, methods are functions that belong to a class. When you define methods inside a class, you allow the objects created from that class to perform specific actions. These actions are called behaviours. By grouping related methods within a class, your code becomes more organised and easier to maintain.

Imagine you're creating a **Dog** class. Each dog might have attributes, such as name and age. By adding methods like **bark()** or **sit()**, you describe what each dog object can do.

Defining methods in a class helps make sure that related functions are grouped together. This keeps your code tidy and easier to understand. Rather than writing separate functions for every action, you attach them to the object they relate to. This mirrors the way things work in the real world—an object has its own abilities or behaviours.

- Methods describe what an object can do (its behaviours).
- All methods are defined inside the class and act upon the object's data.
- Methods help keep code better organised and reduce repetition.
- You can call methods on an object using dot notation (e.g., my_dog.bark()).

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name} says Woof!")
```

```
def sit(self):
    print(f"{self.name} sits down.")

my_dog = Dog("Buddy")
my_dog.bark()
my_dog.sit()
```

Term	Explanation
Method	A function defined within a class that belongs to its objects.
Behaviour	The actions an object can perform, defined by its methods.
Dot Notation	How you call a method on an object (e.g., object.method()).

Note: Important tip: Methods almost always have a first parameter called 'self'. This refers to the object itself and allows methods to access the object's attributes and other methods.

Understanding the difference and interaction between attributes and methods is essential for building effective object-oriented applications in Python.

Object-oriented programming (OOP) relies on two key ideas: attributes and methods. You will encounter these terms often when designing Python programs using classes and objects. Understanding how they differ, and how they work together, is crucial for building programs that are both useful and organised.

In Python, **attributes** are pieces of data stored inside an object. They help describe an object's state or characteristics. **Methods** are functions attached to an object. Methods describe the behaviours or actions that an object can perform. You can think of attributes as the 'nouns' (what an object is), and methods as the 'verbs' (what an object does).

Attributes and methods are both defined inside a class. When you create an object from a class, it gets its own copies of these attributes, and it can use the methods defined by the class. This separation of data (attributes) and logic (methods) helps keep your Python code flexible and easy to manage.

- Attributes store data about the object, like colour or age.
- Methods let the object do things, like display information or perform calculations.
- Changing an attribute can change how methods work, because methods often use attributes.
- Attributes hold values, while methods include code instructions.
- Both belong to objects, but they play different roles.

```
class Car:
    def __init__(self, colour, speed):
        self.colour = colour    # Attribute
        self.speed = speed     # Attribute

    def accelerate(self, amount): # Method
        self.speed += amount
        print(f"The car's new speed is {self.speed} km/h.")

# Creating a Car object
my_car = Car('red', 60)
print(my_car.colour)           # Accessing attribute
my_car.accelerate(10)          # Calling method
```

Aspect	Attribute	Method
Definition	Stores data	Performs an action
Accessed by	object.attribute	object.method()
Example	my_car.colour	my_car.accelerate(10)

Note: Important tip: You can change an object's attributes directly, but it's safer to use methods to update attributes. This keeps your data valid and your code easier to maintain.

CONSTRUCTORS (__INIT__)

A constructor in Python is a special method called `__init__` that initializes a new object's state.

In Python, when you create a new object from a class, there is sometimes a need to set its initial values. This setup is handled by a special method called a constructor. In Python, the constructor method is always named `__init__`. It is called automatically every time a new object is created from a class.

A constructor sets up the **initial state** for each new object of your class. This means that you can give your object variables (called attributes) values at creation time. This helps ensure your objects start off ready to use, with important information already in place.

The `__init__` method takes `self` as its first parameter. The `self` keyword refers to the object being created. You can also add more parameters if you need to pass in extra values when creating the object. By assigning these values to attributes inside the `__init__` method, each object can hold its own data.

- The constructor method in Python is named `__init__`.
- `__init__` runs automatically when you make a new object.
- You use `__init__` to assign values to an object's attributes.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
print(dog1.name)  # Output: Buddy
print(dog1.age)   # Output: 3
```

Term	Meaning
<code>__init__</code>	Special method that runs when an object is created
<code>self</code>	The object being created or operated on
attribute	A value held by an object, like a variable attached to it

Note: Important tip: If you do not define an `__init__` method, Python uses a default that does nothing. Define your own `__init__` if your objects need initial values or setup.

The `__init__` method is automatically called when an object is created from a class.

In Python, classes are used to create new types of objects. When you create an object from a class, Python runs a special method called `__init__`. This method helps set up the object with initial values and prepares it for use right after creation.

Whenever you see code that creates an object, such as `my_object = MyClass()`, Python will look for an `__init__` method in `MyClass`. If the method exists, Python runs it automatically to handle any set-up steps needed for the new object.

Think of `__init__` as a way to initialise your object's data or state right at the moment it is made. This is useful for giving each object its own settings that might differ from others of the same class.

- The `__init__` method is a part of every Python class, even if you do not write one yourself. Python uses a default one if it is missing.
- You can add parameters to `__init__` (besides `self`) to allow custom values when creating an object.
- Initialising attributes in `__init__` helps every object track its own information.

- The `__init__` method always takes at least one argument: `self`. This refers to the object being created.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

my_dog = Dog('Buddy', 5)
print(my_dog.name)  # Output: Buddy
print(my_dog.age)   # Output: 5
```

Action	Effect
Define a class with <code>__init__</code>	Allows setup when creating objects
Pass values to <code>__init__</code>	Customises each object's state
Omit <code>__init__</code>	Object created, but no custom initialisation

Note: Important tip: The double underscores in `__init__` mean it is a special method. Never call `__init__` directly. Instead, always create objects using the class name, and let Python handle `__init__` for you.

You use `__init__` to set up initial values for attributes in the new object.

In Python, object-oriented programming lets you define your own types, known as classes. When you create a new object using a class, you often want to set up certain information as soon as the object is created. This is where the `__init__` method comes in.

The `__init__` method, pronounced 'dunder init', stands for 'initialise'. It is a special function that runs automatically every time you create a new object from a class. You use this method to provide initial values for the object's attributes.

An attribute is a value that is stored inside an object. For example, if you create a class representing a car, you might want to keep track of the car's colour, make, or year. Using `__init__`, you can make sure every new car object starts with the correct details straight away.

- The `__init__` method always has at least one parameter called `self`.
- Other parameters allow you to pass in values to set the object's attributes.
- Attributes are usually set by writing `self.attribute_name = value` inside `__init__`.
- Every time you create an object, `__init__` is called without you needing to do anything extra.
- You can set default values for attributes if no value is provided.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

dog1 = Dog('Fido', 'Border Collie')
print(dog1.name)    # Outputs: Fido
print(dog1.breed)   # Outputs: Border Collie
```

Part	What it does
self	Refers to the object being created. Needed to store data in the new object.
name, breed	Parameters sent when making a Dog. Let you set values when the object is created.
self.name = name	Stores the dog's name inside the object.
__init__	Runs automatically when you make a new Dog object.

Note: Important tip: The `__init__` method is not required, but without it your objects will not be able to collect starting details easily. Using `__init__` helps keep your code organised and clear.

The first parameter of `__init__` must be 'self', which refers to the instance being created.

When you define a class in Python, you can include a special method called a constructor. In Python, this constructor is written as `__init__`. The `__init__` method runs automatically when you create a new object from the class. One key rule in Python is that the first parameter of the `__init__` method must be called 'self'.

The word **self** refers to the particular object (or instance) that you are creating. When you initialise an object, 'self' allows the method to access or set attributes on that specific object.

To help explain, imagine you are making a blueprint (a class) for building vehicles. When you actually build (instantiate) a vehicle, you need to keep track of its features, such as colour or model. 'self' points to the specific vehicle being built, so you can store details on it.

- 'self' must always be the first parameter in `__init__`.
- It enables you to refer to the object's attributes (like name, age, or colour).
- 'self' is a convention, not a Python keyword, but it is required for correct behaviour.


```
class Vehicle:
    def __init__(self, colour, model):
        self.colour = colour
        self.model = model
```

Part	Explanation
self	Refers to the current instance (object) being created.
colour	A parameter you pass in when creating the object.
model	Another parameter for the object.
self.colour = colour	Stores the input value on the object's 'colour' property.

Note: Do not omit 'self' or use another name for it. Always use 'self' as the first parameter in your `__init__` method for clarity and consistency in Python code.

Constructors help ensure that each object starts with the correct data and configuration.

In Python, constructors play a crucial role in controlling how objects are created. When you make a new object from a class, you often want it to start with some specific data or setup. That is the job of the constructor. The constructor makes sure each new object begins with the right information and settings, so your program behaves as you expect.

A constructor is a special method called `__init__` in Python. It runs automatically every time you create a new instance of a class. You use it to pass in data, check values, or do any setup that needs to happen for your object to work properly.

Imagine you are building a class to represent a person. You want each person to have a name and an age when they are created. If you use a constructor, you ensure that these pieces of information are set up correctly every time you make a new person object. This avoids errors later on, such as missing data or incorrect values.

- Constructors assign initial values to the object's attributes.
- They can check or adjust data before storing it.
- Every new object calls the constructor automatically.
- They help prevent errors from missing or invalid data.
- You can set sensible default values if needed.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
# Creating new Person objects
a = Person('James', 30)
b = Person('Amira', 25)
```

Object	Initial State
a	name = 'James', age = 30
b	name = 'Amira', age = 25

Note: Important tip: Always use constructors to initialise an object's essential attributes. This helps keep your objects reliable and easy to use.

BASIC INHERITANCE

Inheritance allows you to create new classes based on existing ones, promoting code reuse and organisation.

Inheritance is a key concept in object-oriented programming. It enables you to build new classes by extending existing ones, which helps you avoid repeating code and keeps your programs organised. When a new class is created from another, the new class automatically gets all the properties and behaviours of the original.

The class you start with is called the **parent (or base) class**. The new class that inherits from it is known as the **child (or derived) class**. This relationship allows the child class to reuse code and add its own features.

Python makes inheritance straightforward. If you have a class that does most of what you want, but you need to tweak or extend a few things, you can create a child class. This child keeps all the features of the parent and lets you add or override behaviours as required.

- Reduces code duplication by reusing common functionality
- Simplifies maintenance since changes to the parent class are reflected in child classes
- Helps group related classes together, improving program structure
- Lets you add or change features by extending inherited behaviour

```
class Animal:
    def speak(self):
        print("Animal sounds")

class Dog(Animal):
    def speak(self):
```

```
print("Woof!")

my_dog = Dog()
my_dog.speak() # Outputs: Woof!
```

Term	Description
Parent class	The existing class that provides features to be inherited
Child class	The new class that inherits features from the parent
Override	Replacing a method in the child class with a new version

Note: Important tip: A child class can use everything from its parent unless you override it. This lets you add specific behaviours, while still keeping all the abilities of the parent class.

The 'child' or derived class inherits attributes and methods from the 'parent' or base class.

Inheritance is a key principle in object-oriented programming. In Python, a class can be created based on another class. The new class is known as the child or derived class, while the class it inherits from is called the parent or base class.

When you create a child class, it automatically receives (or **inherits**) all of the public attributes and methods from the parent class. This means you can use the features of the parent class in your child class without rewriting code.

This makes your code more efficient and organised. You can build on existing code and only add or change features for specific needs. For example, you might have a parent class called `Animal`, and create a child class called `Dog`. The `Dog` class will inherit everything from `Animal`, then add things unique to dogs.

- The child class is defined by placing the parent class name in parentheses.
- Attributes and methods from the parent are available in the child by default.
- You can add extra attributes or methods to the child class.
- You can override methods to change how they behave in the child class.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} barks.")
```

```
d = Dog('Fido')
d.speak() # Output: Fido barks.
```

Term	Meaning
Parent/Base Class	The original class that other classes inherit from (e.g., Animal)
Child/Derived Class	The new class that inherits from the parent (e.g., Dog)
Inheritance	The process where a class takes on properties and behaviour from another class

Note: Important tip: Inheritance lets you reuse code and helps to keep your program DRY (Don't Repeat Yourself). Only use inheritance if the classes are logically related.

Use the syntax `class ChildClass(ParentClass):` to define a class that inherits from another class.

Inheritance is an important concept in object-oriented programming. It allows you to create a new class by reusing code from an existing class. In Python, you do this by defining a new class and specifying the name of the class you want to inherit from in parentheses.

To create a class that inherits from another, use the following syntax: **`class ChildClass(ParentClass):`**
 The new class (the child) gains the attributes and behaviour of the existing class (the parent).

Inheritance means the child class automatically gets all the methods and attributes of the parent class. You can also add new methods or override existing ones in the child class.

- The child class is also called a subclass.
- The parent class is also called a base or superclass.
- Inheritance helps you write less code by reusing existing functionality.
- You can add extra features or change behaviour in the child class.

```
class Animal:
    def speak(self):
        print("This animal makes a sound.")

class Dog(Animal):
    def speak(self):
        print("Woof!")

my_dog = Dog()
my_dog.speak() # Output: Woof!
```

Term	Meaning
ParentClass	The class being inherited from (also called base or superclass)
ChildClass	The class inheriting from the parent (also called subclass)
Inheritance	Feature that allows a new class to use code from an existing class

Note: Always put the parent class name in parentheses when defining your child class. A child class can inherit from more than one parent class, but for now, focus on single inheritance to keep things simple.

Inherited methods and attributes can be extended or overridden in the child class as needed.

In object-oriented programming (OOP), inheritance allows you to create a new class (child) based on an existing class (parent). The child class inherits the parent's methods and attributes. However, you are not restricted to using them exactly as they are. You can add new features, change how existing features work, or completely replace them in the child class. This is called extending or overriding.

In Python, **extending** means adding extra methods or attributes to the child class, building on top of what it already inherits. **Overriding** is when you provide a new version of a method or attribute in the child, giving it a different behaviour from the parent. Both are useful tools to adapt and specialise code as your application grows.

Let's look at a simple example. Suppose you have a parent class called `Animal` with a method called `speak`. If you create a child class called `Dog`, you can override `speak` to say something specific for dogs. You can also extend `Dog` by adding new attributes or methods, like `breed` or `fetch`.

- Extending allows you to add new features to a child class.
- Overriding lets you change inherited behaviour to suit your needs.
- The child class can access both its own features and those inherited from the parent.
- Overriding is common when a general action needs to be made more specific.
- You use the same method name to override but provide a new version in the child.

```
class Animal:
    def speak(self):
        print('The animal makes a sound.')

class Dog(Animal):
    def __init__(self, breed):
        self.breed = breed    # Extending: add a new attribute
    def speak(self):          # Overriding: change parent method
        print('The dog barks.')
    def fetch(self):           # Extending: new method
        print('The dog fetches the ball.')
```

```
my_dog = Dog('Border Collie')
my_dog.speak()    # Output: The dog barks.
my_dog.fetch()   # Output: The dog fetches the ball.
```

Term	What it means
Inheritance	The child class receives attributes and methods from the parent.
Extending	The child adds its own new attributes or methods.
Overriding	The child provides a new version of a method or attribute, replacing the parent's version.

Note: Important tip: You can still access the parent's method from the child using the `super()` function. This is useful if you want to build on the parent's behaviour, not just replace it.

Inheritance is helpful for situations where multiple classes share common functionality, allowing you to avoid code duplication.

Inheritance is a key part of object-oriented programming. It allows you to define a new class based on an existing class. The new class, called a child or subclass, can use the features of the existing class, known as the parent or base class. This makes it easy to share code between classes and avoid repeating yourself in similar parts of your program.

Imagine you are writing a program for a car hire company. You need to represent different types of vehicles, such as cars and trucks. Both types will need to store details like make, model, and year. Both will also need to display their information. **Instead of writing the same code for each class separately, you can use inheritance.** This helps keep your code tidy and makes changes easier later on. Using inheritance, you create a parent class that contains the shared attributes and behaviours. The child classes then inherit these and add any features that are unique to their type. There is no need to write the same code twice. This is also called 'code reuse'.

- Inheritance saves time by letting you reuse existing code.
- It helps reduce errors that come from copying code in more than one place.
- Maintaining your program is easier because changes to shared behaviour only need to be made once in the parent class.
- You can create more specialised classes based on a general blueprint.

```
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
```

```

        self.year = year

    def display_info(self):
        print(f"{self.year} {self.make} {self.model}")

class Car(Vehicle):
    def __init__(self, make, model, year, num_doors):
        super().__init__(make, model, year)
        self.num_doors = num_doors

class Truck(Vehicle):
    def __init__(self, make, model, year, load_capacity):
        super().__init__(make, model, year)
        self.load_capacity = load_capacity

```

Feature	Inheritance Benefit
Shared features (attributes & methods)	Write once in parent class; all children receive them
Special features	Add only to relevant subclasses, keeping code organised
Bug fixes/updates	Change in parent automatically updates all children

Note: Important tip: Try to use inheritance when classes are closely related and truly share behaviour—not just because they happen to be similar. Too much or incorrect inheritance can make your code harder to follow.

WHEN TO USE OOP

OOP is useful for organising code around real-world entities and their behaviours, making programs easier to understand and maintain.

Object-Oriented Programming (OOP) is a way of structuring your code so that it closely reflects the real world. In OOP, you group related data and actions together into objects. Each object represents something, like a person, order, or car, and contains both the data about that thing and the functions for working with it.

Imagine you are writing a program for a simple library system. Instead of keeping separate lists for book titles, authors, and whether the book is on loan, with OOP you would create a **Book** class. This class can hold information about each book and have methods for borrowing and returning books. This helps you organise your code around real-world entities and their behaviours.

This approach makes programs much easier to understand. When you read well-written object-oriented code, you can often guess what classes and objects represent just by their names. Maintenance tasks, such as fixing bugs or adding new features, become more straightforward, because the structure of your code follows clear, logical groupings.

- OOP allows you to model real-world items like users, vehicles, or products directly in code.
- Related functionality and data are kept together, so you don't have to manage everything in one big block.
- If you need to make changes, you usually only need to update one class or a small part of your program.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.on_loan = False

    def borrow(self):
        self.on_loan = True
        print(f"{self.title} has been borrowed.")

    def return_book(self):
        self.on_loan = False
        print(f"{self.title} has been returned.")

book1 = Book("1984", "George Orwell")
book1.borrow()
book1.return_book()
```

OOP Concept	Real-World Example
Class	Recipe, Blueprint, or Instruction Set
Object	Actual Cake, Built House, or Individual Car
Method	Action you can do with the object, like drive a car or borrow a book

Note: Important tip: Use OOP when your problem involves many similar items that have their own data and actions. OOP is especially helpful for larger programs, or when you want to expand your software over time.

Use OOP when your project involves complex data structures that benefit from encapsulation and abstraction.

Object-oriented programming (OOP) is a powerful approach to designing software. It helps you organise complex projects by grouping data and behaviour into logical units. You should consider OOP when your project involves complicated data structures that need to be managed in a consistent and maintainable way.

OOP provides two key concepts: **encapsulation** and **abstraction**. Encapsulation means bundling data and methods together in a class, while abstraction lets you hide the internal workings and only expose what is necessary.

When working with simple scripts or small tasks, you might not need OOP. However, as your project grows in size or complexity, OOP becomes very useful. It provides a way to break problems into smaller, more understandable pieces. This makes your code easier to test, maintain, and scale.

- You have multiple related data items with shared behaviour (for example, a car, a bike, and a bus all being types of vehicles).
- Your program manages entities with many properties and actions.
- You need to minimise how much of your program's data is accessible to the user or other parts of the code.
- You'd like to update or improve parts of your system without changing everything else.
- You want clear, reusable code that models real-world objects or systems.

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.speed = 0

    def accelerate(self, increment):
        self.speed += increment
        print(f"Accelerating... Current speed: {self.speed} km/h")

    def brake(self):
        self.speed = 0
        print("Car stopped.")

# Using the Car class
my_car = Car('Toyota', 'Corolla', 2021)
my_car.accelerate(20)
my_car.brake()
```

Concept	Benefit
Encapsulation	Keeps data and methods together, improving code safety.
Abstraction	Hides unneeded details, showing a simpler interface.
Inheritance	Allows sharing and extension of code between related classes.

Note: Important tip: If your project only performs a few simple tasks, OOP might add unnecessary complexity. Use it when you see the benefits of grouping data and behaviour, especially with complex data.

Apply OOP principles to enable code reuse through inheritance and modularity.

Object-Oriented Programming, or OOP, is a programming paradigm that focuses on organising code using ideas based on real-world objects. By using classes and objects, you can create reusable and maintainable code structures. One of the main benefits of OOP is the ability to reuse code through inheritance and modularity. This reduces duplication and helps you build flexible programs.

Inheritance is a key feature of OOP that allows you to create a new class based on an existing class. The new class inherits the attributes and methods of the original. **This means you can add or override features, without rewriting common code.** Inheritance encourages code sharing, making your programs easier to expand and maintain over time.

Modularity in OOP refers to structuring your code into smaller, independent parts (typically classes or modules). Each part focuses on a specific piece of functionality. This approach makes your code base organised, easier to understand, and simpler to test or debug.

- Inheritance enables you to define general behaviour once, then specialise it for different scenarios.
- Modular code lets you isolate functions and responsibilities in your program, making them reusable and interchangeable.
- OOP helps avoid repeating yourself—if your program uses the same logic in multiple places, inheritance and modularity can centralise and simplify the code.

```
class Animal:
    def speak(self):
        print("This animal makes a sound.")

class Dog(Animal):
    def speak(self):
        print("Woof!")
```

```
class Cat(Animal):
    def speak(self):
        print("Meow!")

# Reuse Animal's structure; only change what is needed for each animal
dog = Dog()
cat = Cat()
dog.speak()    # Outputs: Woof!
cat.speak()    # Outputs: Meow!
```

Concept	Description
Inheritance	Share behaviour from a base class with derived classes.
Modularity	Split code into parts that can be reused and maintained separately.
Code Reuse	Write once, use in multiple places—reducing errors and saving time.

Note: Important tip: When you notice repeated code or logic, consider creating a class that both shares the repeated elements and lets you specialise when necessary. This improves your program's reliability and makes it much easier to add new features later.

Choose OOP when you need to represent relationships between different objects, such as parent-child hierarchies.

Object-oriented programming (OOP) is powerful when you need to model and manage relationships between different types of objects. This is especially useful if the objects have common features and you want to organise your code around real-world relationships. OOP allows you to structure your programs by grouping data and behaviour together, making your code clearer and easier to manage.

When you work with problems involving **hierarchies or connected groups of things**, OOP lets you represent these relationships naturally. For example, in a family tree, employees in a company, or types of vehicles, using the parent-child structure helps you organise shared and unique features effectively.

A parent-child hierarchy is a common relationship in software design. You start with a base object (the 'parent'), then create specialised versions ('children') that inherit features from the parent but also have their own unique features. This makes code easier to update and extend, allowing you to apply changes in one place and affect multiple related objects.

- OOP is ideal for modelling real-world systems like animals, vehicles, organisations, or products.
- Hierarchical relationships let you reuse code, rather than writing it repeatedly.
- It makes maintenance easier because changes to shared features update child objects too.
- You reduce errors by centralising common logic and behaviours.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print('Some sound')

class Dog(Animal):
    def speak(self):
        print('Woof!')

class Cat(Animal):
    def speak(self):
        print('Meow!')

# Usage
pet = Dog('Ruby')
pet.speak()    # Output: Woof!
```

Object	Relationship
Animal	Parent (Base class)
Dog	Child (inherits from Animal)
Cat	Child (inherits from Animal)

Note: Important tip: Choose OOP when you expect your program to grow, require sharing code between similar objects, or need to easily add new types that fit in a hierarchy. OOP helps keep your code neat and understandable as complexity increases.

OOP is especially helpful in large projects where collaboration and scalability are important.

Object-Oriented Programming (OOP) is a design approach in programming that groups code into logical units: objects and classes. While OOP can be used in small scripts, it becomes especially valuable in large codebases, particularly where many people work together or where the software must grow and change over time.

When you build something complex, such as a web application or enterprise software, you might have thousands of lines of code. **OOP helps organise this code so teams can work together without**

confusion. This is crucial for collaboration and keeping the project robust as it scales or new features are added.

In collaborative environments, each developer or team may work on different parts of a program at the same time. OOP divides these parts into classes, making it easier to assign and track work. Because classes are self-contained, changes in one area are less likely to break code elsewhere. This isolation supports teamwork and reduces errors.

- Encapsulates related code and data together, reducing accidental changes.
- Makes it easier for new team members to understand parts of a project.
- Supports code reuse through inheritance, saving effort.
- Allows for testing and updating small modules without risk to the whole program.
- Facilitates adding new features without rewriting existing code.

```
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def start_engine(self):
        print(f"{self.make} {self.model} engine started.")

car = Vehicle('Toyota', 'Corolla')
car.start_engine()
```

Feature	OOP Benefit
Collaboration	Teams work on different classes without interference
Maintenance	Easier to debug and update specific parts
Scalability	Supports larger projects by organising code logically

Note: If your project is small or simple (like a short script), OOP may be more complex than necessary. Use OOP when your codebase grows, your team expands, or when requirements often change.

To go further, faster. To accelerate change.

Thank you for participating in the Python Foundations course. We hope you found the material informative and engaging. If you need further assistance, please don't hesitate to reach out to our Strategic Defence Team:

John Rutherford: john.rutherford@lumifygroup.com

Strategic Account Manager

Rick Bykerk: ricky.bykerk@lumifygroup.com

Strategic Account Manager

www.lumifygroup.com



LUMIFY
learn. work. people.